

Reactive navigation in partially familiar planar environments using semantic perceptual feedback

The International Journal of
Robotics Research
2022, Vol. 41(1) 85–126
© The Author(s) 2021
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/02783649211048931
journals.sagepub.com/home/ijr



Vasileios Vasilopoulos¹, Georgios Pavlakos², Karl Schmeckpeper³,
Kostas Daniilidis³ and Daniel E. Koditschek¹

Abstract

This article solves the planar navigation problem by recourse to an online reactive scheme that exploits recent advances in simultaneous localization and mapping (SLAM) and visual object recognition to recast prior geometric knowledge in terms of an offline catalog of familiar objects. The resulting vector field planner guarantees convergence to an arbitrarily specified goal, avoiding collisions along the way with fixed but arbitrarily placed instances from the catalog as well as completely unknown fixed obstacles so long as they are strongly convex and well separated. We illustrate the generic robustness properties of such deterministic reactive planners as well as the relatively modest computational cost of this algorithm by supplementing an extensive numerical study with physical implementation on both a wheeled and legged platform in different settings.

Keywords

Reactive motion planning, collision avoidance, vision and sensor-based control

1. Introduction

This article advances the formally demonstrable capabilities of online reactive motion planners by disentangling the topology of navigation from the geometry of perception. Specifically, we appeal to a semantically aware perceptual oracle for the instantaneous recognition and localization of previously memorized objects and abstract away their geometric details in realtime as they are encountered, yielding a sequence of topologically representative motion planning problems solvable by recourse to purely reactive online methods. Recent advances in simultaneous localization and mapping (SLAM) and visual object recognition afford a working empirical realization of this provably correct scheme in both quasi-static and highly dynamic physical planar robots.

1.1. Motivation and prior work

Even as legged (Ilhan et al., 2018; Johnson et al., 2011; Wooden et al., 2010) and aerial (Amazon, Inc., 2014; Gao et al., 2019; Mohta et al., 2018; Tang and Kumar, 2018) robots engage increasingly realistic, unstructured environments, intuition suggests that prior experience ought to yield deterministic navigation guarantees, postponing

statistical predictions of performance to estimated (Trautman et al., 2015), learned (Henry et al., 2010), or simulated (Karaman and Frazzoli, 2012) characterizations of truly bewilderingly dense or moving environments. Similarly, sampling-based methods, motivated by the typically high-dimensional configuration spaces arising from combined task and motion planning (Garrett et al., 2018), can achieve asymptotic optimality (Vega-Brown and Roy, 2018), but no guarantee of convergence (or task completion) under partial prior knowledge or limited sampling, and their probabilistic completeness guarantees can be slow to be realized in practice when confronting settings with narrow passages (Noreen et al., 2016), even in 2D

¹Department of Electrical and Systems Engineering, University of Pennsylvania, Philadelphia, PA, USA

²Department of Electrical Engineering and Computer Sciences, UC Berkeley, Berkeley, CA, USA

³Department of Computer and Information Science, University of Pennsylvania, Philadelphia, PA, USA

Corresponding author:

Vasileios Vasilopoulos, Department of Electrical and Systems Engineering, School of Engineering and Applied Science, University of Pennsylvania, 220 South 33rd Street, 201 Moore Building, Philadelphia, PA 19104, USA.

Email: vvasilo@seas.upenn.edu

environments. More importantly, our recent parallel work (Vasilopoulos et al., 2020), that uses the reactive planning principles presented in this article, shows that existing state-of-the-art path replanning algorithms for unknown 2D environments (Otte and Frazzoli, 2015) can cycle repeatedly in the presence of both unforeseen obstacles and narrow passages as they search for alternative openings, before eventually (and after protracted cycling) reporting failure (incorrectly) and halting.

1.1.1. Reactive navigation. Heretofore, deterministically safe, convergent reactive methods have required substantial prior knowledge of a static environment, whether encoded using navigation functions (Filippidis and Kyriakopoulos, 2012; Lionis et al., 2008; Rimon and Koditschek, 1992), harmonic potential functions (Conner et al., 2011; Vlantis et al., 2018), or pre-computed sequences of “funnels” (Majumdar and Tedrake, 2017). In contrast, sensor-driven planners in this general tradition (Borenstein and Koren, 1989, 1991; Brock and Khatib, 1999; Fiorini and Shiller, 1998; Johnson et al., 2011; Khatib, 1986; Paranjape et al., 2015; Simmons, 1996; van den Berg et al., 2011, 2008) have guaranteed collision avoidance but have offered no assurance of convergence to a designated goal.

Recent advances in the theory of sensor-based navigation (Arslan and Koditschek, 2016a,b, 2017) relying on the properties of metric projections on convex sets (Kuntz and Scholtes, 1994) (and other parallel approaches (Ataka et al., 2018; Chang and Marsden, 2003; Ilhan et al., 2018; Paternain et al., 2017)) add the key feature of guaranteed convergence to a designated goal, by trading away prior knowledge for the presumption of simplicity: unknown obstacles can be successfully negotiated in realtime without losing global convergence guarantees if they are “round” (i.e., very strongly convex in a sense made precise in Arslan and Koditschek (2018)).

However, this presumption, along with the additional requirement for enough separation between the obstacles in the workspace, limit the domain of application for such methods to geometrically simple environments and might prohibit successful navigation in complicated, unstructured environments with non-convex geometry. Hence, other reactive approaches either seek to appropriately modify the input reference signal to account for unanticipated (potentially non-convex) disturbances (Revzen et al., 2012), or rely on stochastic frameworks that are empirically shown to improve performance with non-convex obstacles (Reverdy et al., 2015), with no guarantees of convergence.

1.1.2. Realtime perception. In this work, we address these shortcomings by appeal to an agent’s memory evoked by execution-time perceptual cues. Recent advances in semantic SLAM (Atanasov et al., 2016; Bowman et al., 2017) and object pose extraction using convolutional neural net architectures (Kar et al., 2015; Kong et al., 2017; Pavlakos et al., 2017) now provide an avenue for systematically composing

partial prior knowledge about the robot’s workspace within a deterministic framework well suited to the vector field planning methods reviewed above.

Contrasting recent work has recruited end-to-end learning to achieve obstacle avoiding reactions within semantically labeled representations of familiar environments (Gupta et al., 2017), or supplemented such deep-learned representations with reference paths (Kumar et al., 2018), or optimally generated waypoint sequences (Bansal et al., 2019) that guide the robot to its destination. Although such approaches cannot guarantee safe convergence to the robot’s destination, they promote the importance of landmark-based navigation, already highlighted by parallel work in biology (Jayakumar et al., 2019). However, characteristically, the input to such architectures is raw visual data thereby generating egocentric reactions that are hostage to the experience of one particular environment. In contrast, our compositional use of semantically tagged, learned-object recognizers affords systematic re-use across many different environments and achieves formal deterministic guarantees as well, at least up to their (admittedly still far from formally justifiable) idealization as perfect realtime perceptual oracles.

1.1.3. Topologically informed navigation. Work on the topology of motion planning (Farber, 2006; Farber et al., 2019) has overtaken earlier investigation of reactive (i.e., vector field) navigation planners (Rimon, 1990; Rimon and Koditschek, 1989) to the point that, comparatively, only preliminary results on their intrinsic limitations have been reported (Baryshnikov and Shapiro, 2014). It seems clear that our success in achieving such strong results for a broad class of partially known environments is due to the simplicity of the problem class (punctured 2D manifolds have the homotopy type of a bouquet of circles), but we are not in a position to opine firmly on the likely limitations of this approach in higher-dimensional settings.

Recently, several contributions have focused on either finding invariants for homology classes to facilitate optimal path search in known environments (Bhattacharya et al., 2015), exploiting data to enforce topological constraints (Pokorný and Kragic, 2015), or conceptualizing sensor measurements related to the shape of an object in a topologically meaningful way using persistent homology (Mueller and Birk, 2018). In contrast, we extract geometric and topological information about the robot’s workspace at execution time in order to construct a map between a geometrically complicated mapped space and a (topologically equivalent but geometrically simple) model space that can be used for planning purposes. To this end, we employ methods from the field of computational geometry for implicit description of geometric shape using R-functions (Shapiro, 2007), convex decomposition (Keil, 1985), and logic operations with polygons (Clementini et al., 1993; Douglas and Peucker, 1973; Egenhofer and Herring, 1991).

1.2. Summary of contributions

We consider the navigation problem in a 2D workspace cluttered with unknown convex obstacles, along with “familiar” non-convex obstacles that belong to classes of known geometries, but whose number and placement are *a priori* unknown. We assume a limited-range onboard sensor and a catalog of known obstacles, along with a “mapping oracle” for their online identification and localization in the physical workspace. This framework allows the robot to explore the geometry and topology of its workspace in realtime as it navigates toward its goal, by recognizing and incorporating in its stored semantic map “familiar” obstacles, whose number and placement are otherwise unknown, awaiting discovery at execution time.

Based on the aforementioned description, we propose a representation of the environment taking the form of a “multi-layer” collection of topological spaces whose realtime interaction can be exploited to integrate the geometrically naive sensor-driven methods of Arslan and Koditschek (2016b) with the offline geometrically sensitive methods of Rimon and Koditschek (1992).

Specifically, we adapt the construction of Rimon and Koditschek (1989) to generate a realtime smooth change of coordinates (a *diffeomorphism*) of the mapped space of the environment into a (locally) topologically equivalent but geometrically more favorable model space, relative to which the sensor-based reactive methods of Arslan and Koditschek (2016b) can be applied directly. We prove that the conjugate vector field defined by appropriately transforming the reactive model space back through this diffeomorphism induces a vector field on the robot’s physical configuration space that inherits the same formal guarantees of obstacle avoidance and convergence. As the robot’s knowledge about the geometry and topology of its workspace at execution time is constantly updated, we adopt a hybrid dynamical systems description of our navigation framework, and show that the resulting hybrid system both inherits the consistency properties outlined in Johnson et al. (2016) and safely drives the robot to the goal without violating given command limits.

We extend the construction to the case of a differential drive robot, by pulling back the extended field over planar rigid transformations introduced for this purpose in Arslan and Koditschek (2016b) through a suitable polar coordinate transformation of the tangent lift of our original planar diffeomorphism and demonstrate, once again, that the physical differential drive robot inherits the same obstacle avoidance and convergence properties as those guaranteed for the geometrically simple model robot (Arslan and Koditschek, 2016b).

We believe that this is the first *doubly reactive controller* (Arslan and Koditschek, 2016b) (i.e., a navigation framework wherein not only the robot’s trajectory but also the control vector field that generates it are computed online at

execution time) that can handle arbitrary polygonal shapes in realtime without the need for specific separation assumptions between the familiar obstacles, by combining perception and object recognition for the familiar obstacles with local range measurements (e.g., LIDAR) for the unknown obstacles, to yield provably correct navigation in geometrically complicated environments. Furthermore, unlike Rapidly-exploring Random Trees (RRT)-based (LaValle and Kuffner, 2001) or Probabilistic RoadMap (PRM)-based (Kavraki et al., 1996) algorithms, and similarly to other vector-field based approaches, our framework is capable of solving the overall “kinodynamic” problem online, instead of executing separate trajectory and motion planning, for both a fully actuated particle and a differential drive robot.

Finally, by coupling the semantic SLAM framework of (Bowman et al., 2017) and the object detection pipeline of (Pavlakos et al., 2017) with our reactive planning architecture, we are able to localize against isolated semantic cues while navigating, instead of localizing against entire scenes (Gupta et al., 2017) or visual geometric features (Hesch et al., 2014). Therefore, by training just on data from the objects the robot is expected to encounter, we introduce modularity and robustness in our approach, while simultaneously performing online planning that does not rely on specific features of a deep network architecture (e.g., number or type of layers) (Bansal et al., 2019; Gupta et al., 2017; Kumar et al., 2018).

1.3. Organization of the article

The article is organized as follows. Section 2 describes the problem and establishes our assumptions. Section 3 describes the physical, semantic, mapped, and model planning spaces (summarized in Figure 1) used in the diffeomorphism construction between the mapped and model spaces, whose properties are established next in Section 4. Section 5 provides the formal hybrid systems description framework and the correctness proofs for both a fully actuated (Theorem 3) and differential drive (Theorem 4) velocity-controlled planar robot, comprising the central theoretical contribution of this article.

Based on these results, Section 6 continues with a description of the implemented mapped space recovery and reactive planning algorithms, for both a fully actuated and a differential drive robot, shown in Figure 2(d) and (e). Section 7 presents a variety of illustrative numerical studies, and Section 8 continues with a brief description of the experimental setup, realizing the deployed perception (relying on prior work and shown in Figure 2(a) and (c)) and motion planning (Figure 2(d) and (e)) algorithms on both the Turtlebot (TurtleBot2, 2019) and the Minitaur (Ghost Robotics, 2016) robot. Section 9 continues with our experimental results, and Section 10 concludes by summarizing our findings and sketching some of the

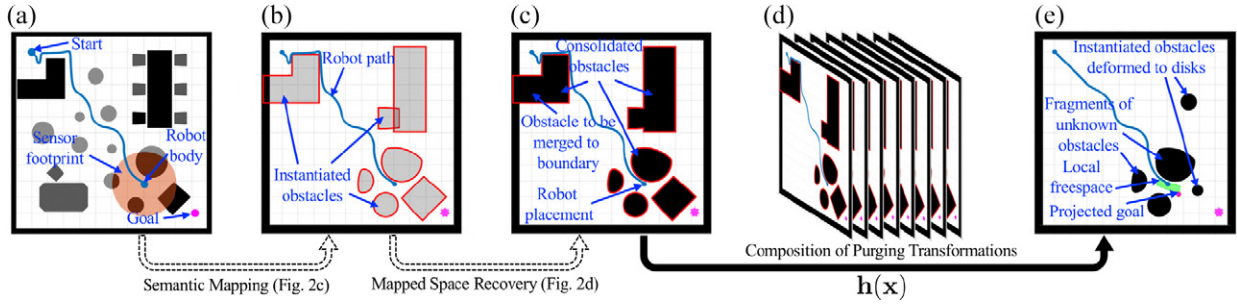


Fig. 1. Snapshot illustration of key realtime computation and associated models: the robot moves in the physical space (a) (Section 3.1), depicted as the blue trace of its centroid, toward a goal (pink) discovering along the way (black) both familiar objects of known geometry but unknown location (dark gray) and unknown obstacles (light gray), with an onboard sensor of limited range (orange disk). These obstacles are localized, dilated, and stored permanently in the semantic space (b) (Section 3.2) if they have familiar geometry, or temporarily, with just the corresponding sensed fragments, if they are unknown. The consolidated obstacles (resolved in realtime from the unions of overlapping localized familiar obstacles), along with the sensed fragments of the unknown obstacles, are then stored in the mapped space (c) (Section 3.3). A nonlinear change of coordinates, $h(x)$, into a topologically equivalent but geometrically simplified model space (e) (Section 3.4, depicting the robot's placement and prior trajectory amongst the h -deformed convex images of the mapped obstacles) is computed instantaneously each time a new perceptual event instantiates more obstacles to be localized in the semantic space, thus redefining the mapped space. The map, h , is a diffeomorphism, computed via composition of “purging” transformations between intermediate spaces (d) (Section 4.2) that abstract the consolidated localized polygonal obstacles by successively pruning away their geometric details to yield topologically equivalent disks. A doubly reactive control scheme for convex environments (Arslan and Koditschek, 2016b) defines a vector field on the model space which is transformed in realtime through the diffeomorphism to generate the input in the physical space (Section 5).

future work now in progress building on these results. Finally, the appendix provides details of computational methods and proofs.

2. Problem formulation

Similarly to Vasilopoulos and Koditschek (2018), we consider a disk-shaped robot with radius $r > 0$, centered at $x \in \mathbb{R}^2$, navigating a closed, compact, polygonal, potentially non-convex workspace $\mathcal{W} \subset \mathbb{R}^2$, with known outer boundary $\partial\mathcal{W}$, towards a target location $x_d \in \mathcal{W}$. The robot is assumed to possess a sensor with fixed range R , capable of recognizing “familiar” objects, as well as estimating the distance of the robot to nearby obstacles.¹ We also define the *enclosing workspace*, as the convex hull of the closure of the workspace $\mathcal{W}$:

$$\mathcal{W}_e := \{x \in \mathbb{R}^2 \mid x \in \text{Conv}(\overline{\mathcal{W}})\} \quad (1)$$

The workspace is cluttered by a finite, unknown number of fixed, disjoint obstacles, denoted by $\tilde{\mathcal{O}} := \{\tilde{\mathcal{O}}_1, \tilde{\mathcal{O}}_2, \dots\}$. By convention, the set $\tilde{\mathcal{O}}$ also includes potentially non-convex “intrusions” of the boundary of the physical workspace $\mathcal{W}$ into the enclosing workspace $\mathcal{W}_e$, that can be described as the connected components of $\mathcal{W}_e \setminus \mathcal{W}$. We use the notation in Arslan and Koditschek (2016b) and define the *freespace* as

$$\mathcal{F} := \left\{ x \in \mathcal{W}_e \mid \overline{B(x, r)} \subseteq \mathcal{W}_e \setminus \bigcup_i \tilde{\mathcal{O}}_i \right\} \quad (2)$$

where $B(x, r)$ is the open ball centered at x with radius r , and $\overline{B(x, r)}$ denotes its closure. Similarly to the enclosing workspace, $\mathcal{W}_e$, we define the *enclosing freespace*, $\mathcal{F}_e$ as

$$\mathcal{F}_e := \{x \in \mathbb{R}^2 \mid x \in \text{Conv}(\overline{\mathcal{F}})\} \quad (3)$$

Although none of the positions of any obstacles in $\tilde{\mathcal{O}}$ are *a priori* known, a subset $\tilde{\mathcal{P}} := \{\tilde{P}_i\}_{i \in \mathcal{N}_P} \subseteq \tilde{\mathcal{O}}$ of these obstacles, indexed by $\mathcal{N}_P := \{1, \dots, N_P\} \subset \mathbb{N}$, is assumed to be “familiar” in the sense of having an *a priori* known, readily recognizable, potentially non-convex, polygonal geometry (i.e., belonging to a known catalog of *geometry classes*), which the robot can identify and localize instantaneously from online sensory measurement, as described in Section 8. We require that this subset also includes all connected components of $\mathcal{W}_e \setminus \mathcal{W}$. The remaining obstacles in $\tilde{\mathcal{C}} := \tilde{\mathcal{O}} \setminus \tilde{\mathcal{P}}$, indexed by $\mathcal{N}_C := \{1, \dots, N_C\} \subset \mathbb{N}$ are assumed to be strictly convex but are in all other regards (location and specific shape) completely unknown to the robot, while nevertheless satisfying a curvature condition given in Arslan and Koditschek (2016b: Assumption 2). This condition is interpreted in Arslan and Koditschek (2016b) as the requirement for each convex obstacle $\tilde{C} \in \tilde{\mathcal{C}}$ to be sufficiently “round”, because mere obstacle convexity with flat edges can still generate undesired local minima when using a greedy reactive controller.

To simplify our notation, we neglect the robot dimensions, by dilating each obstacle in $\tilde{\mathcal{O}}$ by r , and assume that the robot operates in $\mathcal{F}$. We denote the set of dilated obstacles derived from $\tilde{\mathcal{O}}$, $\tilde{\mathcal{P}}$ and $\tilde{\mathcal{C}}$, by $\mathcal{O}$, $\mathcal{P}$ and $\mathcal{C}$, respectively. As obstacles in $\tilde{\mathcal{P}}$ are polygonal, and dilations of polygonal

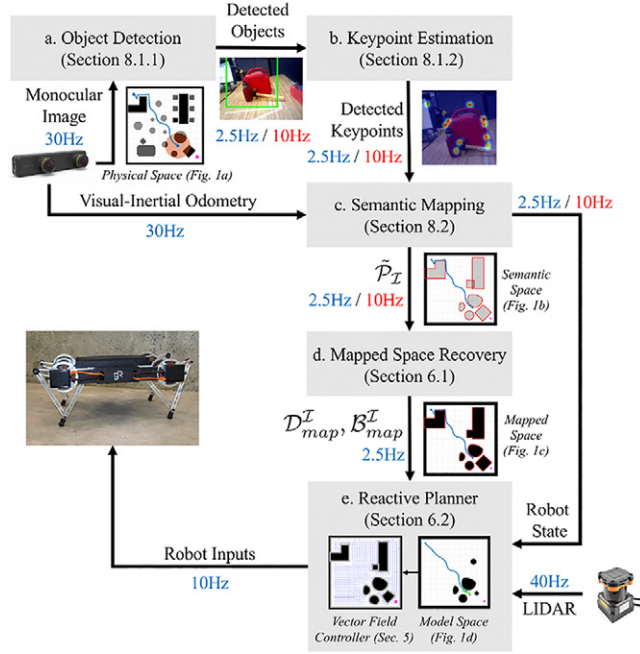


Fig. 2. A summary of our online reactive planning architecture. Using the camera image, two separate neural network architectures (configured in serial and run either onboard at 2.5 Hz, or offboard at 10 Hz) (a) detect familiar obstacles (Redmon and Farhadi, 2018) (Section 8.1.1) and (b) localize corresponding semantic keypoints (Pavlakos et al., 2017) (Section 8.1.2). (c) The keypoint locations on the image and an egomotion estimate provided by visual inertial odometry are used by the semantic mapping module (Bowman et al., 2017) (Section 8.2) to provide updated robot ($\mathbf{x}$) and obstacle poses ($\tilde{\mathcal{P}}_{\mathcal{I}}$) on the plane. (d) The mapped space tracking algorithm (Section 6.1, Algorithm 1), run onboard at 2.5 Hz, uses $\tilde{\mathcal{P}}_{\mathcal{I}}$ to generate the list of obstacles in the mapped space $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$. (e) The reactive planning module (Section 6.2, Algorithm 2), run onboard at 10 Hz, uses $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$, along with LIDAR data for unknown obstacles, to provide the robot inputs and close the control loop.

obstacles are not in general polygonal, we approximate obstacles in $\mathcal{P}$ with conservative polygonal supersets. Note that because the set $\tilde{\mathcal{P}}$ is required to contain all connected components of $\mathcal{W}_e \setminus \mathcal{W}$, that describe non-convex “intrusions” of the boundary of the physical workspace $\mathcal{W}$ into the enclosing workspace $\mathcal{W}_e$, the set $\mathcal{P}$ is similarly required to contain the dilations of these intrusions. For obstacles in $\mathcal{C}$ we require the following separation assumptions, introduced in Arslan and Koditschek (2016b).

Assumption 1. Each obstacle $C_i \in \mathcal{C}$ has a positive clearance² $d(C_i, C_j) > 0$ from any obstacle $C_j \in \mathcal{C}$, with $i \neq j$, and a positive clearance $d(C_i, \partial\mathcal{F}) > 0$ from the boundary of the freespace $\mathcal{F}$.

Then, similarly to (Rimon and Koditschek, 1992), we describe each polygonal obstacle $P_i \in \mathcal{P} \subseteq \mathcal{O}$ by an *obstacle function*, a real-valued map providing an implicit representation of the form

$$P_i = \{\mathbf{x} \in \mathbb{R}^2 | \beta_i(\mathbf{x}) \leq 0\} \quad (4)$$

that the robot can construct online from the cataloged geometry after it has localized P_i , as detailed in Appendix B. We also require the following technical assumption.

Assumption 2. For each $P_i \in \mathcal{P}$, there exists $\epsilon_i > 0$ such that the set $S_{\beta_i} := \{\mathbf{x} | \beta_i(\mathbf{x}) \leq \epsilon_i\}$ has a positive clearance $d(S_{\beta_i}, \mathcal{C}) > 0$ from any obstacle $C \in \mathcal{C}$.

Note that Assumptions 1 and 2 constrain the shape (convex) and placements (sufficiently separated) only of obstacles that have never previously been encountered. Familiar (polygonal, dilated by r) obstacles $P_i \in \mathcal{P}$, while fixed, can be placed completely arbitrarily with no further prior information: in particular, they can overlap unrestrictedly, with no jeopardy to our formal results, because we rely on the sensor oracle to recognize and locate them in realtime. Obstacles in $\mathcal{P}$ are similarly allowed to overlap with the boundary of the enclosing freespace $\partial\mathcal{F}_e$. To control the scope of the present article, we simply assume that a path to the goal always exists, i.e., the robot operates in a non-adversarial environment.

Assumption 3. The freespace $\mathcal{F}$ is path-connected.

Finally, in Section 5.2, we impose the technical Assumption 4 precluding the possibility that any of the (topologically unavoidable) unstable saddle points of our control law coincide with a cataloged “knot point” of any familiar obstacle (a condition that we conjecture should be generic in the configuration space of obstacle placements).

Table 1. Key symbols used throughout this article, associated with the problem formulation in Section 2. See also Table 2 for notation associated with the environment representation in Section 3, Table 3 for notation associated with the diffeomorphism construction in Section 4, and Table 4 for notation associated with our reactive controller in Section 5.

$\mathcal{W} \subset \mathbb{R}^2$	Closed, compact, polygonal, potentially non-convex workspace
$\mathcal{W}_e \subset \mathbb{R}^2$	Enclosing workspace (1)
$\mathcal{F} \subset \mathcal{W}$	Freespace (2)
$\mathcal{F}_e \subset \mathcal{W}_e$	Enclosing freespace (3)
$r \in \mathbb{R}$	Robot radius
$R \in \mathbb{R}$	Sensor range
$\mathbf{x}_d \in \mathcal{F}$	Goal location
$\tilde{\mathcal{O}} := \{\tilde{\mathcal{O}}_1, \tilde{\mathcal{O}}_2, \dots\} \subseteq \mathbb{R}^2$	Set of fixed, disjoint obstacles
$\tilde{\mathcal{P}} := \{\tilde{P}_i\}_{i \in \mathcal{N}_p} \subseteq \tilde{\mathcal{O}}$	Set of “familiar,” polygonal obstacles, indexed by the set $\mathcal{N}_p := \{1, \dots, N_p\} \subset \mathbb{N}$
$\tilde{\mathcal{C}} := \tilde{\mathcal{O}} \setminus \tilde{\mathcal{P}} = \{\tilde{C}_i\}_{i \in \mathcal{N}_c}$	Set of completely unknown obstacles, indexed by the set $\mathcal{N}_c := \{1, \dots, N_c\} \subset \mathbb{N}$
$\mathcal{O}, \mathcal{P}, \mathcal{C}$	Set of obstacles in $\tilde{\mathcal{O}}, \tilde{\mathcal{P}}, \tilde{\mathcal{C}}$ respectively, dilated by the robot radius, r

Table 2. Key symbols related to the environment representation in Section 3.

$\tilde{\mathcal{P}}_{\mathcal{I}} := \{\tilde{P}_i\}_{i \in \mathcal{I}} \subseteq \tilde{\mathcal{P}}$	Set of (constantly updated) instantiated familiar polygonal obstacles, Section 3.1
$\mathcal{I} \subseteq \mathcal{N}_p$	Index set of the $ \mathcal{I} $ presently instantiated obstacles in $\tilde{\mathcal{P}}_{\mathcal{I}}$, Section 3.1
$\mathcal{F}_{sem}^{\mathcal{I}}$	Semantic space corresponding to $\mathcal{I} \in 2^{\mathcal{N}_p}$, Section 3.2
$\mathcal{F}_{map}^{\mathcal{I}}$	Mapped space corresponding to $\mathcal{I} \in 2^{\mathcal{N}_p}$, Section 3.3
$\mathcal{F}_{model}^{\mathcal{I}}$	Model space corresponding to $\mathcal{I} \in 2^{\mathcal{N}_p}$, Section 3.4
$\mathcal{P}_{sem}^{\mathcal{I}} := \coprod_{i \in \mathcal{I}} P_i$	Set of familiar, polygonal obstacles instantiated in the semantic space, Section 3.2
$\mathcal{C}_{sem} := \{C_i\}_{i \in \mathcal{J}_c} \subseteq \mathcal{C}$	Set of unknown obstacles in the semantic space, indexed by $\mathcal{J}_c \subseteq \mathcal{N}_c$, Section 3.2
$\mathcal{P}_{map}^{\mathcal{I}} := \bigcup_{i \in \mathcal{I}} P_i = \{P_i\}_{i \in \mathcal{J}^{\mathcal{I}}}$	Set of consolidated familiar obstacles in the mapped space, indexed by $\mathcal{J}^{\mathcal{I}}$, Section 3.3
$\mathcal{C}_{map} := \mathcal{C}_{sem}$	Set of unknown obstacles in the mapped space, indexed by $\mathcal{J}_c \subseteq \mathcal{N}_c$, Section 3.3
$\mathcal{B}_{map}^{\mathcal{I}} := \{B_i\}_{i \in \mathcal{J}_B^{\mathcal{I}}}$	Connected components of $\mathcal{P}_{map}^{\mathcal{I}}$ to be merged into $\partial \mathcal{F}_e$, indexed by $\mathcal{J}_B^{\mathcal{I}}$, Section 3.3
$\mathcal{D}_{map}^{\mathcal{I}} := \{D_i\}_{i \in \mathcal{J}_D^{\mathcal{I}}}$	Connected components of $\mathcal{P}_{map}^{\mathcal{I}}$ to be deformed into disks, indexed by $\mathcal{J}_D^{\mathcal{I}}$, Section 3.3
$\mathbf{x}_i^* \in \mathbb{R}^2, i \in \mathcal{J}_D^{\mathcal{I}}$	Centers of the $ \mathcal{J}_D^{\mathcal{I}} $ disks in the model space, Section 3.4
$\rho_i \in \mathbb{R}, i \in \mathcal{J}_D^{\mathcal{I}}$	Radii of the $ \mathcal{J}_D^{\mathcal{I}} $ disks in the model space, Section 3.4

Based on these assumptions and further positing first-order, fully actuated robot dynamics $\dot{\mathbf{x}} = \mathbf{u}(\mathbf{x})$, the problem consists of finding a Lipschitz continuous controller $\mathbf{u} : \mathcal{F} \rightarrow \mathbb{R}^2$, that leaves the freespace $\mathcal{F}$ positively invariant and asymptotically steers almost all configurations in $\mathcal{F}$ to the given goal $\mathbf{x}_d \in \mathcal{F}$. We have also summarized key symbols used throughout this article in Table 1.

3. Navigational representation of the environment

In this section, we introduce associated notation for the four distinct representations of the environment that we will refer to as *planning spaces* and use in the construction of our algorithm. Figure 1 illustrates the role of these spaces and the transformations that relate them in constructing and analyzing a realtime generated vector field that guarantees safe passage to the goal. The notation related to the environment representation in this section is listed in Table 2. The new technical contribution is an adaptation of the methods of Rimon and Koditschek (1989) to the realtime

construction of a diffeomorphism, $\mathbf{h}$, where the requirement for fast, online performance demands an algorithm that is as simple as possible and with few tunable parameters. Hence, because the reactive controller in Arslan and Koditschek (2016b) is designed to (provably) handle convex shapes, sensed obstacles not recognized by the semantic SLAM process are simply assumed to be convex (implemented by designing $\mathbf{h}$ to resolve to the identity transformation in the neighborhood of “unfamiliar” objects) and the control response defaults to that prior construction.

3.1. Physical space

The *physical space* is a complete description of the geometry of the unknown actual world and while inaccessible to the robot is used for purposes of analysis. It describes the enclosing workspace $\mathcal{W}_e$, punctured with the obstacles $\tilde{\mathcal{O}}$. This gives rise to the freespace $\mathcal{F}$, given in (2), consisting of all placements of the robot’s centroid that entail no intersections of its body with any interior obstacles or intrusions from the boundary. The robot navigates this space toward

the goal, discovering and localizing new obstacles along the way. Those discovered obstacles which are not convex are (by assumption) “familiar” and are then “instantiated” (recalled, and registered from memory to populate the accumulating record of discovery in the semantic space) as we next discuss. Those which are “unfamiliar” are presumed convex and registered as such in the companion spaces next to be presented.³

We denote by $\tilde{\mathcal{P}}_{\mathcal{I}} := \{\tilde{P}_i\}_{i \in \mathcal{I}} \subseteq \tilde{\mathcal{P}}$ the finite set of (constantly updated) physically “instantiated” familiar objects, indexed by $\mathcal{I} \subseteq \mathcal{N}_{\mathcal{P}}$, that drives the construction of the semantic, mapped and model spaces described next. As explained in Section 5.1, such elements $\mathcal{I}$ of the power set $2^{\mathcal{N}_{\mathcal{P}}}$ also index the modes of our hybrid system.

3.2. Semantic space

The *semantic space* $\mathcal{F}_{\text{sem}}^{\mathcal{I}}$ records the robot’s evolving information about the environment aggregated from the raw sensor data about the observable portions of a subset of unrecognized (and, therefore, presumed convex) obstacles from $\mathcal{C}$, together with the polygonal boundaries of the $|\mathcal{I}|$ familiar obstacles, that are instantiated at the moment the sensory data triggers the identification and localization a familiar obstacle.

Definition 1. A familiar obstacle $\tilde{P} \in \tilde{\mathcal{P}}$ is considered to be “instantiated,” if it has been sensed using the robot’s sensor of fixed range R , recognized, localized, and its dilation $P \in \mathcal{P}$ is permanently included in the semantic space.⁴

We denote the set of unrecognized obstacles in the semantic space by $\mathcal{C}_{\text{sem}} := \{C_i\}_{i \in \mathcal{J}_{\mathcal{C}}}$ indexed by $\mathcal{J}_{\mathcal{C}} \subseteq \mathcal{N}_{\mathcal{C}}$, and the set of familiar obstacles in the semantic space by $\mathcal{P}_{\text{sem}}^{\mathcal{I}} := \prod_{i \in \mathcal{I}} P_i$.

It is important to note that this environment is constantly updated, both by discovering and storing new familiar obstacles in the semantic map and by discarding old information and storing new information regarding obstacles in $\mathcal{C}$.³ Here, the robot is treated as a point particle, since all obstacles are dilated by r in the passage from the workspace to the freespace representation of valid placements.

3.3. Mapped space

Although the semantic space contains all the relevant geometric information (identity and pose) about the obstacles the robot has encountered, it does not explicitly contain any topological information about the explored environment, as represented by the disjoint union operation in the definition of $\mathcal{P}_{\text{sem}}^{\mathcal{I}}$. This is because Assumption 2 does not exclude overlaps between obstacles in $\mathcal{P}$. Their algorithmically effective consolidation in realtime reduces the number while increasing the geometric complexity of the actual freespace obstacles the robot must negotiate along the way to the goal. To do so, we need therefore to take unions of overlapping obstacles in $\mathcal{P}_{\text{sem}}^{\mathcal{I}}$, making up $\mathcal{P}_{\text{map}}^{\mathcal{I}} := \bigcup_{i \in \mathcal{I}} P_i = \{P_i\}_{i \in \mathcal{J}^{\mathcal{I}}}$ (i.e., a new set of

consolidated familiar obstacles indexed by $\mathcal{J}^{\mathcal{I}}$ with $|\mathcal{J}^{\mathcal{I}}| \leq |\mathcal{I}|$), as well as copies of the sensed fragments of unknown obstacles from $\mathcal{C}_{\text{sem}}$ (i.e., $\mathcal{C}_{\text{map}} := \mathcal{C}_{\text{sem}}$) to form the *mapped space* $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ as

$$\mathcal{F}_{\text{map}}^{\mathcal{I}} := \mathcal{F}_e \setminus (\mathcal{P}_{\text{map}}^{\mathcal{I}} \cup \mathcal{C}_{\text{map}}) \quad (5)$$

Note that, by Assumption 2, the convex obstacles are assumed to be far enough away from the familiar obstacles, such that no overlap occurs in the above union.

Next, we focus on the connected components of $\mathcal{P}_{\text{map}}^{\mathcal{I}}$; because Assumption 2 allows overlaps between obstacles in $\mathcal{P}$ and the boundary of the enclosing freespace $\partial \mathcal{F}_e$, for any connected component P of $\mathcal{P}_{\text{map}}^{\mathcal{I}}$ such that $P \cap \partial \mathcal{F}_e \neq \emptyset$, we take $B := P \cap \mathcal{F}_e$ and include B in a new set $\mathcal{B}_{\text{map}}^{\mathcal{I}}$, indexed by $\mathcal{J}_{\mathcal{B}}^{\mathcal{I}}$. The rest of the connected components in $\mathcal{P}_{\text{map}}^{\mathcal{I}}$, which do not intersect $\partial \mathcal{F}_e$, are included in a separate set $\mathcal{D}_{\text{map}}^{\mathcal{I}}$, indexed by $\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}$. The idea here is that obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$ should be merged to the boundary of the enclosing freespace $\partial \mathcal{F}_e$, and obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ should be deformed to disks, because $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ need to be diffeomorphic.

3.4. Model space

The *model space* $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ has the same boundary as $\mathcal{F}_e$ (i.e., $\partial \mathcal{F}_{\text{model}}^{\mathcal{I}} := \partial \mathcal{F}_e$) and consists of copies of the sensed fragments of the $|\mathcal{J}_{\mathcal{C}}|$ unrecognized visible convex obstacles in $\mathcal{C}_{\text{map}}$, and a collection of $|\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}|$ Euclidean disks corresponding to the $|\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}|$ consolidated obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ that are deformed to disks. The centers $\{\mathbf{x}_i^*\}_{i \in \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}}$ and radii $\{\rho_i\}_{i \in \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}}$ of the $|\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}|$ disks are chosen so that $\overline{\mathbf{B}(\mathbf{x}_i^*, \rho_i)}$ is contained in the interior of $D_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, as required in Rimón and Koditschek (1989). The obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$ are merged into $\partial \mathcal{F}_e$, to make $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ topologically equivalent, through a map $\mathbf{h}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}}$. We can, therefore, write $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ as

$$\mathcal{F}_{\text{model}}^{\mathcal{I}} = \mathcal{F}_e \setminus \left(\bigcup_{i \in \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}} \overline{\mathbf{B}(\mathbf{x}_i^*, \rho_i)} \cup \mathcal{C}_{\text{map}} \right) \quad (6)$$

Note here that we need to distinguish between familiar but unanticipated obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{B}_{\text{map}}^{\mathcal{I}}$, and completely unknown convex obstacles in $\mathcal{C}_{\text{map}}$, because our algorithm handles them quite differently. From a practical point of view, the robot instantiates familiar obstacles using its full sensor suite equipped with a learned catalog, while completely unknown obstacles that the robot can only partially observe are handled solely based on distance measurements (e.g., LIDAR). More importantly, from a formal point of view, this distinction is needed in the definition of our global diffeomorphism (to be presented next in Section 4), because only the familiar obstacles affect its construction. Moreover, the previously unseen unfamiliar obstacles

cannot be simply registered in the model space, but they must be formally mapped there through that diffeomorphism, which is constructed so that it defaults to the identity transform on their boundaries.

3.5. Implicit representation of obstacles

We note here that the construction of the map $\mathbf{h}^I$ between the mapped space $\mathcal{F}_{\text{map}}^I$ and the model space $\mathcal{F}_{\text{model}}^I$, described next in Section 4 and shown in Figure 1, relies on the existence of smooth implicit functions $\beta : \mathbb{R}^2 \rightarrow \mathbb{R}$ for polygons, constructed such that $\beta(\mathbf{x}) = 0$ implies that $\mathbf{x}$ lies on the boundary of the polygon. Although the construction of such functions is a separate problem on its own, here we derive implicit representations using so-called “R-functions” from the constructive solid geometry literature (Shapiro, 2007). In Appendix B, we describe the method used for the construction of such implicit functions for polygonal obstacles that have the desired property of being analytic everywhere except for the polygon vertices. We have already successfully implemented such constructions in a similar setting for star-shaped obstacles in Vasilopoulos and Koditschek (2018).

4. The diffeomorphism between the mapped and model spaces

In this section, we describe our method of constructing the diffeomorphism, $\mathbf{h}^I$, between the mapped space $\mathcal{F}_{\text{map}}^I$ and

the model space $\mathcal{F}_{\text{model}}^I$. We assume that the robot has already recognized, localized, and stored the $|\mathcal{J}^I|$ consolidated familiar polygonal obstacles in $\mathcal{P}_{\text{map}}^I$ in its map, and has subsequently identified obstacles to be merged to the boundary of the enclosing freespace $\partial\mathcal{F}_e$, stored in $\mathcal{B}_{\text{map}}^I$, and obstacles to be deformed to disks, stored in $\mathcal{D}_{\text{map}}^I$.

The idea is then to compose a sequence of “purging” diffeomorphisms, which coincide with the identity map except on a small “collar” around each component of $\mathcal{P}_{\text{map}}^I$, that produce successively less-complicated isolated shapes. The final simplified shapes are then conveniently deformed into a disk if the corresponding obstacle belongs in $\mathcal{D}_{\text{map}}^I$, or into the boundary of $\mathcal{F}_e$ if the corresponding obstacle belongs in $\mathcal{B}_{\text{map}}^I$, in order to generate the model space $\mathcal{F}_{\text{model}}^I$. Before describing these transformations, we first provide some background on the used obstacle representation methodology in Section 4.1, and provide associated notation in Table 3.

4.1. Obstacle representation

In order to construct the map $\mathbf{h}^I$ between the mapped space and the model space, we assume that the robot has access to the triangulation of each one of the obstacles stored in both $\mathcal{D}_{\text{map}}^I$ and $\mathcal{B}_{\text{map}}^I$. This triangulation can be efficiently constructed online upon recognition of each obstacle, using the *Two Ears Theorem* (Meisters, 1975), and the associated *Ear Clipping Method* (ElGindy et al., 1993).⁶

Table 3. Key symbols related to the diffeomorphism construction from $\mathcal{F}_{\text{map}}^I$ to $\mathcal{F}_{\text{model}}^I$, described in Section 4.

$\mathcal{T}_{P_i}$	Tree of triangles, constructed from the dual graph of the triangulation of P_i , Section 4.1
$\mathcal{V}_{P_i}$	Set of vertices of $\mathcal{T}_{P_i}$, identified with triangles in the triangulation of P_i , Section 4.1
$\mathcal{E}_{P_i}$	Set of edges of $\mathcal{T}_{P_i}$ encoding triangle adjacency in the triangulation of P_i , Section 4.1
$\mathcal{F}_{\text{map},j_i}^I \subset \mathbb{R}^2$	Mapped space before the purging of leaf triangle $j_i \in \mathcal{V}_{P_i}$, Section 4.2
$\mathcal{F}_{\text{map},p(j_i)}^I \subset \mathbb{R}^2$	Mapped space after the purging of leaf triangle $j_i \in \mathcal{V}_{P_i}$, Section 4.2
$\hat{\mathcal{F}}_{\text{map}}^I \subset \mathbb{R}^2$	Mapped space after the purging of all leaf triangles, Sections 4.2 and 4.3
$\mathbf{x}_{j_i}^* \in \mathbb{R}^2$	Admissible center for the purging transformation of a leaf triangle $j_i \in \mathcal{V}_{P_i}$, Definition 2
$\mathbf{x}_{r_i}^* \in \mathbb{R}^2$	Admissible center for the transformation of a root triangle $r_i \in \mathcal{V}_{P_i}$, Definitions 4 and 6
$\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$	Vertices of a leaf triangle $j_i \in \mathcal{V}_{P_i}$, Section 4.2
$\mathbf{x}_{1r_i}, \mathbf{x}_{2r_i}, \mathbf{x}_{3r_i}$	Vertices of a root triangle $r_i \in \mathcal{V}_{P_i}$, Section 4.3
$\mathcal{Q}_{j_i} \subset \mathbb{R}^2$	Quadrilateral $\mathbf{x}_{3j_i}\mathbf{x}_{1j_i}\mathbf{x}_{j_i}^*\mathbf{x}_{2j_i}\mathbf{x}_{3j_i}$ associated with a leaf triangle $j_i \in \mathcal{V}_{P_i}$, Section 4.2
$\mathcal{Q}_{r_i} \subset \mathbb{R}^2$	Quadrilateral $\mathbf{x}_{3r_i}\mathbf{x}_{1r_i}\mathbf{x}_{r_i}^*\mathbf{x}_{2r_i}\mathbf{x}_{3r_i}$ associated with a root triangle $r_i \in \mathcal{V}_{P_i}$, Section 4.3
$\overline{\mathcal{Q}}_{j_i} \subset \mathbb{R}^2$	Admissible polygonal collar associated with a leaf triangle $j_i \in \mathcal{V}_{P_i}$, Definition 3
$\overline{\mathcal{Q}}_{r_i} \subset \mathbb{R}^2$	Admissible polygonal collar associated with a root triangle $r_i \in \mathcal{V}_{P_i}$, Definitions 5 and 7
$\gamma_{j_i}, \gamma_{r_i} : \mathbb{R}^2 \rightarrow \mathbb{R}$	Implicit function associated with $\mathcal{Q}_{j_i}$ (7) or $\mathcal{Q}_{r_i}$ (17)
$\delta_{j_i}, \delta_{r_i} : \mathbb{R}^2 \rightarrow \mathbb{R}$	Implicit function associated with $\overline{\mathcal{Q}}_{j_i}$ (8) or $\overline{\mathcal{Q}}_{r_i}$ (18)
$\sigma_{\gamma_{j_i}}, \sigma_{\gamma_{r_i}} : \mathbb{R}^2 \rightarrow [0, 1]$	Auxiliary C^∞ switch associated with $\mathcal{Q}_{j_i}$ (11) or $\mathcal{Q}_{r_i}$ (19)
$\sigma_{\delta_{j_i}}, \sigma_{\delta_{r_i}} : \mathbb{R}^2 \rightarrow [0, 1]$	Auxiliary C^∞ switch associated with $\overline{\mathcal{Q}}_{j_i}$ (12) or $\overline{\mathcal{Q}}_{r_i}$ (20)
$\sigma_{j_i}, \sigma_{r_i} : \mathbb{R}^2 \rightarrow [0, 1]$	C^∞ switch of the transformation of a leaf triangle j_i (13) or a root triangle r_i (21)
$\nu_{j_i}, \nu_{r_i} : \mathbb{R}^2 \rightarrow [0, 1]$	Deforming factor for a leaf triangle j_i (14) or a root triangle r_i (22) and (23)
$\mathbf{h}_{j_i}^I : \mathcal{F}_{\text{map},j_i}^I \rightarrow \mathcal{F}_{\text{map},p(j_i)}^I$	Purging transformation mapping a leaf triangle j_i to its parent $p(j_i)$ (16)
$\mathbf{g}^I : \mathcal{F}_{\text{map}}^I \rightarrow \hat{\mathcal{F}}_{\text{map}}^I$	Composition of purging transformations mapping $\mathcal{F}_{\text{map}}^I$ to $\hat{\mathcal{F}}_{\text{map}}^I$, Section 4.2.6
$\hat{\mathbf{h}}^I : \hat{\mathcal{F}}_{\text{map}}^I \rightarrow \mathcal{F}_{\text{model}}^I$	Diffeomorphism between $\hat{\mathcal{F}}_{\text{map}}^I$ and $\mathcal{F}_{\text{model}}^I$ (26)
$\mathbf{h}^I : \mathcal{F}_{\text{map}}^I \rightarrow \mathcal{F}_{\text{model}}^I$	Diffeomorphism between $\mathcal{F}_{\text{map}}^I$ and $\mathcal{F}_{\text{model}}^I$ (27)

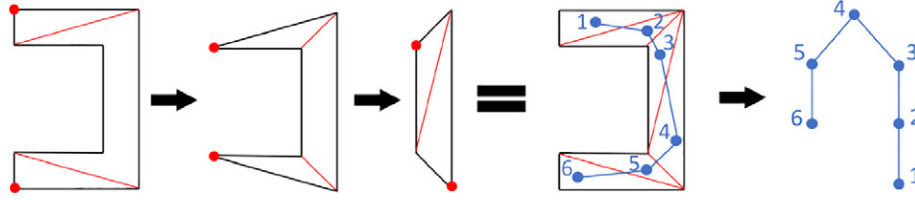


Fig. 3. Triangulation of a non-convex obstacle using the Ear Clipping Method. The original polygon is guaranteed to have at least two ears (red dots) by the Two Ears Theorem, which induce triangles that can be removed from the polygon. By repeating this process, we get the final triangulation and its dual graph, which is guaranteed to be a tree. This tree can be restructured by setting the root to be the triangle of maximal surface area, to yield the order of purging transformations in descending depth; in this particular example this order is $1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 5 \rightarrow 4$.

Briefly, an *ear* of a simple polygon is a vertex of the polygon such that the line segment between the two neighbors of the vertex lies entirely in the interior of the polygon. The Two Ears Theorem guarantees that every simple polygon has at least two such ears, and the Ear Clipping Method uses this result to efficiently construct polygon triangulations in $\mathcal{O}(n^2)$ time. Namely, an ear and its two neighbors form a triangle that is not crossed by any other part of the polygon and can, therefore, be safely removed. Removing a triangle of this type produces a polygon with one less vertex than the original polygon; we can repeat the process to eventually get a single triangle and complete the triangulation. An example is shown in Figure 3.

Except for its utility in constructing triangulations, the Two Ears Theorem guarantees that the dual graph of the triangulation of a simple polygon with no holes constructed with the Ear Clipping Method (i.e., a graph with one vertex per triangle and one edge per pair of adjacent triangles) is, in fact, a tree (O’Rourke, 1987).

Therefore, in order to construct a tree of triangles $\mathcal{T}_{P_i} := (\mathcal{V}_{P_i}, \mathcal{E}_{P_i})$ corresponding to a polygon P_i , with $\mathcal{V}_{P_i}$ a set of vertices identified with triangles (i.e., vertices of the dual of the formal triangulation) and $\mathcal{E}_{P_i}$ a set of edges encoding triangle adjacency, we can triangulate P_i using the Ear Clipping Method, pick any triangle as root, and construct $\mathcal{T}_{P_i}$ based on the adjacency properties induced by the dual graph of the triangulation, as shown in Figure 3. If $P_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, we pick as root the triangle with the largest surface area, whereas if $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$, we pick as root a triangle adjacent to $\partial\mathcal{F}_e$. This will give us a *tree of triangles* for P_i in a notion similar to Rimón and Koditschek (1992). Our goal is then to successively “purge” this tree, triangle by triangle, in order of descending depth, until we reach the root triangle. Then, we can use a diffeomorphism similar to Vasilopoulos and Koditschek (2018) to map the exterior and boundary of the root triangle onto the exterior and boundary of a topologically equivalent disk if $P_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, or merge the root triangle into $\partial\mathcal{F}_e$ if $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$. These operations are all performed online; we provide some computational performance metrics with our experiments in Section 9.

We describe the algorithm for each purging transformation of the leaf nodes in Section 4.2 and the (final) root triangle purging transformation in Section 4.3. Finally,

Section 4.4 defines the diffeomorphism between the mapped and model spaces, along with associated qualitative properties.

4.2. Intermediate spaces related by leaf purging transformations

In this section, we describe the purging transformation that maps the boundary of a leaf triangle $j_i \in \mathcal{V}_{P_i}$ onto the boundary of its parent $p(j_i) \in \mathcal{V}_{P_i}$, as shown in Figure 4(1a) and (2a). This gives rise to a composition of transformations between a succession of *intermediate spaces*, each including the triangle j_i , and $\mathcal{F}_{\text{map}, p(j_i)}^{\mathcal{I}}$, where j_i has been mapped onto the boundary of its parent. Each of these transformations is in principle similar and performing a role analogous to the corresponding purging transformation in Rimón and Koditschek (1992), with two important differences. First, it deforms space only “locally” around the triangle, without taking into consideration other triangles or polygons, affording better numerical stability because only one triangle is considered at a time. Second, the method presented in Rimón and Koditschek (1992) is limited to parent–child pairs that strongly overlap instead of just being adjacent, which makes the method impractical for the arbitrary polygonal shapes and meshes involved in this work.

4.2.1. Center of the transformation and surrounding polygonal collars. Let the vertices of the triangle $j_i \in \mathcal{V}_{P_i}$ be $\mathbf{x}_{1j_i}$, $\mathbf{x}_{2j_i}$, and $\mathbf{x}_{3j_i}$ in counterclockwise order, with $\mathbf{x}_{1j_i}\mathbf{x}_{2j_i}$ the common edge between j_i and $p(j_i)$.

Definition 2. An *admissible center* for the purging transformation of the leaf triangle $j_i \in \mathcal{V}_{P_i}$, denoted by $\mathbf{x}_{j_i}^*$, is a point along the triangle median from $\mathbf{x}_{3j_i}$ and in the interior of its parent $p(j_i) \in \mathcal{V}_{P_i}$.

Such a point is always possible to find, because the two triangles share a common edge. By picking an admissible center, we ensure that the quadrilateral $\mathcal{Q}_{j_i}$ with boundary $\mathbf{x}_{3j_i}\mathbf{x}_{1j_i}\mathbf{x}_{j_i}^*\mathbf{x}_{2j_i}$ is convex, as shown in Figure 4(1a) and (2a).

Definition 3. An *admissible polygonal collar* for the purging transformation of the leaf triangle j_i is a convex polygon $\overline{\mathcal{Q}}_{j_i}$ such that:

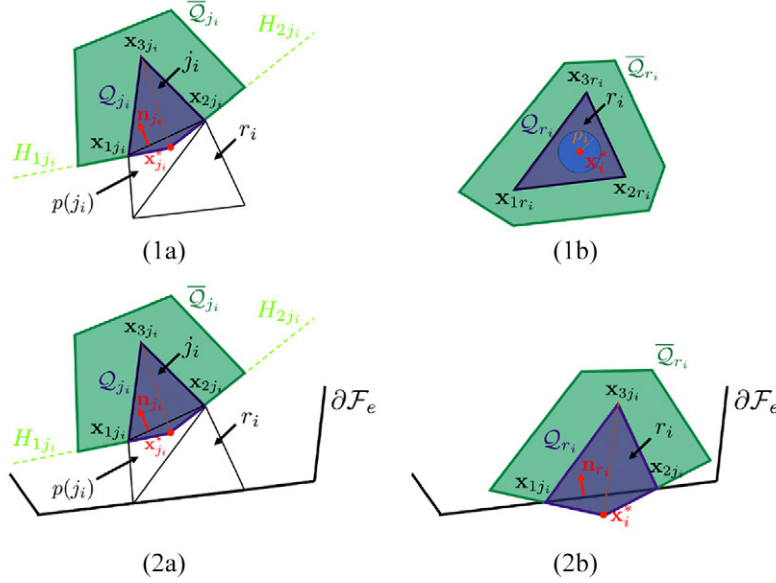


Fig. 4. Illustration of features used in the transformation of: (1a) a leaf triangle j_i onto its parent $p(j_i)$ and (1b) a root triangle r_i onto a disk centered at $\mathbf{x}_i^*$ with radius ρ_i for an obstacle in $\mathcal{D}_{\text{map}}^I$; (2a) a leaf triangle j_i onto its parent $p(j_i)$ and (2b) a root triangle r_i onto a disk centered at $\mathbf{x}_i^*$ with radius ρ_i for an obstacle in $\partial \mathcal{F}_e$.

1. the edges $\mathbf{x}_{1j_i}\mathbf{x}_{j_i}^*$ and $\mathbf{x}_{j_i}^*\mathbf{x}_{2j_i}$ are edges of $\overline{\mathcal{Q}}_{j_i}$;
2. $\overline{\mathcal{Q}}_{j_i}$ does not intersect the interior of any triangle $k \in \mathcal{V}_P$ with $k \neq j_i, k \neq p(j_i), \forall P$;
3. $\overline{\mathcal{Q}}_{j_i}$ does not intersect any obstacle $C \in \mathcal{C}_{\text{map}}$;
4. $\overline{\mathcal{Q}}_{j_i}$ does not intersect the boundary of $\mathcal{F}_{\text{map},j_i}^I$;
5. $\mathcal{Q}_{j_i} \subset \overline{\mathcal{Q}}_{j_i}$.

Examples of such polygons are shown in Figure 4(1a) and (2a). This polygon is responsible for limiting the effect of the purging transformation in its interior, while keeping its value equal to the identity everywhere else. Intuitively, the requirements in Definition 3 will limit the effect of the purging transformation in a region that encloses the triangle j_i and is away from the boundary of any other obstacle. The importance of enforcing the edges $\mathbf{x}_{1j_i}\mathbf{x}_{j_i}^*$ and $\mathbf{x}_{j_i}^*\mathbf{x}_{2j_i}$ to be edges of $\overline{\mathcal{Q}}_{j_i}$ will become evident in the construction of the provable properties of the diffeomorphism, summarized in Proposition 1. We provide more details about the construction of admissible collars in Appendix C.

For the following, we also construct implicit functions $\gamma_{j_i}(\mathbf{x})$ and $\delta_{j_i}(\mathbf{x})$ corresponding to the leaf triangle $j_i \in \mathcal{V}_{P_i}$, as described in Appendix B, such that

$$\mathcal{Q}_{j_i} = \{\mathbf{x} \in \mathbb{R}^2 \mid \gamma_{j_i}(\mathbf{x}) \leq 0\} \quad (7)$$

$$\overline{\mathcal{Q}}_{j_i} = \{\mathbf{x} \in \mathbb{R}^2 \mid \delta_{j_i}(\mathbf{x}) \geq 0\} \quad (8)$$

4.2.2. Description of the C^∞ switches. In order to simplify the diffeomorphism construction, we depart from the construction of analytic switches (Rimon and Koditschek, 1989) and rely instead on the C^∞ function $\zeta_\mu : \mathbb{R} \rightarrow \mathbb{R}$ (Hirsch, 1976) described by

$$\zeta_\mu(\chi) = \begin{cases} e^{-\mu/\chi}, & \chi > 0 \\ 0, & \chi \leq 0 \end{cases} \quad (9)$$

and parametrized by $\mu > 0$, that has derivative

$$\zeta_{\mu'}(\chi) = \begin{cases} \frac{\mu \zeta_\mu(\chi)}{\chi^2}, & \chi > 0 \\ 0, & \chi \leq 0 \end{cases} \quad (10)$$

Based on that function, we can then define the auxiliary C^∞ switches

$$\sigma_{\gamma_{j_i}}(\mathbf{x}) := \eta_{\mu_{\gamma_{j_i}}, \epsilon_{j_i}} \circ \gamma_{j_i}(\mathbf{x}) \quad (11)$$

$$\sigma_{\delta_{j_i}}(\mathbf{x}) := \zeta_{\mu_{\delta_{j_i}}} \circ \frac{\delta_{j_i}(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_{j_i}^*\|} \quad (12)$$

with $\eta_{\mu, \epsilon}(\chi) := \zeta_\mu(\epsilon - \chi)/\zeta_\mu(\epsilon)$, and $\mu_{\gamma_{j_i}}, \mu_{\delta_{j_i}}, \epsilon_{j_i} > 0$ tunable parameters. Note that $\sigma_{\gamma_{j_i}}$ is exactly equal to 1 on the boundary of $\mathcal{Q}_{j_i}$ and equal to 0 when $\gamma_{j_i}(\mathbf{x}) \geq \epsilon_{j_i}$, whereas $\sigma_{\delta_{j_i}}$ is 0 outside $\overline{\mathcal{Q}}_{j_i}$. The parameters $\mu_{\gamma_{j_i}}$ and $\mu_{\delta_{j_i}}$ are used to tune the “slope” of $\sigma_{\gamma_{j_i}}$ on the boundary of $\mathcal{Q}_{j_i}$ and how fast $\sigma_{\delta_{j_i}}$ approaches 1 in the interior of $\overline{\mathcal{Q}}_{j_i}$, respectively.

Based on the above, we define the C^∞ switch of the purging transformation for the leaf triangle $j_i \in \mathcal{V}_{P_i}$ as a function $\sigma_{j_i} : \mathcal{F}_{\text{map},j_i}^I \rightarrow \mathbb{R}$, defined by

$$\sigma_{j_i}(\mathbf{x}) := \begin{cases} \frac{\sigma_{\gamma_{j_i}}(\mathbf{x})\sigma_{\delta_{j_i}}(\mathbf{x})}{\sigma_{\gamma_{j_i}}(\mathbf{x})\sigma_{\delta_{j_i}}(\mathbf{x}) + (1 - \sigma_{\gamma_{j_i}}(\mathbf{x}))}, & \mathbf{x} \neq \mathbf{x}_{1j_i}, \mathbf{x}_{2j_i} \\ 1, & \mathbf{x} = \mathbf{x}_{1j_i}, \mathbf{x}_{2j_i} \end{cases} \quad (13)$$

In this way, we see that $\sigma_{j_i}(\mathbf{x}) = 0$ when $\sigma_{\gamma_{j_i}}(\mathbf{x}) = 0$ or $\sigma_{\delta_{j_i}}(\mathbf{x}) = 0$ (i.e., when $\gamma_{j_i}(\mathbf{x}) \geq \epsilon_{j_i}$ or outside $\overline{\mathcal{Q}}_{j_i}$), $\sigma_{j_i}(\mathbf{x}) = 1$ when $\sigma_{\gamma_{j_i}}(\mathbf{x}) = 1$ (i.e., on the boundary of $\mathcal{Q}_{j_i}$).

and σ_{j_i} varies between 0 and 1 everywhere, because $\sigma_{\gamma_{j_i}}$ and $\sigma_{\delta_{j_i}}$ also vary between 0 and 1. Based on Definitions 2 and 3, it is straightforward to show the following lemma.

Lemma 1. *The function $\sigma_{j_i} : \mathcal{F}_{\text{map},j_i}^{\mathcal{I}} \rightarrow \mathbb{R}$ is smooth away from the triangle vertices $\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$, none of which lies in the interior of $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$.* $\square$

Proof. See Appendix E.1.

4.2.3. Description of the deforming factors. The deforming factors are the functions $\nu_{j_i} : \mathcal{F}_{\text{map},j_i}^{\mathcal{I}} \rightarrow \mathbb{R}$, responsible for mapping the boundary of the leaf triangle $j_i \in \mathcal{V}_{P_i}$ onto the boundary of its parent $p(j_i)$. Based on Definitions 2 and 3 and as shown in Figure 4(1a) and (2a), this implies that the functions ν_{j_i} are responsible for mapping the polygonal chain $\mathbf{x}_{2j_i}\mathbf{x}_{3j_i}\mathbf{x}_{1j_i}$ onto the shared edge $\mathbf{x}_{2j_i}\mathbf{x}_{1j_i}$ between j_i and $p(j_i)$. For this reason, we construct ν_{j_i} as follows:

$$\nu_{j_i}(\mathbf{x}) := \frac{(\mathbf{x}_{1j_i} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i}}{(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i}} \quad (14)$$

with

$$\mathbf{n}_{j_i} := \mathbf{R}_{\frac{\pi}{2}} \frac{\mathbf{x}_{2j_i} - \mathbf{x}_{1j_i}}{\|\mathbf{x}_{2j_i} - \mathbf{x}_{1j_i}\|}, \quad \mathbf{R}_{\frac{\pi}{2}} := \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad (15)$$

the normal vector corresponding to the shared edge between j_i and $p(j_i)$.

4.2.4. The map between $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$ and $\mathcal{F}_{\text{map},p(j_i)}^{\mathcal{I}}$. Based on the above, we then construct the map between $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$ and $\mathcal{F}_{\text{map},p(j_i)}^{\mathcal{I}}$ with the j_i th leaf triangle of P_i purged, as

$$\mathbf{h}_{j_i}^{\mathcal{I}}(\mathbf{x}) := \sigma_{j_i}(\mathbf{x}) \left(\mathbf{x}_{j_i}^* + \nu_{j_i}(\mathbf{x})(\mathbf{x} - \mathbf{x}_{j_i}^*) \right) + (1 - \sigma_{j_i}(\mathbf{x}))\mathbf{x} \quad (16)$$

4.2.5. Qualitative properties of the map between $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$ and $\mathcal{F}_{\text{map},p(j_i)}^{\mathcal{I}}$. We first verify that the construction is a smooth change of coordinates between the intermediate mapped spaces.

Lemma 2. *The map $\mathbf{h}_{j_i}^{\mathcal{I}} : \mathcal{F}_{\text{map},j_i}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{map},p(j_i)}^{\mathcal{I}}$ is smooth away from the triangle vertices $\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$, none of which lies in the interior of $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$.* $\square$

Proof. See Appendix E.1.

Proposition 1. *The map $\mathbf{h}_{j_i}^{\mathcal{I}}$ is a C^∞ diffeomorphism between $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$ and $\mathcal{F}_{\text{map},p(j_i)}^{\mathcal{I}}$ away from the triangle vertices $\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$, none of which lies in the interior of $\mathcal{F}_{\text{map},j_i}^{\mathcal{I}}$.* $\square$

Proof. See Appendix E.1.

4.2.6. Composition of leaf purging transformations. The application of the purging transformation described above will result in a tree for P_i with one less vertex and one less edge. Therefore, similarly to Rimón and Koditschek (1992), we can keep applying such purging transformations by composition, during execution time, for all leaf triangles of all obstacles P in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ (in any order), until we reach their root triangles. We denote by $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ this final intermediate space, where all obstacles in $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ have been deformed to their root triangles $\{r_i\}$, and by $\mathbf{g}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ the map between $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$, arising from this composition of purging transformations. Since $\mathbf{g}^{\mathcal{I}}$ is a composition of diffeomorphisms, we immediately obtain the following result.

Corollary 1. *The map $\mathbf{g}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ is a C^∞ diffeomorphism between $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ away from sharp corners, none of which lie in the interior of freespace for any of the intermediate or final spaces.*

4.3. Purging of root triangles

After the successive application of the leaf purging transformations presented in Section 4.2, familiar obstacles in $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ are reduced to triangles in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$. These triangles either are homeomorphic to a disk in the interior of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ if they correspond to obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$, or have a common edge with $\partial\mathcal{F}_e$ if they correspond to obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$. Therefore, the final step to generate the model space $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ is to transform each of the root triangles corresponding to obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ to disks, following a procedure similar to Vasilopoulos and Koditschek (2018), and merge the root triangles corresponding to obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$ to $\partial\mathcal{F}_e$.

4.3.1. Center of the transformation and surrounding polygonal collars for obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$. Here we assume that $P_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$. Let the vertices of the root triangle $r_i \in \mathcal{V}_{P_i}$ be $\mathbf{x}_{1r_i}, \mathbf{x}_{2r_i}$ and $\mathbf{x}_{3r_i}$ in counterclockwise order, as shown in Figure 4(1b).

Definition 4. *An admissible center for the transformation of the root triangle r_i , corresponding to a polygon $P_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, is a point $\mathbf{x}_i^*$ in the interior of r_i .*

Without loss of generality, we pick $\mathbf{x}_i^*$ to be the barycenter of r_i , and the radius of the transformation to be a number $\rho_i < d(\mathbf{x}_i^*, \partial r_i)$, following the admissibility assumptions made in Rimón and Koditschek (1992). We also set $\mathcal{Q}_{r_i}$ to be the closure of r_i itself, and define a polygonal collar $\overline{\mathcal{Q}}_{r_i}$ for the root triangle transformation as follows.

Definition 5. *An admissible polygonal collar for the transformation of the root triangle r_i , corresponding to a polygon $P_i \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, is a convex polygon $\overline{\mathcal{Q}}_{r_i}$ such that:*

1. $\overline{\mathcal{Q}}_{r_i} \cap \overline{\mathcal{Q}}_{r_j} = \emptyset$, with $i \neq j$;
2. $\overline{\mathcal{Q}}_{r_i}$ does not intersect any obstacle $C \in \mathcal{C}_{\text{map}}$;
3. $\overline{\mathcal{Q}}_{r_i}$ does not intersect $\partial\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$;
4. $\mathcal{Q}_{r_i} \subset \overline{\mathcal{Q}}_{r_i}$.

An example of such a polygon is shown in Figure 4(1b). Again, this polygon is responsible for limiting the effect of the transformation in its interior, while keeping it equal to the identity map everywhere else. Similarly to Section 4.2, we also construct implicit functions $\gamma_{r_i}(\mathbf{x})$ and $\delta_{r_i}(\mathbf{x})$ for each root triangle r_i , such that

$$\mathcal{Q}_{r_i} = \{\mathbf{x} \in \mathbb{R}^2 \mid \gamma_{r_i}(\mathbf{x}) \leq 0\} \quad (17)$$

$$\overline{\mathcal{Q}}_{r_i} = \{\mathbf{x} \in \mathbb{R}^2 \mid \delta_{r_i}(\mathbf{x}) \geq 0\} \quad (18)$$

4.3.2. Description of the C^∞ switches for obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$. Following the notation of Section 4.2, we can define the auxiliary C^∞ switches

$$\sigma_{\gamma_{r_i}}(\mathbf{x}) := \eta_{\mu_{\gamma_{r_i}}, \epsilon_{r_i}} \circ \gamma_{r_i}(\mathbf{x}) \quad (19)$$

$$\sigma_{\delta_{r_i}}(\mathbf{x}) := \zeta_{\mu_{\delta_{r_i}}} \circ \frac{\delta_{r_i}(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_i^*\|} \quad (20)$$

with $\eta_{\mu, \epsilon}(\chi) := \zeta_\mu(\epsilon - \chi)/\zeta_\mu(\epsilon)$, ζ defined as in (9) and $\mu_{\gamma_{r_i}}, \mu_{\delta_{r_i}}, \epsilon_{r_i} > 0$ tunable parameters.

Based on the above, we then define the C^∞ switch of the transformation of the root triangle r_i as the function $\sigma_{r_i} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{map}}^{\mathcal{I}}$ given by

$$\sigma_{r_i}(\mathbf{x}) := \frac{\sigma_{\gamma_{r_i}}(\mathbf{x})\sigma_{\delta_{r_i}}(\mathbf{x})}{\sigma_{\gamma_{r_i}}(\mathbf{x})\sigma_{\delta_{r_i}}(\mathbf{x}) + (1 - \sigma_{\gamma_{r_i}}(\mathbf{x}))} \quad (21)$$

It can be seen that the function σ_{r_i} will be 0 outside $\overline{\mathcal{Q}}_{r_i}$, exactly equal to 1 on the boundary of $\mathcal{Q}_{r_i}$ (i.e., on the boundary of the root triangle r_i) and varies smoothly between 0 and 1 everywhere else.

We can easily show the following lemma, as the function ζ eliminates all the singular points of δ_{r_i} that correspond to the vertices of $\overline{\mathcal{Q}}_{r_i}$.

Lemma 3. *The switch $\sigma_{r_i} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathbb{R}$ is smooth away from the triangle vertices $\mathbf{x}_{1r_i}, \mathbf{x}_{2r_i}, \mathbf{x}_{3r_i}$, none of which lies in the interior of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$.*

4.3.3. Description of the deforming factors for obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$. Here, the deforming factors are the functions $\nu_{r_i} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathbb{R}$, responsible for transforming each root triangle corresponding to an obstacle in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ to a disk in $\mathbb{R}^2$. The deforming factors we use are inspired by those in Rimon and Koditschek (1992), but do not depend on the values of the implicit functions γ_{r_i} . Namely, the deforming factors are given based on the desired final radii ρ_i as

$$\nu_{r_i}(\mathbf{x}) := \frac{\rho_i}{\|\mathbf{x} - \mathbf{x}_i^*\|} \quad (22)$$

4.3.4. Center of the transformation and surrounding polygonal collars for obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$. Next we focus on obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$. The procedure here is slightly different, because we want to merge the root triangle r_i to the

boundary of the enclosing freespace $\mathcal{F}_e$. Namely, we assume that the vertices of the triangle r_i are $\mathbf{x}_{1r_i}, \mathbf{x}_{2r_i}, \mathbf{x}_{3r_i}$ in counterclockwise order, with $\mathbf{x}_{1r_i}\mathbf{x}_{2r_i}$ the common edge between r_i and $\partial\mathcal{F}_e$, as shown in Figure 4(2b).⁷ Then, we pick an admissible center similarly to Definition 2, as follows.

Definition 6. *An admissible center for the transformation of the root triangle r_i , corresponding to a polygon $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$, denoted by $\mathbf{x}_i^*$, is a point along the triangle median from $\mathbf{x}_{3r_i}$, such that $\mathbf{x}_i^* \in \mathbb{R}^2 \setminus \mathcal{F}_e$.*

We also define $\mathcal{Q}_{r_i}$ to be the convex quadrilateral with boundary $\mathbf{x}_{3r_i}\mathbf{x}_{1r_i}\mathbf{x}_i^*\mathbf{x}_{2r_i}\mathbf{x}_{3r_i}$. The collars used are defined similarly to Definition 3, as the transformation itself is designed to be quite similar with the purging transformation.

Definition 7. *An admissible polygonal collar for the transformation of the root triangle r_i , corresponding to a polygon $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$, is a convex polygon $\overline{\mathcal{Q}}_{r_i}$ such that:*

1. the edges $\mathbf{x}_{1r_i}\mathbf{x}_i^*$ and $\mathbf{x}_i^*\mathbf{x}_{2r_i}$ are edges of $\overline{\mathcal{Q}}_{r_i}$;
2. $\overline{\mathcal{Q}}_{r_i} \cap \overline{\mathcal{Q}}_{r_j} = \emptyset$, with $i \neq j$;
3. $\overline{\mathcal{Q}}_{r_i}$ does not intersect any obstacle $C \in \mathcal{C}_{\text{map}}$;
4. $\mathcal{Q}_{r_i} \subset \overline{\mathcal{Q}}_{r_i}$.

4.3.5. Description of the C^∞ switches for obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$. With the definition of $\mathcal{Q}_{r_i}$ and $\overline{\mathcal{Q}}_{r_i}$ as described above, we associate implicit functions $\gamma_{r_i}(\mathbf{x})$ and $\delta_{r_i}(\mathbf{x})$ as in (17), (18), auxiliary switches $\sigma_{\gamma_{r_i}}$ and $\sigma_{\delta_{r_i}}$ as in (19) and (20), and overall C^∞ switch of the transformation of the root triangle r_i as the function $\sigma_{r_i} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{map}}^{\mathcal{I}}$ given in (21).

4.3.6. Description of the deforming factors for obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$. Finally, in order to merge the root triangle into the boundary $\partial\mathcal{F}_e$, we define the deforming factors similarly to (14), as the functions $\nu_{r_i} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathbb{R}$, given by

$$\nu_{r_i}(\mathbf{x}) := \frac{(\mathbf{x}_{1r_i} - \mathbf{x}_i^*)^\top \mathbf{n}_{r_i}}{(\mathbf{x} - \mathbf{x}_i^*)^\top \mathbf{n}_{r_i}} \quad (23)$$

with

$$\mathbf{n}_{r_i} := \mathbf{R}_{\frac{\pi}{2}} \frac{\mathbf{x}_{2r_i} - \mathbf{x}_{1r_i}}{\|\mathbf{x}_{2r_i} - \mathbf{x}_{1r_i}\|}, \quad \mathbf{R}_{\frac{\pi}{2}} := \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad (24)$$

the normal vector corresponding to the shared edge between r_i and $\partial\mathcal{F}_e$.

4.3.7. The map between $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. First of all, we define

$$\sigma_d(\mathbf{x}) := 1 - \sum_{i \in \mathcal{J}_{\mathcal{B}}^{\mathcal{I}} \cup \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}} \sigma_{r_i}(\mathbf{x}) \quad (25)$$

Using the above constructions and Definitions 4, 5, 6, and 7 we are led to the following results.

Lemma 4. *At any point $\mathbf{x} \in \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$, at most one of the switches $\{\sigma_{r_i}\}_{i \in \mathcal{J}_B^{\mathcal{I}} \cup \mathcal{J}_D^{\mathcal{I}}}$ can be non-zero.*

Corollary 2. *The set $\{\sigma_{r_i}\}_{i \in \mathcal{J}_B^{\mathcal{I}} \cup \mathcal{J}_D^{\mathcal{I}}} \cup \{\sigma_d\}$ defines a partition of unity over $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$.*

With the construction of σ_{r_i} (as in (21)) and ν_{r_i} (as in either (22) or (23), depending on whether i belongs to $\mathcal{J}_D^{\mathcal{I}}$ or $\mathcal{J}_B^{\mathcal{I}}$, respectively) for each root triangle, we can now construct the map $\hat{\mathbf{h}}^{\mathcal{I}} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}}$ given by

$$\hat{\mathbf{h}}^{\mathcal{I}}(\mathbf{x}) := \sum_{i \in \mathcal{J}_B^{\mathcal{I}} \cup \mathcal{J}_D^{\mathcal{I}}} \sigma_{r_i}(\mathbf{x}) [\mathbf{x}_i^* + \nu_{r_i}(\mathbf{x})(\mathbf{x} - \mathbf{x}_i^*)] + \sigma_d(\mathbf{x})\mathbf{x} \quad (26)$$

4.3.8. Qualitative properties of the map between $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. We can again verify that the construction is a smooth change of coordinates between $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. Using Lemma 3 and the fact that the deforming factors ν_{r_i} are smooth in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ (because the centers $\mathbf{x}_i^*$ do not belong in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$) for all i , we get the following result.

Lemma 5. *The map $\hat{\mathbf{h}}^{\mathcal{I}} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}}$ is smooth away from any sharp corners, none of which lie in the interior of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$.*

Proposition 2. *The map $\hat{\mathbf{h}}^{\mathcal{I}}$ is a C^∞ diffeomorphism between $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ away from any sharp corners, none of which lie in the interior of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$.*

Proof. See Appendix E.1. $\square$

4.4. The map between the mapped space and the model space

Based on the construction of $\mathbf{g}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ in Section 4.2.6 and $\hat{\mathbf{h}}^{\mathcal{I}} : \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}}$ in Section 4.3.7, we can finally write the map between the mapped space and the model space as the function $\mathbf{h}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}}$ given by

$$\mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \hat{\mathbf{h}}^{\mathcal{I}} \circ \mathbf{g}^{\mathcal{I}}(\mathbf{x}) \quad (27)$$

It is straightforward to obtain the following result, because both $\mathbf{g}^{\mathcal{I}}$ and $\hat{\mathbf{h}}^{\mathcal{I}}$ are C^∞ diffeomorphisms away from sharp corners.

Corollary 3. *The map $\mathbf{h}^{\mathcal{I}}$ is a C^∞ diffeomorphism between $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ away from any sharp corners, none of which lie in the interior of $\mathcal{F}_{\text{map}}^{\mathcal{I}}$.*

An illustration of the behavior of the map $\mathbf{h}^{\mathcal{I}}$ through a visualization of the values of $\det(D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}})$ for a specific example with a single polygon is included in Figure 5.

5. Reactive controller

The preceding analysis in Section 4 describes the diffeomorphism construction between $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ for a

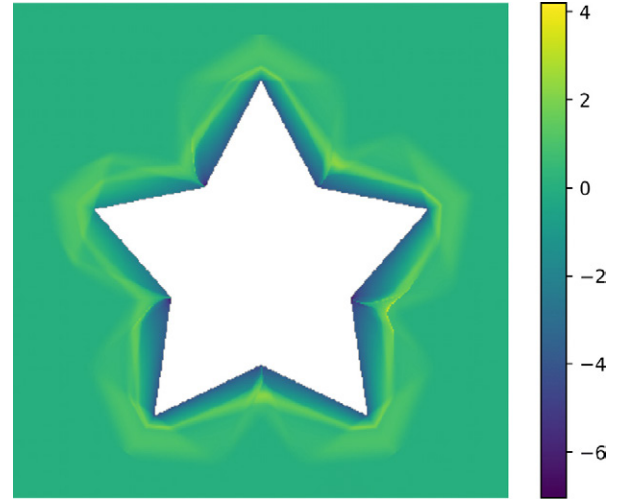


Fig. 5. Values of $\det(D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}})$ for a single polygon in logarithmic scale, showing the local nature of the diffeomorphism ($\mathbf{h}^{\mathcal{I}}$ becomes equal to the identity transform away from the polygon) and the fact that $\mathbf{h}^{\mathcal{I}}$ is smooth away from sharp corners, that do not lie in the interior of the freespace.

given index set $\mathcal{I}$ of instantiated familiar obstacles. However, the onboard sensor might discover new obstacles and, subsequently, incorporate them in the semantic map, updating the set $\mathcal{I}$. Therefore, we have to provide a hybrid systems description of our reactive controller, where each mode is defined by an index set $\mathcal{I} \in 2^{\mathcal{N}_p}$ of familiar obstacles stored in the semantic map, the guards describe the sensor trigger events where a previously “unexplored” obstacle is discovered and incorporated in the semantic map (thereby changing $\mathcal{P}_{\text{map}}^{\mathcal{I}}$, along with $\mathcal{D}_{\text{map}}^{\mathcal{I}}$, $\mathcal{B}_{\text{map}}^{\mathcal{I}}$), and the resets describe transitions to new modes that might result in discrete “jumps” of the robot position in the model space. We then need to show that the resulting hybrid controller, for both the fully actuated robot and the differential drive robot, must succeed in the navigation task.

In the following, Section 5.1 provides the hybrid systems description, Section 5.2 describes the reactive controller applied in each mode of the hybrid system, Section 5.3 summarizes the qualitative properties of our hybrid controller, and Section 5.4 describes our method of generating bounded inputs, each time for both the fully actuated and the differential drive robot. Table 4 summarizes associated notation used throughout this Section.

5.1. Hybrid systems description of navigation framework

5.1.1. Fully actuated robots. First, we consider a fully actuated particle with state $\mathbf{x} \in \mathcal{F}$, and dynamics

$$\dot{\mathbf{x}} = \mathbf{u} \quad (28)$$

Table 4. Key symbols related to the hybrid systems formulation (top, Section 5.1) and the reactive controller construction in each mode of the hybrid system (bottom, Section 5.2) for both a fully actuated robot and a differential drive robot.

$\mathcal{I} \in 2^{\mathcal{N}_p}$	Mode of the hybrid system, Sections 3.1, 5.1
$\mathcal{F}^{\mathcal{I}}, \mathcal{F}_{sem}^{\mathcal{I}}, \mathcal{F}_{map}^{\mathcal{I}}, \mathcal{F}_{model}^{\mathcal{I}} \in \mathbb{R}^2$	Physical, semantic, mapped, model freespace for $\mathcal{I}$, Sections 3, 5.1
$\mathbf{h}^{\mathcal{I}} : \mathcal{F}_{map}^{\mathcal{I}} \rightarrow \mathcal{F}_{model}^{\mathcal{I}}$	Map between $\mathcal{F}_{map}^{\mathcal{I}}$ and $\mathcal{F}_{model}^{\mathcal{I}}$ for mode $\mathcal{I}$, Section 4
$\Gamma \subset 2^{\mathcal{N}_p} \times 2^{\mathcal{N}_p}$	Directed graph of discrete transitions over the modes, Section 5.1
$\mathcal{D} := \coprod_{\mathcal{I} \in 2^{\mathcal{N}_p}} \mathcal{F}^{\mathcal{I}}$	Collection of domains for a fully actuated robot, Section 5.1
$\mathcal{G} := \coprod_{(\mathcal{I}, \mathcal{I}') \in \Gamma} G^{\mathcal{I}, \mathcal{I}'}$	Collection of guards for a fully actuated robot (29)
$\mathcal{R} : \mathcal{G} \rightarrow \mathcal{D}$	Continuous reset map for a fully actuated robot (30)
$\mathcal{U} : \mathcal{D} \rightarrow T\mathcal{D}$	Hybrid vector field for a fully actuated robot (31)
$\mathcal{H} := (2^{\mathcal{N}_p}, \Gamma, \mathcal{D}, \mathcal{U}, \mathcal{G}, \mathcal{R})$	Hybrid system for a fully actuated robot, Theorem 3
$\bar{\mathcal{D}} := \coprod_{\mathcal{I} \in 2^{\mathcal{N}_p}} (\mathcal{F}^{\mathcal{I}} \times S^1)$	Collection of domains for a differential drive robot, Section 5.1
$\bar{\mathcal{G}} := \coprod_{(\mathcal{I}, \mathcal{I}') \in \Gamma} \bar{G}^{\mathcal{I}, \mathcal{I}'}$	Collection of guards for a differential drive robot (33)
$\bar{\mathcal{R}} : \bar{\mathcal{G}} \rightarrow \bar{\mathcal{D}}$	Continuous reset map for a differential drive robot (34)
$\bar{\mathcal{U}} : \bar{\mathcal{D}} \rightarrow T\bar{\mathcal{D}}$	Hybrid vector field for a differential drive robot (35)
$\bar{\mathcal{H}} := (2^{\mathcal{N}_p}, \Gamma, \bar{\mathcal{D}}, \bar{\mathcal{U}}, \bar{\mathcal{G}}, \bar{\mathcal{R}})$	Hybrid system for a differential drive robot, Theorem 4
$\mathbf{x} \in \mathcal{F}_{map}^{\mathcal{I}}$	Fully actuated robot position in $\mathcal{F}_{map}^{\mathcal{I}}$, Section 5.2
$\mathbf{y} := \mathbf{h}^{\mathcal{I}}(\mathbf{x}) \in \mathcal{F}_{model}^{\mathcal{I}}$	Fully actuated robot position in $\mathcal{F}_{model}^{\mathcal{I}}$, Section 5.2
$\mathbf{y}_d := \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)$	Goal position in $\mathcal{F}_{model}^{\mathcal{I}}$, Section 5.2
$\mathcal{LF}(\mathbf{y}) \subset \mathcal{F}_{model}^{\mathcal{I}}$	Local freespace at $\mathbf{y} \in \mathcal{F}_{model}^{\mathcal{I}}$ (37)
$\mathbf{v}^{\mathcal{I}} : \mathcal{F}_{model}^{\mathcal{I}} \rightarrow T\mathcal{F}_{model}^{\mathcal{I}}$	Vector field controller for a fully actuated robot in $\mathcal{F}_{model}^{\mathcal{I}}$ (36)
$\mathbf{u}^{\mathcal{I}} : \mathcal{F}_{map}^{\mathcal{I}} \rightarrow T\mathcal{F}_{map}^{\mathcal{I}}$	Vector field controller for a fully actuated robot in $\mathcal{F}_{map}^{\mathcal{I}}$ (38)
$\bar{\mathbf{h}}^{\mathcal{I}} : \mathcal{F}_{map}^{\mathcal{I}} \times S^1 \rightarrow \mathcal{F}_{model}^{\mathcal{I}} \times S^1$	Diffeomorphism between $\mathcal{F}_{map}^{\mathcal{I}} \times S^1$ and $\mathcal{F}_{model}^{\mathcal{I}} \times S^1$ (41)
$\bar{\mathbf{x}} := (\mathbf{x}, \psi) \in \mathcal{F}_{map}^{\mathcal{I}} \times S^1$	Differential drive robot state in $\mathcal{F}_{map}^{\mathcal{I}} \times S^1$, Section 5.2
$\bar{\mathbf{y}} := (\mathbf{y}, \varphi) = \bar{\mathbf{h}}^{\mathcal{I}}(\bar{\mathbf{x}}) \in \mathcal{F}_{model}^{\mathcal{I}} \times S^1$	Differential drive robot state in $\mathcal{F}_{model}^{\mathcal{I}} \times S^1$, Section 5.2
$\xi^{\mathcal{I}} : S^1 \rightarrow S^1$	Angle transformation between $\mathcal{F}_{map}^{\mathcal{I}} \times S^1$ and $\mathcal{F}_{model}^{\mathcal{I}} \times S^1$ (42)
$\mathbf{y}_{d, \parallel}(\bar{\mathbf{y}}), \mathbf{y}_{d, G}(\bar{\mathbf{y}}) \in \mathcal{F}_{model}^{\mathcal{I}}$	Linear (57) and angular (58) local goals for $\bar{\mathbf{y}} \in \mathcal{F}_{model}^{\mathcal{I}} \times S^1$
$\bar{\mathbf{v}}^{\mathcal{I}} := (\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}}) \in \mathbb{R}^2$	Linear and angular inputs for a unicycle robot in $\mathcal{F}_{model}^{\mathcal{I}} \times S^1$ (53)
$\bar{\mathbf{u}}^{\mathcal{I}} := (\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}}) \in \mathbb{R}^2$	Linear and angular inputs for a unicycle robot in $\mathcal{F}_{map}^{\mathcal{I}} \times S^1$ (52)

As different subsets of instantiated obstacles in $\tilde{\mathcal{P}}$, indexed by $\mathcal{I}$, result in different consolidated polygonal obstacles stored in $\mathcal{P}_{map}^{\mathcal{I}}$, it is natural to index the *modes of the hybrid controller* according to elements $\mathcal{I}$ of the power set $2^{\mathcal{N}_p}$. Every execution, from any initial state, is required to start in the *initial mode*, indexed by $\mathcal{I} = \emptyset$. We also define a *terminal mode* as follows.

Definition 8. *The terminal mode of the hybrid system is indexed by the improper subset, $\mathcal{I} = \mathcal{N}_p$, where all familiar obstacles in the workspace have been instantiated in the set $\mathcal{P}_{sem}^{\mathcal{I}}$, in the sense of Definition 1.*

We denote the freespace in the semantic, mapped, and model spaces, associated with a unique subset $\mathcal{I}$ of $\mathcal{N}_p$, by $\mathcal{F}_{sem}^{\mathcal{I}}, \mathcal{F}_{map}^{\mathcal{I}}, \mathcal{F}_{model}^{\mathcal{I}}$, respectively, as in Section 3. We also denote the corresponding perceived physical freespace by $\mathcal{F}^{\mathcal{I}}$, with $\mathcal{F}^{\mathcal{I}} := \mathcal{F}_{map}^{\mathcal{I}}$, because the dilation of obstacles by r in the passage from the physical to the semantic space and the obstacle merging in the passage from the semantic to the mapped space do not alter the freespace description. The domain $\mathcal{D}$ of our hybrid system is then defined as the collection $\mathcal{D} := \coprod_{\mathcal{I} \in 2^{\mathcal{N}_p}} \mathcal{F}^{\mathcal{I}}$.

Following the notation in Johnson et al. (2016), we can then denote by $\Gamma \subset 2^{\mathcal{N}_p} \times 2^{\mathcal{N}_p}$ the *set of discrete*

transitions for the hybrid system, forming a directed graph structure over the set of modes $2^{\mathcal{N}_p}$. The collection of *guards* associated with Γ can be described as $\mathcal{G} := \coprod_{(\mathcal{I}, \mathcal{I}') \in \Gamma} G^{\mathcal{I}, \mathcal{I}'}$, with $G^{\mathcal{I}, \mathcal{I}'} \subset \mathcal{F}^{\mathcal{I}}$ given by

$$\begin{aligned}
 G^{\mathcal{I}, \mathcal{I}'} &:= \{\mathbf{x} \in \mathcal{F}^{\mathcal{I}} \mid \mathcal{I}' = \mathcal{I} \cup \mathcal{I}_u \\
 \mathcal{I}_u &\neq \emptyset, \mathcal{I}_u \cap \mathcal{I} = \emptyset, \\
 \overline{\mathcal{B}(\mathbf{x}, R)} \cap \tilde{\mathcal{P}}_i &\neq \emptyset \forall i \in \mathcal{I}_u, \\
 \overline{\mathcal{B}(\mathbf{x}, R)} \cap \tilde{\mathcal{P}}_{\mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_u)} &= \emptyset\}
 \end{aligned} \tag{29}$$

with $\tilde{\mathcal{P}}_{\mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_u)} := \{\tilde{\mathcal{P}}_i\}_{i \in \mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_u)}$.

In addition, the *reset* $\mathcal{R} : \mathcal{G} \rightarrow \mathcal{D}$ is the continuous map that restricts simply as $R^{\mathcal{I}, \mathcal{I}'} := \mathcal{R}|_{G^{\mathcal{I}, \mathcal{I}'}} : G^{\mathcal{I}, \mathcal{I}'} \rightarrow \mathcal{F}^{\mathcal{I}'}$, with

$$R^{\mathcal{I}, \mathcal{I}'}(\mathbf{x}) = \mathbf{x} \tag{30}$$

the identity map. Note, however, that although the robot cannot experience discrete jumps in the physical space, the model space $\mathcal{F}_{model}^{\mathcal{I}'}$ is likely to be a discontinuously different space from $\mathcal{F}_{model}^{\mathcal{I}}$ (i.e., there is no guaranteed inclusion from $\mathcal{F}_{model}^{\mathcal{I}'}$ into $\mathcal{F}_{model}^{\mathcal{I}}$), hence the model position in the new space bears no obvious relationship to that in the prior. Namely, the position of the robot in the model space after a

transition from mode $\mathcal{I}$ to mode $\mathcal{I}'$ will be given by $\mathbf{h}^{\mathcal{I}'} \circ (\mathbf{h}^{\mathcal{I}})^{-1}(\mathbf{y})$, with $\mathbf{y} \in \mathcal{F}_{\text{model}}^{\mathcal{I}}$.

Finally, we can construct the *hybrid vector field* $\mathbf{U} : \mathcal{D} \rightarrow T\mathcal{D}$ that restricts to a vector field $\mathbf{U}^{\mathcal{I}} := \mathbf{U}|_{\mathcal{F}^{\mathcal{I}}} : \mathcal{F}^{\mathcal{I}} \rightarrow T\mathcal{F}^{\mathcal{I}}$, that can be written as

$$\mathbf{U}^{\mathcal{I}}(\mathbf{x}) := \mathbf{u}^{\mathcal{I}}(\mathbf{x}) \quad (31)$$

with $\mathbf{u}^{\mathcal{I}}$ given in (38) and described in the next section.

Based on the above definitions, we define the *navigational hybrid system for fully actuated robots* as the tuple $\mathbf{H} := (2^{\mathcal{N}_p}, \Gamma, \mathcal{D}, \mathbf{U}, \mathbf{G}, \mathbf{R})$ describing the modes, discrete transitions, domains, associated vector fields, guards and resets.

5.1.2. Differential drive robots. Next, we focus on a differential drive robot, whose state is $\bar{\mathbf{x}} := (\mathbf{x}, \psi) \in \mathcal{F} \times S^1 \subset SE(2)$, and its dynamics are given by⁸

$$\dot{\bar{\mathbf{x}}} = \mathbf{B}(\psi)\bar{\mathbf{u}} \quad (32)$$

with $\mathbf{B}(\psi) := \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}^{\top}$ and $\bar{\mathbf{u}} := (v, \omega)$, with $v, \omega \in \mathbb{R}$ the linear and angular input, respectively.

The analysis here is fairly similar; the modes and discrete transitions are identical. However, the robot operates on a subset of $SE(2)$ and, therefore, the domains must be described as $\bar{\mathcal{D}} := \coprod_{\mathcal{I} \in 2^{\mathcal{N}_p}} (\mathcal{F}^{\mathcal{I}} \times S^1)$. Consequently, the collection of *guards* that will result in transitions between different modes according to Γ are described as $\bar{\mathcal{G}} := \coprod_{(\mathcal{I}, \mathcal{I}') \in \Gamma} \bar{\mathcal{G}}^{\mathcal{I}, \mathcal{I}'}$, with $\bar{\mathcal{G}}^{\mathcal{I}, \mathcal{I}'} \subset (\mathcal{F}^{\mathcal{I}} \times S^1)$ given by

$$\begin{aligned} \bar{\mathcal{G}}^{\mathcal{I}, \mathcal{I}'} &:= \{\bar{\mathbf{x}} = (\mathbf{x}, \psi) \in \mathcal{F}^{\mathcal{I}} \times S^1 \mid \mathcal{I}' = \mathcal{I} \cup \mathcal{I}_u \\ \mathcal{I}_u &\neq \emptyset, \mathcal{I}_u \cap \mathcal{I} = \emptyset, \\ \bar{\mathbf{B}}(\mathbf{x}, \mathcal{R}) \cap \tilde{\mathcal{P}}_i &\neq \emptyset \text{ for all } i \in \mathcal{I}_u, \\ \bar{\mathbf{B}}(\mathbf{x}, \mathcal{R}) \cap \tilde{\mathcal{P}}_{\mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_u)} &= \emptyset\} \end{aligned} \quad (33)$$

In addition, the *reset* $\bar{\mathbf{R}} : \bar{\mathcal{G}} \rightarrow \bar{\mathcal{D}}$ is the continuous map that restricts simply as $\bar{\mathbf{R}}^{\mathcal{I}, \mathcal{I}'} := \bar{\mathbf{R}}|_{\bar{\mathcal{G}}^{\mathcal{I}, \mathcal{I}'}} : \bar{\mathcal{G}}^{\mathcal{I}, \mathcal{I}'} \rightarrow (\mathcal{F}^{\mathcal{I}'} \times S^1)$, with

$$\bar{\mathbf{R}}^{\mathcal{I}, \mathcal{I}'}(\bar{\mathbf{x}}) = \bar{\mathbf{x}} \quad (34)$$

the identity map.

Finally, the fact that the robot operates in $SE(2)$ gives rise to a new *hybrid vector field* $\bar{\mathbf{U}} : \bar{\mathcal{D}} \rightarrow T\bar{\mathcal{D}}$, that restricts to a vector field $\bar{\mathbf{U}}^{\mathcal{I}} := \bar{\mathbf{U}}|_{\mathcal{F}^{\mathcal{I}} \times S^1} : \mathcal{F}^{\mathcal{I}} \times S^1 \rightarrow T(\mathcal{F}^{\mathcal{I}} \times S^1)$, that can be written as

$$\bar{\mathbf{U}}^{\mathcal{I}}(\bar{\mathbf{x}}) := \mathbf{B}(\psi)\bar{\mathbf{u}}^{\mathcal{I}} \quad (35)$$

with the inputs $\bar{\mathbf{u}}^{\mathcal{I}} = (v^{\mathcal{I}}, \omega^{\mathcal{I}})$ given as in (52) and described in the next section.

Based on the above definitions, we define the *navigational hybrid system for differential drive robots* as the tuple

$\bar{\mathbf{H}} := (2^{\mathcal{N}_p}, \Gamma, \bar{\mathcal{D}}, \bar{\mathbf{U}}, \bar{\mathbf{G}}, \bar{\mathbf{R}})$ describing the modes, discrete transitions, domains, associated vector fields, guards, and resets.

5.2. Reactive controller in each hybrid mode

The preceding analysis of the hybrid system allows us to now describe the constituent controllers in each mode $\mathcal{I}$ of the hybrid system, for both the fully actuated and the differential drive robot. For the results pertaining to each separate mode, we are going to assume that $\mathcal{I}$ describes the terminal mode of the hybrid system, in the notion of Definition 8.

With this assumption, we can arrive to Theorems 1 and 2, that allow us to establish the main results about our hybrid controller in Theorems 3 and 4. We assume that the robot operates in $\mathcal{F}_{\text{map}}^{\mathcal{I}}$,⁹ and the set of consolidated obstacles $\mathcal{P}_{\text{map}}^{\mathcal{I}}$ in mode $\mathcal{I}$ has been identified.

5.2.1. Fully actuated robots. The dynamics of the fully actuated particle in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ with state $\mathbf{y} = \mathbf{h}^{\mathcal{I}}(\mathbf{x}) \in \mathcal{F}_{\text{model}}^{\mathcal{I}}$ can be described by $\dot{\mathbf{y}} = \mathbf{v}^{\mathcal{I}}(\mathbf{y})$ with the input $\mathbf{v}^{\mathcal{I}}(\mathbf{y})$ given in (Arslan and Koditschek, 2016b) as¹⁰

$$\mathbf{v}^{\mathcal{I}}(\mathbf{y}) = -(\mathbf{y} - \Pi_{\mathcal{L}(\mathbf{y})}(\mathbf{y}_d)) \quad (36)$$

with $\mathbf{y}_d = \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)$, and the convex *local freespace* for $\mathbf{y}$, $\mathcal{L}(\mathbf{y})$, defined as the Voronoi cell in Arslan and Koditschek (2016b: Equations (7) and (24)):

$$\mathcal{L}(\mathbf{y}) := \left\{ \mathbf{q} \in \mathcal{F}_{\text{model}}^{\mathcal{I}} \mid \|\mathbf{q} - \mathbf{y}\| \leq \|\mathbf{q} - \Pi_{\bar{\mathcal{O}}_i}(\mathbf{y})\|, \forall i \right\} \cap \bar{\mathbf{B}}(\mathbf{y}, \frac{R_{\text{model}}}{2}) \quad (37)$$

Here, i spans the obstacles in both $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ (represented in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ by $\bar{\mathbf{B}}(\mathbf{x}_i^*, \rho_i)$) and $\mathcal{C}_{\text{map}}$ (transferred to $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ with an identity map), and R_{model} is the range of the virtual sensor used for obstacle detection in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. Similarly to Vasilopoulos and Koditschek (2018), using the diffeomorphism construction in (27), we construct our controller as the vector field $\mathbf{u}^{\mathcal{I}} = : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow T\mathcal{F}_{\text{map}}^{\mathcal{I}}$ given by

$$\mathbf{u}^{\mathcal{I}}(\mathbf{x}) = k[D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}]^{-1} \cdot (\mathbf{v}^{\mathcal{I}} \circ \mathbf{h}^{\mathcal{I}}(\mathbf{x})) \quad (38)$$

with $k > 0$. Note here that the strategy employed never requires the explicit computation of $(\mathbf{h}^{\mathcal{I}})^{-1}$, which would make our numerical realization quite difficult; instead, it merely requires inversion of $[D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}]$.

We note that if the range of the virtual sensor R_{model} used to construct $\mathcal{L}(\mathbf{y})$ in the model space is smaller than the range of our sensor R , the vector field $\mathbf{u}^{\mathcal{I}}$ is Lipschitz continuous because $\mathbf{v}^{\mathcal{I}}(\mathbf{y})$ is shown to be Lipschitz continuous in Arslan and Koditschek (2016b), $\mathbf{y} = \mathbf{h}^{\mathcal{I}}(\mathbf{x})$ is a smooth change of coordinates away from sharp corners, and the robot discovers obstacles before actually using them for navigation, because $R_{\text{model}} < R$. We are led to the following result.

Corollary 4. *With $\mathcal{I}$ the terminal mode of the hybrid controller, the vector field $\mathbf{u}^\mathcal{I} : \mathcal{F}_{\text{map}}^\mathcal{I} \rightarrow T\mathcal{F}_{\text{map}}^\mathcal{I}$ generates a unique continuously differentiable partial flow.*

To ensure completeness (i.e., absence of finite time escape through boundaries in $\mathcal{F}_{\text{map}}^\mathcal{I}$) we must verify that the robot never collides with any obstacle in the environment, i.e., leaves its freespace positively invariant. However, this property follows almost directly from the fact that the vector field $\mathbf{u}^\mathcal{I}$ on $\mathcal{F}_{\text{map}}^\mathcal{I}$ is the pushforward of the complete vector field $\mathbf{v}^\mathcal{I}$ through $(\mathbf{h}^\mathcal{I})^{-1}$, guaranteed to insure that $\mathcal{F}_{\text{model}}^\mathcal{I}$ remain positively invariant under its flow as shown in Arslan and Koditschek (2016b), away from sharp corners on the boundary of $\mathcal{F}_{\text{map}}^\mathcal{I}$. Therefore, we immediately obtain the following result.

Proposition 3. *With $\mathcal{I} = \mathcal{N}_\mathcal{P}$ the terminal mode of the hybrid controller, the freespace interior $\mathcal{F}_{\text{map}}^\mathcal{I}$ is positively invariant under the law (38).*

Next, we focus on the stationary points of $\mathbf{u}^\mathcal{I}$.

Lemma 6. *With $\mathcal{I} = \mathcal{N}_\mathcal{P}$ the terminal mode of the hybrid controller:*

1. *the set of stationary points of control law (38) is given as $\{\mathbf{x}_d\} \cup \{(\mathbf{h}^\mathcal{I})^{-1}(\mathbf{s}_i)\}_{i \in \mathcal{J}_\mathcal{D}} \cup \{\mathbf{g}_k\}_{k \in \mathcal{J}_\mathcal{C}}$, where*

$$\mathbf{s}_i = \mathbf{x}_i^* - \rho_i \frac{\mathbf{h}^\mathcal{I}(\mathbf{x}_d) - \mathbf{x}_i^*}{\|\mathbf{h}^\mathcal{I}(\mathbf{x}_d) - \mathbf{x}_i^*\|} \quad (39a)$$

$$\mathcal{G}_k = \left\{ \mathbf{q} \in \mathcal{F}_{\text{map}}^\mathcal{I} \mid d(\mathbf{q}, C_k) = r, \kappa(\mathbf{q}) = 1 \right\} \quad (39b)$$

with

$$\kappa(\mathbf{q}) := \frac{(\mathbf{q} - \Pi_{\bar{C}_k}(\mathbf{q}))^\top (\mathbf{q} - \mathbf{h}^\mathcal{I}(\mathbf{x}_d))}{\|\mathbf{q} - \Pi_{\bar{C}_k}(\mathbf{q})\| \cdot \|\mathbf{q} - \mathbf{h}^\mathcal{I}(\mathbf{x}_d)\|}$$

2. *the goal $\mathbf{x}_d$ is the only locally stable equilibrium of control law (38) and all the other stationary points $\{(\mathbf{h}^\mathcal{I})^{-1}(\mathbf{s}_i)\}_{i \in \mathcal{J}_\mathcal{D}} \cup \{\mathbf{g}_k\}_{k \in \mathcal{J}_\mathcal{C}}$, each associated with an obstacle, are non-degenerate saddles.*

Proof. See Appendix E.2. $\square$

Note that there is a slight complication here; each stationary point $\mathbf{s}_i, i \in \mathcal{J}_\mathcal{D}$ lies on the boundary of the corresponding ball $\bar{B}(\mathbf{x}_i^*, \rho_i)$ in the model space and, thus, by construction of the diffeomorphism $\mathbf{h}^\mathcal{I}$, it might not lie in the domain of $(\mathbf{h}^\mathcal{I})^{-1}$ because it could correspond to a sharp corner (i.e., a polygon vertex) in the mapped space. Although such problems can only occur for a thin subset of obstacle placements, we explicitly impose the following assumption to facilitate our formal results.

Assumption 4. *The stationary points of control law (36) in the model space lie in the domain of the map $(\mathbf{h}^\mathcal{I})^{-1}$ between $\mathcal{F}_{\text{model}}^\mathcal{I}$ and $\mathcal{F}_{\text{map}}^\mathcal{I}$.*

As such pathological cases can only occur for a “thin” (empty interior) subset of obstacle placements and the considered stationary points are shown to be non-degenerate saddles, it should be highlighted that Assumption 4 has only theoretical and no practical implications, and does not affect the controller’s performance in any way.

Then, using Lemma 6, we arrive at the following result, that establishes (almost) global convergence to the goal $\mathbf{x}_d$.

Proposition 4. *With $\mathcal{I}$ the terminal mode of the hybrid controller, the goal $\mathbf{x}_d$ is an asymptotically stable equilibrium of (38), whose region of attraction includes the freespace $\mathcal{F}_{\text{map}}^\mathcal{I}$ except a set of measure zero.*

Proof. See Appendix E.2. $\square$

We can now immediately conclude the following central summary statement.

Theorem 1. *With $\mathcal{I}$ the terminal mode of the hybrid controller, the reactive controller in (38) leaves the freespace $\mathcal{F}_{\text{map}}^\mathcal{I}$ positively invariant, and its unique continuously differentiable flow, starting at almost any robot placement $\mathbf{x} \in \mathcal{F}_{\text{map}}^\mathcal{I}$, asymptotically reaches the goal location $\mathbf{x}_d$, while strictly decreasing $\|\mathbf{h}^\mathcal{I}(\mathbf{x}) - \mathbf{h}^\mathcal{I}(\mathbf{x}_d)\|$ along the way.*

A depiction of the vector field in (38) for the terminal mode $\mathcal{I}$ (Definition 8) from one of our numerical examples presented in Section 7 is included in Figure 6.

5.2.2. Differential drive robots. As the robot operates in $SE(2)$ instead of $\mathbb{R}^2$, we first need to come up with a

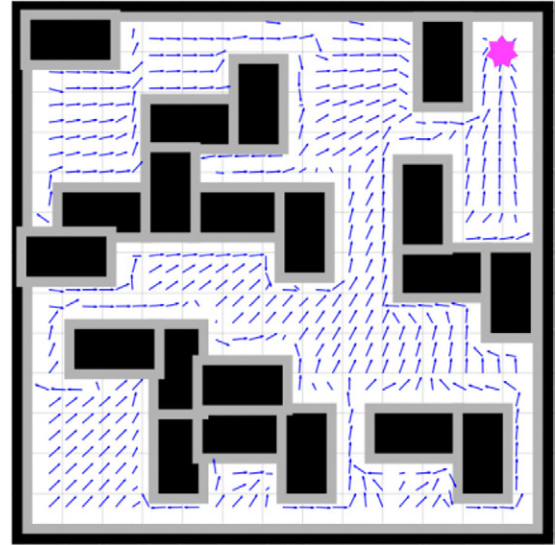


Fig. 6. Depiction of the vector field in (38) for the terminal mode $\mathcal{I}$ from one of our numerical examples presented in Section 7 with several overlapping obstacles. Note how the vector field guarantees safety around each obstacle, with the goal in purple attracting globally.

smooth diffeomorphism $\bar{\mathbf{h}}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1 \rightarrow \mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ away from sharp corners on the boundary of $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$, and then establish the results about our controller.

Following our previous work (Vasilopoulos and Koditschek, 2018), we construct our map $\bar{\mathbf{h}}^{\mathcal{I}}$ from $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ to $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ as

$$\bar{\mathbf{y}} = (\mathbf{y}, \varphi) = \bar{\mathbf{h}}^{\mathcal{I}}(\bar{\mathbf{x}}) := (\mathbf{h}^{\mathcal{I}}(\mathbf{x}), \xi^{\mathcal{I}}(\bar{\mathbf{x}})) \quad (40)$$

with $\bar{\mathbf{x}} = (\mathbf{x}, \psi) \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$, $\bar{\mathbf{y}} := (\mathbf{y}, \varphi) \in \mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ and

$$\varphi = \xi^{\mathcal{I}}(\bar{\mathbf{x}}) := \angle(\mathbf{e}(\bar{\mathbf{x}})) \quad (41)$$

Here, $\angle \mathbf{e} := \text{atan2}(e_2, e_1)$ and

$$\mathbf{e}(\bar{\mathbf{x}}) = \Pi_{\mathbf{y}} \cdot D_{\bar{\mathbf{x}}} \bar{\mathbf{h}}^{\mathcal{I}} \cdot \mathbf{B}(\psi) \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} \quad (42)$$

with $\Pi_{\mathbf{y}}$ denoting the projection onto the first two components. The reason for choosing φ as in (42) will become evident later, in our effort to control the equivalent differential drive robot dynamics in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$.

Proposition 5. *The map $\bar{\mathbf{h}}^{\mathcal{I}}$ in (40) is a C^∞ diffeomorphism from $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ to $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ away from sharp corners, none of which lie in the interior of $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$.*

Proof. See Appendix E.2. $\square$

Then, using (40), we can find the pushforward of the differential drive robot dynamics in (32) as

$$\begin{aligned} \bar{\mathbf{y}} &= \frac{d}{dt} \begin{bmatrix} \mathbf{h}^{\mathcal{I}}(\mathbf{x}) \\ \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix} \\ &= \left[D_{\bar{\mathbf{x}}} \bar{\mathbf{h}}^{\mathcal{I}} \circ (\bar{\mathbf{h}}^{\mathcal{I}})^{-1} \right] \cdot \left(\mathbf{B} \circ (\bar{\mathbf{h}}^{\mathcal{I}})^{-1}(\bar{\mathbf{y}}) \right) \cdot \bar{\mathbf{u}}^{\mathcal{I}} \end{aligned} \quad (43)$$

Based on the above, we can then write

$$\bar{\mathbf{y}} = \begin{bmatrix} \dot{\mathbf{y}} \\ \dot{\varphi} \end{bmatrix} = \frac{d}{dt} \begin{bmatrix} \mathbf{h}^{\mathcal{I}}(\mathbf{x}) \\ \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix} = \mathbf{B}(\varphi) \bar{\mathbf{v}}^{\mathcal{I}} \quad (44)$$

with $\bar{\mathbf{v}}^{\mathcal{I}} = (\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}})$, and the inputs $(\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}})$ related to $(\mathbf{v}^{\mathcal{I}}, \omega^{\mathcal{I}})$ through

$$\hat{\mathbf{v}}^{\mathcal{I}} = \|\mathbf{e}(\bar{\mathbf{x}})\| \mathbf{v}^{\mathcal{I}} \quad (45)$$

$$\hat{\omega}^{\mathcal{I}} = \mathbf{v}^{\mathcal{I}} D_{\mathbf{x}} \xi^{\mathcal{I}} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} + \frac{\partial \xi^{\mathcal{I}}}{\partial \psi} \omega^{\mathcal{I}} \quad (46)$$

with $D_{\mathbf{x}} \xi^{\mathcal{I}} = \begin{bmatrix} \frac{\partial \xi^{\mathcal{I}}}{\partial x} & \frac{\partial \xi^{\mathcal{I}}}{\partial y} \end{bmatrix}$. Here, we can calculate

$$D_{\mathbf{x}} \xi^{\mathcal{I}} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} = \frac{\alpha_1(\bar{\mathbf{x}}) \alpha_3(\bar{\mathbf{x}}) + \alpha_2(\bar{\mathbf{x}}) \alpha_4(\bar{\mathbf{x}})}{\|\mathbf{e}(\bar{\mathbf{x}})\|^2} \quad (47)$$

with the auxiliary terms $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ defined as

$$\alpha_1(\bar{\mathbf{x}}) := -([D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{21} \cos \psi + [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{22} \sin \psi) \quad (48)$$

$$\alpha_2(\bar{\mathbf{x}}) := [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{11} \cos \psi + [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{12} \sin \psi \quad (49)$$

$$\begin{aligned} \alpha_3(\bar{\mathbf{x}}) &:= \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{11}}{\partial [\mathbf{x}]_1} \cos^2 \psi + \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{12}}{\partial [\mathbf{x}]_2} \sin^2 \psi \\ &\quad + \left(\frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{11}}{\partial [\mathbf{x}]_2} + \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{12}}{\partial [\mathbf{x}]_1} \right) \sin \psi \cos \psi \end{aligned} \quad (50)$$

$$\begin{aligned} \alpha_4(\bar{\mathbf{x}}) &:= \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{21}}{\partial [\mathbf{x}]_1} \cos^2 \psi + \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{22}}{\partial [\mathbf{x}]_2} \sin^2 \psi \\ &\quad + \left(\frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{21}}{\partial [\mathbf{x}]_2} + \frac{\partial [D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}]_{22}}{\partial [\mathbf{x}]_1} \right) \sin \psi \cos \psi \end{aligned} \quad (51)$$

We provide more details about the calculation of partial derivatives for elements of $D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}$ used above in Section 6 and in Appendix D.

Hence, we have found equivalent differential drive robot dynamics, defined on $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$. The idea now is to use the control strategy in Arslan and Koditschek (2016b) for the dynamical system in (44) to find inputs $\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}}$ in $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$, and then use (45), (46) to find the actual inputs $\mathbf{v}^{\mathcal{I}}, \omega^{\mathcal{I}}$ in $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ that achieve $\hat{\mathbf{v}}^{\mathcal{I}}, \hat{\omega}^{\mathcal{I}}$ as

$$\mathbf{v}^{\mathcal{I}} = \frac{k_v \hat{\mathbf{v}}^{\mathcal{I}}}{\|\mathbf{e}(\bar{\mathbf{x}})\|} \quad (52a)$$

$$\omega^{\mathcal{I}} = \left(\frac{\partial \xi^{\mathcal{I}}}{\partial \psi} \right)^{-1} \left(k_\omega \hat{\omega}^{\mathcal{I}} - \mathbf{v}^{\mathcal{I}} D_{\mathbf{x}} \xi^{\mathcal{I}} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} \right) \quad (52b)$$

with $k_v, k_\omega > 0$ fixed gains.

Namely, inspired by Arslan and Koditschek (2016b) and Astolfi (1999), we design our inputs $\hat{\mathbf{v}}^{\mathcal{I}}$ and $\hat{\omega}^{\mathcal{I}}$ as¹¹

$$\hat{\mathbf{v}}^{\mathcal{I}} = - \begin{bmatrix} \cos \varphi \\ \sin \varphi \end{bmatrix}^\top (\mathbf{y} - \mathbf{y}_{d, \parallel}(\bar{\mathbf{y}})) \quad (53a)$$

$$\hat{\omega}^{\mathcal{I}} = \text{atan} \left(\frac{\begin{bmatrix} -\sin \varphi \\ \cos \varphi \end{bmatrix}^\top (\mathbf{y} - \mathbf{y}_{d, G}(\bar{\mathbf{y}}))}{\begin{bmatrix} \cos \varphi \\ \sin \varphi \end{bmatrix}^\top (\mathbf{y} - \mathbf{y}_{d, G}(\bar{\mathbf{y}}))} \right) \quad (53b)$$

with $\mathcal{LF}(\mathbf{y}) \subset \mathcal{F}_{\text{model}}^{\mathcal{I}}$ the convex polygon defining the local freespace at $\mathbf{y} = \mathbf{h}^{\mathcal{I}}(\mathbf{x})$, and linear and angular local goals $\mathbf{y}_{d, \parallel}(\bar{\mathbf{y}}), \mathbf{y}_{d, G}(\bar{\mathbf{y}})$ given by

$$\mathbf{y}_{d, \parallel}(\bar{\mathbf{y}}) := \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d) \quad (54)$$

$$\mathbf{y}_{d, G}(\bar{\mathbf{y}}) := \frac{\Pi_{\mathcal{LF}(\mathbf{y}) \cap H_G}(\mathbf{y}_d) + \Pi_{\mathcal{LF}(\mathbf{y})}(\mathbf{y}_d)}{2} \quad (55)$$

with $H_{\parallel}$ and H_G the lines defined in Arslan and Koditschek (2016b) as

$$H_{\parallel} = \left\{ \mathbf{z} \in \mathcal{F}_{\text{model}}^{\mathcal{I}} \mid \begin{bmatrix} -\sin \varphi \\ \cos \varphi \end{bmatrix}^\top (\mathbf{z} - \mathbf{y}) = 0 \right\} \quad (56)$$

$$H_G = \{ \alpha \mathbf{y} + (1 - \alpha) \mathbf{y}_d \in \mathcal{F}_{\text{model}}^{\mathcal{I}} \mid \alpha \in \mathbb{R} \} \quad (57)$$

The properties of the differential drive robot control law given in (52) can be summarized in the following theorem.

Theorem 2. *With $\mathcal{I}$ the terminal mode of the hybrid controller, the reactive controller for differential drive robots, given in (52), leaves the freespace $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ positively invariant, and its unique continuously differentiable flow, starting at almost any robot configuration $(\mathbf{x}, \psi) \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$, asymptotically steers the robot to the goal location $\mathbf{x}_d$, without increasing $\|\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)\|$ along the way.*

Proof. See Appendix E.2. $\square$

5.3. Qualitative properties of the hybrid controller

5.3.1. Fully actuated robots. First, we show that the navigational hybrid system H inherits the fundamental consistency properties outlined in Johnson et al. (2016: Theorems 5–9), in order to establish that the hybrid system is well-behaved in the sense of being both deterministic and non-blocking (i.e., generating executions defined for all future times).

Lemma 7. *The hybrid system H has disjoint guards.*

Proof. See Appendix E.2. $\square$

An immediate result following Lemma 7, that does not allow a robot state $\mathbf{x} \in \mathcal{F}^{\mathcal{I}}$ to be contained in more than one guard $G^{\mathcal{I}, \mathcal{I}'}$, and the nice properties of the flow in each separate mode, summarized in Corollary 4, is the following important consistency property.

Corollary 5. *The hybrid system H is deterministic.*

Next, we focus on the non-blocking property. As stated in Johnson et al. (2016), a hybrid execution might be blocked either by conventional finite escape through the boundary of the hybrid domain at a point in the complement of all the guards, by escape through a point in the guard whose reset lies outside of the hybrid domain, or by hybrid ambiguity, i.e., by arriving at a point through the continuous flow that lies in the complement of the guard G and yet still on the boundary of G . We eliminate all cases in the proof of the following result.

Lemma 8. *The hybrid system H is non-blocking.*

Proof. See Appendix E.2. $\square$

Finally, using the last part of the proof of Lemma 8 which shows that the (identity) reset from a given mode cannot lie in the guard of the next mode, we arrive at the following result about the discrete transitions of the hybrid system H .

Corollary 6. *An execution of the hybrid system H undergoes no more than one hybrid transition at a single time t .*

Based on the above, the central result about the hybrid controller for a fully actuated robot can be summarized in the following theorem.

Theorem 3. *For a fully actuated robot with dynamics defined in (28), the deterministic, non-blocking*

navigational hybrid system $H := (2^{\mathcal{N}_p}, \Gamma, D, U, G, R)$, with the restrictions of guards G , resets R and vector fields U defined as in (29), (30), and (31), respectively, leaves the free space $\mathcal{F}$ positively invariant under the Lipschitz continuous, piecewise smooth flow associated with each of its hybrid domains, and, starting at almost any robot placement $\mathbf{x} \in \mathcal{F}$ at time t_0 with an initial mode $\mathcal{I} = \emptyset$, asymptotically reaches a designated goal location $\mathbf{x}_d \in \mathcal{F}$, in a previously unexplored environment satisfying Assumptions 1–4, with a uniquely defined (in both state and mode) execution for all $t > t_0$.

Proof. See Appendix E.2. $\square$

5.3.2. Differential drive robots. We can then follow exactly the same procedure to prove the following statement for the hybrid controller for differential drive robots.

Theorem 4. *For a differential drive robot with dynamics defined in (32), the deterministic, non-blocking navigational hybrid system $\bar{H} := (2^{\mathcal{N}_p}, \bar{\Gamma}, \bar{D}, \bar{U}, \bar{G}, \bar{R})$, with the restrictions of guards $\bar{G}$, resets $\bar{R}$ and vector fields $\bar{U}$ defined as in (33), (34), and (35), respectively, leaves the free space $\mathcal{F} \times S^1$ positively invariant under the Lipschitz continuous, piecewise smooth flow associated with each of its hybrid domains, and, starting at almost any robot placement $\bar{\mathbf{x}} \in \mathcal{F} \times S^1$ at time t_0 with an initial mode $\bar{\mathcal{I}} = \emptyset$, asymptotically reaches a designated goal location $\bar{\mathbf{x}}_d \in \mathcal{F}$, in a previously unexplored environment satisfying Assumptions 1–4, with a uniquely defined (in both state and mode) execution for all $t > t_0$.*

5.4. Generating bounded inputs

Although the control inputs for both a fully actuated robot and a differential drive robot, described in (38) and (52), respectively, can be used in the hybrid systems description of the controller (see (31) and (35)) to yield the desired results of Theorems 3 and 4, we have so far implicitly assumed that there is no bound in the magnitude of $\mathbf{u}^{\mathcal{I}}$ in (38) or the magnitudes of $\mathbf{v}^{\mathcal{I}}, \omega^{\mathcal{I}}$ in (52) for each separate mode $\mathcal{I}$. In this section, we show how to generate bounded inputs without affecting the results of Theorems 3 and 4.

5.4.1. Fully actuated robots. We focus on fully actuated robots first. Let $\mathbf{u}_{\text{nom}}^{\mathcal{I}} : \mathcal{F}_{\text{map}}^{\mathcal{I}} \rightarrow T\mathcal{F}_{\text{map}}^{\mathcal{I}}$ denote the nominal input for mode $\mathcal{I}$, defined using (38) as

$$\mathbf{u}_{\text{nom}}^{\mathcal{I}}(\mathbf{x}) := [D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}]^{-1} \cdot (\mathbf{v}^{\mathcal{I}} \circ \mathbf{h}^{\mathcal{I}}(\mathbf{x})) \quad (58)$$

We can then easily satisfy the requirement $\|\mathbf{u}^{\mathcal{I}}\| \leq u_{\text{max}}$ by picking a gain k such that $0 < k \leq u_{\text{max}}$ and defining our controller as

$$\mathbf{u}^{\mathcal{I}}(\mathbf{x}) := k \frac{\mathbf{u}_{\text{nom}}^{\mathcal{I}}(\mathbf{x})}{\|\mathbf{u}_{\text{nom}}^{\mathcal{I}}(\mathbf{x})\| + \epsilon_{\mathbf{u}}} \quad (59)$$

with $\epsilon_u > 0$ a small number. The modified (bounded) controller in (59) does not affect the results of Theorem 3, because it maintains the heading direction of the original (unbounded) controller in (38), and just limits its magnitude.

5.4.2. Differential drive robots. The analysis is slightly more complicated for differential drive robots, because we have to respect the fact that the actual inputs v^I , ω^I are related to the inputs $\hat{v}^I$, $\hat{\omega}^I$ through (52). However, an important observation, deriving from the proof of Theorem 2, is that the choice of gains $k_v, k_\omega > 0$ in (52) does not affect the positive invariance or convergence properties of the controller, which rely entirely on $\hat{v}^I$, $\hat{\omega}^I$, given in (53).

Therefore, the main idea is to adaptively change the gains online, in order to satisfy the constraints $|v^I| \leq v_{\max}$, $|\omega^I| \leq \omega_{\max}$. Namely, using (52), we look for gains $k_v(\bar{\mathbf{x}})$, $k_\omega(\bar{\mathbf{x}})$ such that

$$k_v(\bar{\mathbf{x}}) \frac{|\hat{v}^I(\bar{\mathbf{x}})|}{\|\mathbf{e}(\bar{\mathbf{x}})\|} \leq v_{\max}$$

$$\left| k_\omega(\bar{\mathbf{x}}) \hat{\omega}^I(\bar{\mathbf{x}}) - k_v(\bar{\mathbf{x}}) \frac{\hat{v}^I(\bar{\mathbf{x}})}{\|\mathbf{e}(\bar{\mathbf{x}})\|} \vartheta(\psi) \right| \leq \frac{\partial \xi^I}{\partial \psi} \omega_{\max}$$

with $\vartheta(\psi) := D_{\mathbf{x}} \xi^I [\cos \psi \quad \sin \psi]^\top$, because $\frac{\partial \xi^I}{\partial \psi} > 0$, as shown in the proof of Proposition 5. A conservative selection of gains that satisfies the above constraints can then be extracted using the triangle inequality as follows:

$$k_v(\bar{\mathbf{x}}) = \min \left(k_{v, \text{nom}}, \frac{\|\mathbf{e}(\bar{\mathbf{x}})\|}{|\hat{v}^I(\bar{\mathbf{x}})|} v_{\max}, \right. \\ \left. \lambda \frac{\partial \xi^I}{\partial \psi} \frac{\|\mathbf{e}(\bar{\mathbf{x}})\|}{|\hat{v}^I(\bar{\mathbf{x}})| |\vartheta(\psi)|} \omega_{\max} \right) \quad (61)$$

$$k_\omega(\bar{\mathbf{x}}) = \min \left(k_{\omega, \text{nom}}, (1 - \lambda) \frac{\partial \xi^I}{\partial \psi} \frac{\omega_{\max}}{|\hat{\omega}^I(\bar{\mathbf{x}})|} \right) \quad (62)$$

with $k_{v, \text{nom}}, k_{\omega, \text{nom}} > 0$ initially provided nominal gains and $\lambda \in (0, 1)$ a tuning parameter. It can be seen that $k_v(\bar{\mathbf{x}})$, $k_\omega(\bar{\mathbf{x}})$ are always positive because $\|\mathbf{e}(\bar{\mathbf{x}})\|$, $\frac{\partial \xi^I}{\partial \psi}$ are always positive.

6. Online reactive planning algorithms

With the description of the diffeomorphism construction and the overall hybrid controller, we are now ready to describe the algorithm we use during execution time to generate our control inputs. As shown in Figure 2 that summarizes the whole architecture, we divide the main algorithm that communicates with the semantic mapping and the perception pipelines¹² in two distinct components. First, the *mapped space recovery* component, described in Section 6.1, is responsible for keeping track of all encountered objects, and extracting the sets of obstacles $\mathcal{D}_{\text{map}}^I$, $\mathcal{B}_{\text{map}}^I$. Next, the *reactive planning* component, described in Section 6.2, uses the input from the mapped space recovery

component to generate the diffeomorphism $\mathbf{h}^I$ (described in Section 4) between the mapped space and the model space during execution time, and provide the commands for the robot according to the hybrid controller (described in Section 5).

6.1. Mapped space recovery

Given as input the aggregated set of localized, recognized familiar obstacles $\tilde{\mathcal{P}}_I$, we first dilate all these elements of $\tilde{\mathcal{P}}_I$ by the robot radius r , to form the components of $\mathcal{P}_{\text{sem}}^I$, and consolidate the connected components resulting from their union into a new set of merged obstacles to form $\mathcal{P}_{\text{map}}^I$. Then, for each connected component P of $\mathcal{P}_{\text{map}}^I$ that intersects the boundary of the enclosing freespace $\mathcal{F}_e$, we take $B = P \cap \mathcal{F}_e$, as described in Section 3.3, and include B in the list of obstacles to be merged into $\partial \mathcal{F}_e$, $\mathcal{B}_{\text{map}}^I$; the rest of the components of $\mathcal{P}_{\text{map}}^I$ are included in the list of obstacles to be deformed into disks, $\mathcal{D}_{\text{map}}^I$.¹³ All these computational steps rely on underlying polygon operations (unions, intersections, differences); the development of such algorithms has been heavily explored in the computational geometry literature (Clementini et al., 1993; Douglas and Peucker, 1973; Egenhofer and Herring, 1991), and here we rely on their efficient implementations, either in the open-source C++ Boost library (Schling, 2011), or in the open-source Shapely package (Shapely, 2019) in Python.

The next step is to triangulate every obstacle P_i in both $\mathcal{D}_{\text{map}}^I$ and $\mathcal{B}_{\text{map}}^I$ using the Ear Clipping Method, find its root triangle r_i and extract the corresponding tree of triangles $\mathcal{T}_{P_i} := (\mathcal{V}_{P_i}, \mathcal{E}_{P_i})$, as described in Section 4.1. For the implementation of the Ear Clipping Method, we use either the open-source Boost library (Schling, 2011), for our C++ implementation, or the open-source tripy package (linuxlewis, 2019), for our Python implementation.

The final operation of the mapped space recovery algorithm is to extract the admissible centers of transformation, $\mathbf{x}_j^*$, according to Definitions 2–7, the corresponding radius of transformation, ρ_i (if $P_i \in \mathcal{D}_{\text{map}}^I$), and the admissible polygonal collars, $\overline{\mathcal{Q}}_j$ for all triangles $j \in \mathcal{V}_{P_i}$ and polygons P_i in $\mathcal{D}_{\text{map}}^I$ and $\mathcal{B}_{\text{map}}^I$. There is not a unique method of performing this operation, and we provide our implemented method along with other details in Appendix C. The mapped space recovery algorithm is summarized in Algorithm 1.

6.2. Reactive planning component

The mapped space recovery algorithm described above just informs the robot about its surroundings, by post-processing aggregated information from the semantic mapping pipeline. In this section, we describe the algorithm for generating actual robot inputs, that closes our control loop.

Given the robot state in the mapped space ($\mathbf{x}$ for a fully actuated robot or $\bar{\mathbf{x}}$ for a differential drive robot), and the list of obstacles in the mapped space, $\mathcal{D}_{\text{map}}^I$, $\mathcal{B}_{\text{map}}^I$, along with their associated triangulation trees and their properties

Algorithm 1 Derivation of the sets of obstacles $\mathcal{D}_{\text{map}}^{\mathcal{I}}$, $\mathcal{B}_{\text{map}}^{\mathcal{I}}$, used in the diffeomorphism construction, and their associated properties, from the aggregated list of known obstacles in the physical space $\tilde{\mathcal{P}}_{\mathcal{I}}$.

Function MappedSpaceRecovery $\tilde{\mathcal{P}}_{\mathcal{I}}$
 $\mathcal{P}_{\text{map}}^{\mathcal{I}} \leftarrow \text{Union}(\text{dilate}(\tilde{\mathcal{P}}_{\mathcal{I}}, r))$
do
 $P \leftarrow \text{pop}(\mathcal{P}_{\text{map}}^{\mathcal{I}})$ $\triangleright$ Pop next component
if $P \cap \partial\mathcal{F}_e \neq \emptyset$ **then**
 $B.\text{geometry} \leftarrow P \cap \mathcal{F}_e$
 $B.\text{tree} \leftarrow \text{EarClipping}(B.\text{geometry})$
 Find root of tree $B.\text{root}$ as in Section 4.1
 Restructure $B.\text{tree}$ around $B.\text{root}$
for $j \in B.\text{tree.vertices}$ **do** $\triangleright$ Definitions 2–7
 $B.\text{tree.vertices}(j).\text{append}(\mathbf{x}_j^*)$
 $B.\text{tree.vertices}(j).\text{append}(\bar{\mathbf{Q}}_j)$
end for
 $\mathcal{B}_{\text{map}}^{\mathcal{I}}.\text{append}(B)$
else
 $D.\text{geometry} \leftarrow P$
 $D.\text{tree} \leftarrow \text{EarClipping}(D.\text{geometry})$
 Find root of tree $D.\text{root}$ as in Section 4.1
 Restructure $D.\text{tree}$ around $D.\text{root}$
for $j \in D.\text{tree.vertices}$ **do** $\triangleright$ Definitions 2–7
 $D.\text{tree.vertices}(j).\text{append}(\mathbf{x}_j^*)$
 $D.\text{tree.vertices}(j).\text{append}(\bar{\mathbf{Q}}_j)$
end for
 Find $\rho = D.\text{radius}$ as in Section 4.3
 $\mathcal{D}_{\text{map}}^{\mathcal{I}}.\text{append}(D)$
end if
while $\mathcal{P}_{\text{map}}^{\mathcal{I}} \neq \emptyset$
return $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$
end function

as computed with Algorithm 1, the first step of the reactive planning algorithm is to compute the state of the robot in the model space ($\mathbf{h}^{\mathcal{I}}(\mathbf{x})$ for a fully actuated robot (27) or $\bar{\mathbf{h}}^{\mathcal{I}}(\bar{\mathbf{x}})$ for a differential drive robot (40)), the diffeomorphism Jacobian $D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}$, and partial derivatives of the terms of the jacobian $\frac{\partial[D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}]_{ij}}{\partial[\mathbf{x}]_k}$ (needed in (52) for a differential drive robot), following the methods outlined in Section 4. We show in Appendix D how to perform this operation inductively, given the general form of $\mathbf{h}^{\mathcal{I}}$ in (27).

Next, we need to properly populate the model space with obstacles, in order to compute the input (59) for a fully actuated robot, or the inputs (52) (using (61) and (62)) for a differential drive robot. This procedure is straightforward for familiar obstacles; obstacles in $\mathcal{B}_{\text{map}}^{\mathcal{I}}$ are not taken into account in the model space, because they are merged into the boundary $\partial\mathcal{F}_e$, and obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ are represented in the model space as disks with radius ρ_i centered at $\mathbf{x}_i^*$, with i spanning the elements of $\mathcal{D}_{\text{map}}^{\mathcal{I}}$. For unknown obstacles in $\mathcal{C}_{\text{map}}$, we use the LIDAR measurements (see Figure 2). Namely, we first pre-process the 2D LIDAR pointcloud by disregarding points that correspond to obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ or $\mathcal{B}_{\text{map}}^{\mathcal{I}}$, because those have already been considered. The

Algorithm 2 Description of the online reactive planning module that uses the state of the robot, LIDAR input, and $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$.

function ReactivePlanning (State, LIDAR, $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$)
if RobotType is FullyActuated **then**
 $\mathbf{x} \leftarrow \text{State}$
 $\mathbf{y} \leftarrow \mathbf{h}^{\mathcal{I}}(\mathbf{x})$ $\triangleright$ Section 4, Appendix D
 Compute $D_{\mathbf{x}}\mathbf{h}^{\mathcal{I}}$ $\triangleright$ Appendix D
 Populate $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ using LIDAR, $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$
 Construct $\mathcal{LF}(\mathbf{y})$ $\triangleright$ (37)
 Compute input $\mathbf{v}^{\mathcal{I}}$ $\triangleright$ (36)
 Compute input $\mathbf{u}^{\mathcal{I}}$ $\triangleright$ (59)
 RobotInput $\leftarrow \mathbf{u}^{\mathcal{I}}$
else if RobotType is DiffDrive **then**
 $\bar{\mathbf{x}} \leftarrow \text{State}$
 $\bar{\mathbf{y}} \leftarrow \bar{\mathbf{h}}^{\mathcal{I}}(\bar{\mathbf{x}})$ $\triangleright$ Section 5.2, (40), Appendix D
 Compute $D_{\bar{\mathbf{x}}}\bar{\mathbf{h}}^{\mathcal{I}}, \frac{\partial[D_{\bar{\mathbf{x}}}\bar{\mathbf{h}}^{\mathcal{I}}]_{ij}}{\partial[\bar{\mathbf{x}}]_k}$ $\triangleright$ Appendix D
 Populate $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ using LIDAR, $\mathcal{D}_{\text{map}}^{\mathcal{I}}, \mathcal{B}_{\text{map}}^{\mathcal{I}}$
 Construct $\mathcal{LF}(\bar{\mathbf{y}})$ $\triangleright$ (37)
 Compute inputs $\bar{\mathbf{v}}^{\mathcal{I}}$ $\triangleright$ (53)
 Compute input $\bar{\mathbf{u}}^{\mathcal{I}}$ $\triangleright$ (52) using (61) and (62)
 RobotInput $\leftarrow \bar{\mathbf{u}}^{\mathcal{I}}$
end if
return RobotInput
end function

pointcloud with the remaining points is then transferred with an identity transform to the model space; this is allowed because, by construction, the diffeomorphism $\mathbf{h}^{\mathcal{I}}$ between $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ defaults to the identity transform (i.e., $\mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \mathbf{x}$) on the boundary of any unknown obstacle, provided that this obstacle is sufficiently separated from any obstacle in $\mathcal{P}_{\text{map}}^{\mathcal{I}}$ (see Assumption 2).

With the (“virtual”) model space constructed, we can then construct the local freespace (37), as in Arslan and Koditschek (2016b: Equation (24)), and, subsequently, compute the input $\mathbf{v}^{\mathcal{I}}$ (36) for a fully actuated robot or the inputs $\bar{\mathbf{v}}^{\mathcal{I}}$ (53) for a differential drive robot. The final step is to compute the “pull-backs” of these inputs in the physical space and enforce bounds, by using (59) for a fully actuated robot, or (52) along with (61) and (62) to adaptively modify the input gains for a differential drive robot. The reactive planning module functionality is summarized in Algorithm 2.

It must be highlighted that the presented reactive planning pipeline (summarized in Figure 2) runs at 10 Hz online and onboard our physical robots’ Nvidia Jetson TX2 modules, during execution time.

7. Numerical results

In this section, we present numerical simulations that illustrate our formal results. Our simulations are run in MATLAB using `ode45`, and $p = 20$ for the R-function construction, as described in Appendix B. Our mapped space recovery (Section 6.1) and reactive planning (Section 6.2) algorithms are implemented in Python and

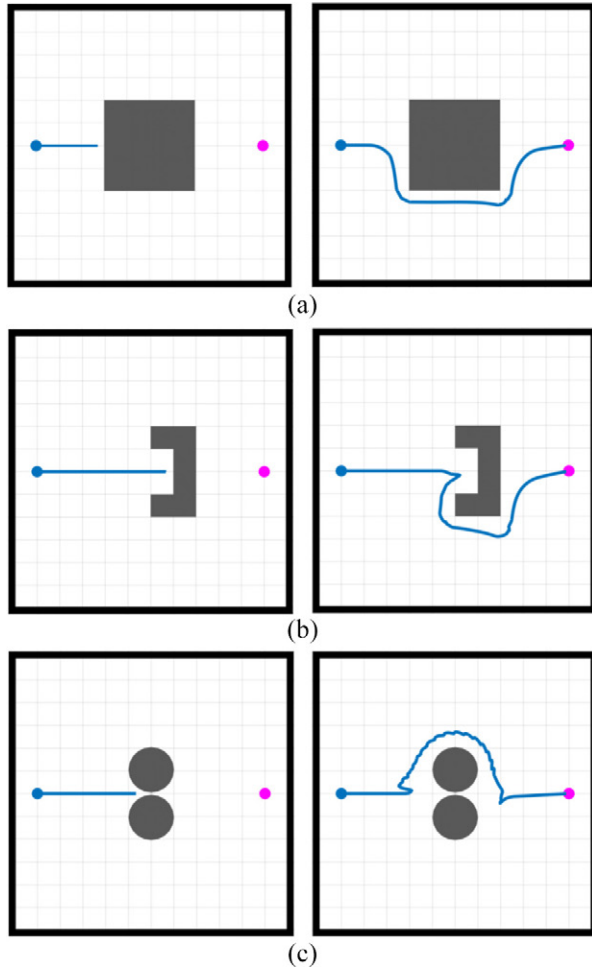


Fig. 7. Comparison with original doubly reactive algorithm for a fully actuated robot (blue) navigating towards a goal (purple): (a) convex obstacle with flat surfaces, (b) non-convex obstacle, and (c) convex obstacles violating the separation assumptions of Arslan and Koditschek (2016b). Left column: Original doubly reactive algorithm (Arslan and Koditschek, 2016b). Right column: Our algorithm.

communicate with MATLAB using the standard MATLAB–Python interface. For our numerical results, we assume perfect robot state estimation and localization of obstacles, using a fixed range sensor that can instantly identify and localize either the entirety of familiar obstacles that intersect its footprint, or the corresponding fragments of unknown obstacles within its range.

7.1. Comparison with original doubly reactive algorithm

We begin with a comparison of our algorithm performance with the original version of the doubly reactive algorithm in Arslan and Koditschek (2016b)), that we use in the model space computed at each instant from the perceptual inputs as depicted in Figure 2(e) and described in Section 6.2. Figure 7 (Extension 1) demonstrates the well-understood limitations of this algorithm (limitations of all online

(Borenstein and Koren, 1991) or offline (Filippidis and Kyriakopoulos, 2012) reactive schemes we are aware of). Namely, in the presence of a flat surface or a non-convex obstacle, or when separation assumptions are violated, the robot gets stuck in undesired local minima, which are locally stable and trap a set of initial conditions whose area becomes arbitrarily large as their “shadows” (i.e., the corresponding basins of attraction) grow; see, e.g., Vasilopoulos et al. (2020: Figure 6(b)). Absent the new methods introduced in this article, handling these spurious basins of attraction would require complex dynamic replanning algorithms (Reverdy et al., 2015; Revzen et al., 2012), whose presentation falls beyond the scope of the present article. In contrast, our algorithm overcomes this limitation, by recourse to the robot’s ability to recognize obstacles at hand (documented empirically in Section 9) and transform them appropriately (as detailed in Section 4) for both a fully actuated and a differential drive robot. The robot radius used in our simulation studies is 0.2 m, the control gains are $k = k_v = k_\omega = 0.4$, and the values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{ri}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{ri}}$ and $\epsilon_{ji} = \epsilon_{ri}$ used in the diffeomorphism construction are 4.0, 0.05, and 2.0, respectively. Finally, the maximum input $u_{\max}$ for the fully actuated robot as well as the maximum linear and angular inputs $v_{\max}, \omega_{\max}$ for the differential drive robot are limited to 0.4.

7.2. Navigation in a cluttered environment with obstacle merging

For the next set of numerical studies, we focus on environments cluttered with several instances of the same familiar obstacle, in different, *a priori* unknown poses. We illustrate the concept in Figure 8 (Extension 1). The robot abstracts away the familiar geometry to explore the unknown topology of the workspace online during execution time. In this particular example, the robot first adopts the hypothesis that an “opening” exists above the initially observed obstacle. With the observation and instantiation of the second obstacle in the semantic map, it is then capable of correcting this hypothesis by merging the obstacle to the boundary of $\mathcal{F}_e$. The properties of the hybrid controller presented in Section 5 guarantee convergence to the goal for both the fully actuated and the differential drive robot, as shown in Figure 9 and the accompanying Extension 1.

We further illustrate the scope of formal results by presenting numerical simulations where the constellation of fixed obstacles incurs the need for multiple mergings between obstacles or between obstacles and the boundary of the enclosing freespace $\mathcal{F}_e$. As guaranteed, both the fully actuated (Theorem 3) and the differential drive (Theorem 4) robots converge to the desired goal from a variety of initial conditions (all but a set of measure zero must converge), as shown in Figure 10 (Extension 1). The robot radius used in our simulations is 0.25 m, the control gains are $k = k_v = k_\omega = 0.4$, and the values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{ri}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{ri}}$ and $\epsilon_{ji} = \epsilon_{ri}$ used in the diffeomorphism

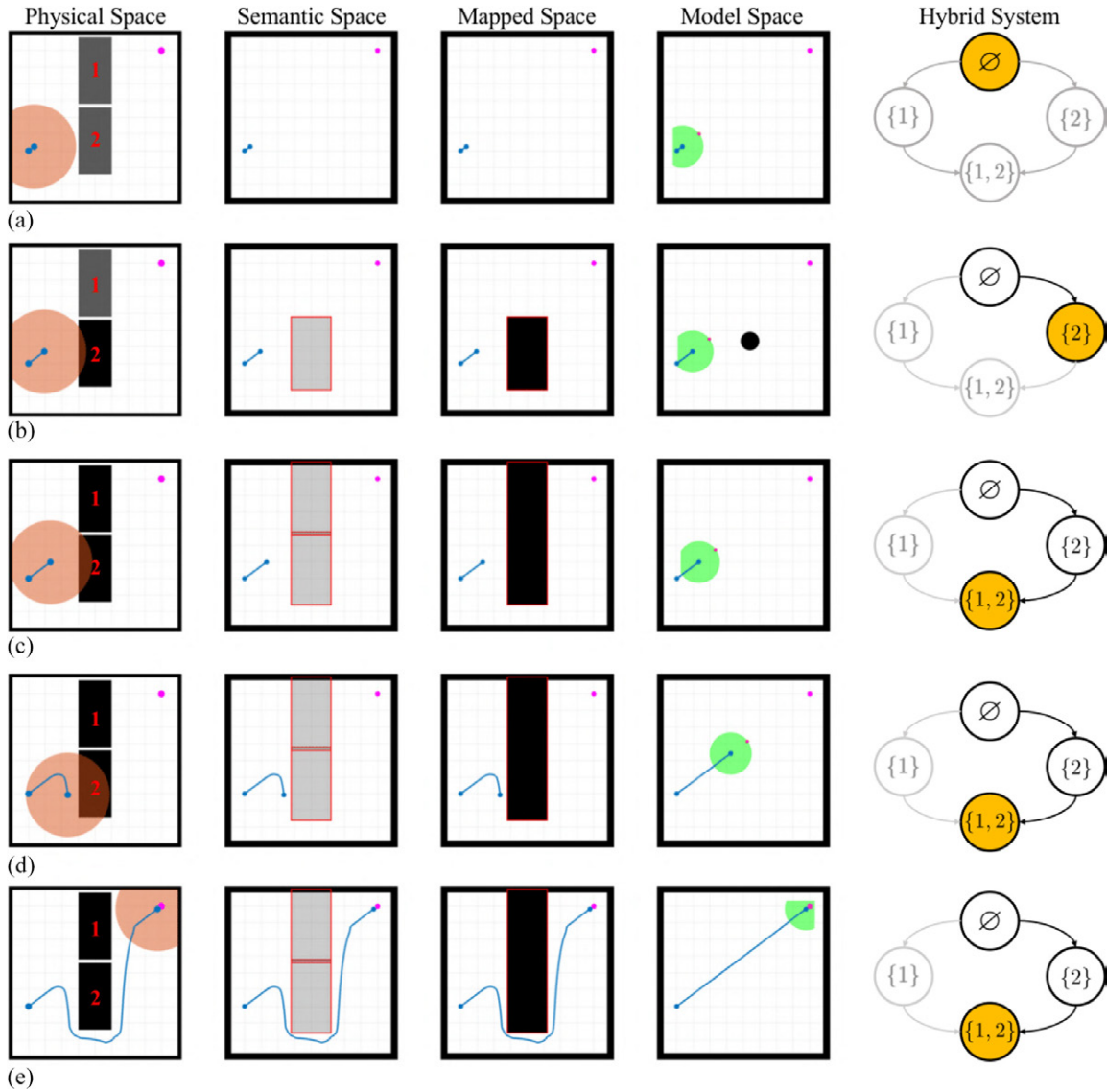


Fig. 8. Illustration of the algorithm with successive snapshots of a single simulation run in the presence of two familiar obstacles with *a priori* unknown pose. (a) The robot starts navigating towards the goal with no prior information about its environment. The initial mode of the hybrid controller is $\mathcal{I} = \emptyset$. (b) The robot discovers the first familiar obstacle (labeled 2 as shown in the physical space), driving the hybrid dynamical system (Section 5) into mode $\mathcal{I} = \{2\}$, wherein it makes an (incorrect) hypothesis about the topological state of the workspace (shown in the mapped space). The robot now computes according to Arslan and Koditschek (2016b) the model control input in the topological model space (shown in the fourth column). (c) The robot discovers the second familiar obstacle (labeled 1 in the physical space), driving the hybrid dynamical system into the terminal (Definition 8) mode $\mathcal{I} = \{1, 2\}$, wherein it corrects the initial hypothesis by merging the union of the two obstacles to the boundary. (d) The reactive field pushing the robot along a direct path to the goal in the unobstructed model space is deformed to generate a sharp correction of course in the geometrically accurate mapped space until, finally, (e) safely navigates to the goal. The deformation of space that aligns the geometrically informed mapped space with its topologically equivalent model space can be visualized by comparing the direct path to the goal the planner generates in the model space with its diffeomorphic image, the curved path connecting the robot's starting point to the goal in the mapped space. Note that the robot has no prior information about the structure of the hybrid system (depicted in the right-most column with unexplored modes in gray): it is driven around the hybrid graph, Γ , by its online perceptual experiences as it accumulates more information about its surroundings. Note, as well, that the cardinality of topological obstacles (the number of punctures in the model space) is independent of the number of semantically localized objects, $|\mathcal{I}|$.

construction are 2.0, 0.05, and 1.0, respectively. The maximum input $u_{\max}$ for the fully actuated robot as well as the maximum linear and angular inputs $v_{\max}$, $\omega_{\max}$ for the differential drive robot are limited to 0.4.

7.3. Navigation among mixed known and unknown obstacles

Finally, Figure 11 (Extension 1) illustrates the convergence guarantees for both a fully actuated as well as a differential

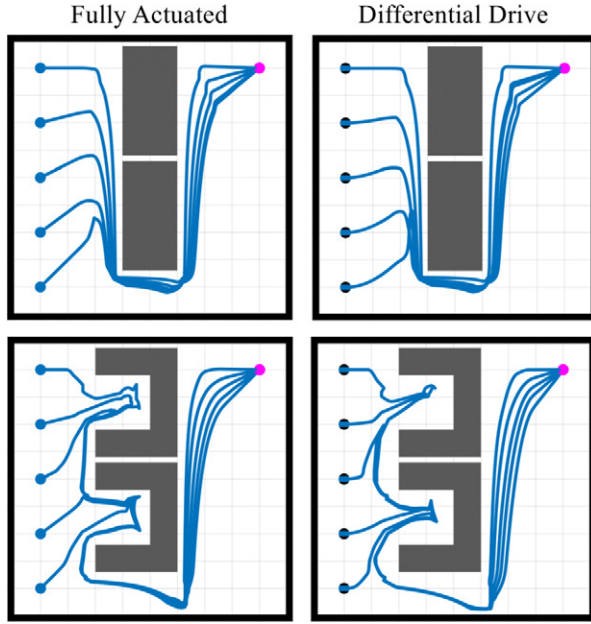


Fig. 9. Numerically simulated illustrations of the navigation planner's behavior from multiple initial conditions for both a fully actuated and a differential drive robot, in the presence of two familiar obstacles with *a priori* completely unknown placement in the workspace. Top: Obstacles with rectangular shape. Bottom: U-shaped obstacles. The hybrid systems theorems presented in Section 5 guarantee the robot will safely navigate to the goal with no collisions along the way.

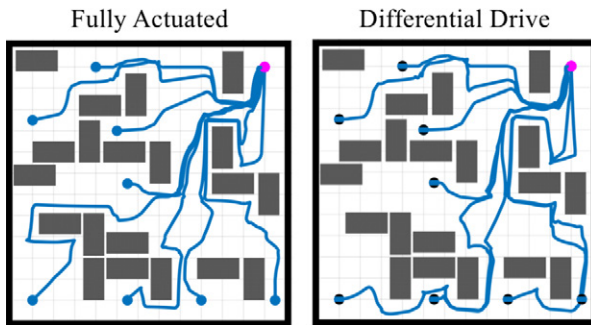


Fig. 10. Simulated trajectories from multiple initial conditions for both a fully actuated and a differential drive robot, in the presence of many instances of the same familiar obstacle with *a priori* unknown pose. The robot explores the geometry and topology of the workspace online during execution time, and the guarantees of the hybrid controller in Section 5 allow it to safely navigate to the goal, without converging to local minima arising from the complicated geometry of the workspace.

drive robot when confronted both by familiar obstacles (with *a priori* unknown pose) as well as completely unknown obstacles (presumed to satisfy the convexity and separation assumptions of Arslan and Koditschek (2016b)), as outlined in Section 2. The robot radius used in our simulations is 0.25 m, the control gains are $k = k_v = k_\omega = 0.4$,

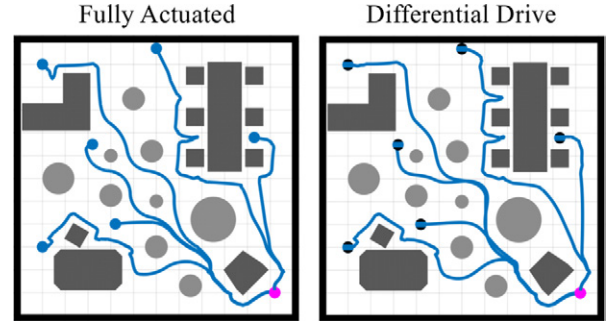


Fig. 11. Simulated trajectories from multiple initial conditions for both a fully actuated robot and a differential drive robot, in the presence of both familiar obstacles with *a priori* unknown pose (dark gray) and completely unknown obstacles (light gray). The guarantees of the hybrid controller in Section 5 allow the robot to always safely navigate to the goal.

and the values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{ri}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{ri}}$, and $\epsilon_{ji} = \epsilon_{ri}$ used in the diffeomorphism construction are 1.6, 0.05, and 0.8 respectively. The maximum input $u_{\max}$ for the fully actuated robot as well as the maximum linear and angular inputs $v_{\max}$, $\omega_{\max}$ for the differential drive robot are limited to 0.4.

8. Experimental setup

Because the reactive planners introduced in this article take the form of first order vector fields (i.e., issuing velocity commands at each state), we use a quasi-static platform, the Turtlebot robot (TurtleBot2, 2019), for the bulk of physical experiments reported next. With the aim of merely suggesting the robustness of these feedback controllers, we also repeat two of those experiments using the highly dynamic Minitaur robot (Ghost Robotics, 2016), whose rough approximation to the quasi-static differential drive motion model is adequate to yield nearly indistinguishable navigation behavior. In the conclusion, we list further factors highlighting the importance of this legged implementation by sketching a longer term agenda emerging from recent results in Vasilopoulos et al. (2018) for integrating this planner into a more complicated system that achieves mobile manipulation tasks with legged robots in dynamic environments.

The experimental setups for our robots are depicted in Figure 12. In both cases, the main computer is an Nvidia TX2 GPU unit (NVIDIA, 2019), responsible for running our mapped space recovery and reactive planning algorithms online, during execution time, according to Figure 2. The GPU unit communicates with a Hokuyo LIDAR (Hokuyo, 2019), used to detect unknown obstacles, and a ZED Mini stereo camera (StereoLabs, 2019b), used for visual-inertial state estimation and for detecting familiar obstacles. As shown in Figure 2, we choose to run our perception and semantic mapping pipelines described next

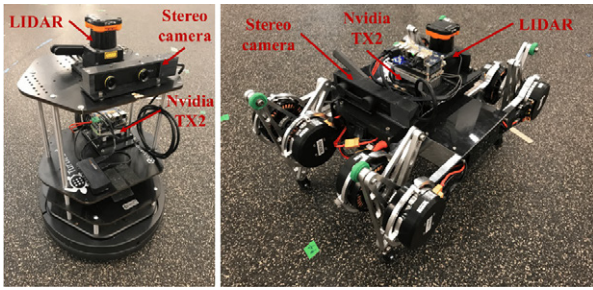


Fig. 12. The platforms used in our experiments: (left) Turtlebot; (right) Minitaur, equipped with a Hokuyo LIDAR for avoidance of unknown obstacles, a stereo camera for object recognition and visual odometry, and an NVIDIA TX2 GPU module as the main onboard computer.

either *onboard* (using the same Nvidia TX2 GPU unit) or *offboard* (on a desktop computer with an Nvidia GeForce RTX 2080 GPU), for faster inference and improved performance. We also assume that the differential drive robot model, presented in (32), is the most suitable motion model for both robots. This is indeed the case for Turtlebot, and an extensive discussion on the empirical anchoring (Full and Koditschek, 1999) of the unicycle template on Minitaur is included in Vasilopoulos et al. (2017).

As the main focus of this article is not the development of new perception or state estimation algorithms, but rather the development of a provably correct planning architecture for partially known environments, we rely to as great an extent as possible on off-the-shelf perception algorithms, implemented in ROS (Quigley et al., 2009), and couple them with our motion planner for the hardware experiments. We are further motivated by the intent for our accompanying software to be modular and easily integrated to existing perception pipelines for future users. We briefly describe the perception and semantic mapping algorithms employed in this article in the following sections, and refer the reader once more to the summary illustration of the whole navigation stack in Figure 2.

8.1. Object detection and keypoint localization

The pipeline we use to detect the objects in the scene and extract the geometric properties needed in order to estimate their 3D pose relies on Pavlakos et al. (2017). The two components involved in this procedure are:

- object detection, which returns 2D bounding boxes for each object;
- keypoint localization, which estimates the 2D locations for a set of predefined keypoints for the specific object instance and class.

The algorithm is described in detail in Pavlakos et al. (2017), but here we give a brief overview of each step in Sections 8.1.1 and 8.1.2, and provide training details for our neural networks in Section 8.1.3.

8.1.1. Object detection. For the task of object detection, we only require the estimation of a 2D bounding box for each object that is visible on the image. We use the YOLOv3 detector (Redmon and Farhadi, 2018) which offers a good trade-off between detection accuracy and inference speed. Given a single RGB image as input, the output of the detector is a 2D bounding box for each object instance, along with the estimated class for this bounding box.

8.1.2. Keypoint localization. For the keypoint localization task, we use a convolutional neural network to accurately estimate the 2D location of the keypoints within the object's bounding box. The keypoints are defined on the 3D model of the object and are selected in advance for each object instance. The keypoint localization network uses as input an RGB image of a specific object, which is cropped using the bounding box information from the detection step. The output of the network is a set of 2D heatmaps. Assuming we select k keypoints for an object, each heatmap is responsible for the localization of the corresponding keypoint. To estimate the locations $\mathbf{W} \in \mathbb{R}^{2 \times k}$ of the k keypoints on the image, we use the 2D heatmap location with the maximum activation for each heatmap as the detected location for the corresponding keypoint. We also consider the value of the activation at this location as the detection confidence d_i for keypoint i . The architecture for this network follows the Stacked Hourglass design (Newell et al., 2016). In practice, we train a single network for all objects of interest and at test time we use only the heatmaps for the specific class, which is already known from the detection step.

8.1.3. Training details. The aforementioned neural networks are trained to detect a predefined set of object instances visualized in Figure 13. The object classes represented for our experiments are chair, table, ladder, cart, gascan, and pelican case. Our goal is to include a variety of instances in terms of the size, shape and visual appearance, in an attempt to simulate the variety of objects that can be encountered in a partially familiar environment. The training data for the particular instances of interest are collected with a semi-automatic procedure, similarly to Pavlakos et al. (2017). Given the bounding box and keypoint annotations for each image, the two networks were trained with their default configurations until convergence.

8.2. Semantic mapping

Our semantic mapping infrastructure relies on the algorithm presented in Bowman et al. (2017), and implemented in C++ using GTSAM (Dellaert, 2012) and its iSAM2 implementation (Kaess et al., 2012) as the optimization back-end. Briefly, this algorithm fuses inertial information (here simply provided by the position tracking implementation from StereoLabs on the ZED Mini stereo camera (StereoLabs, 2019a)), and semantic information (i.e., the detected keypoints and the associated object labels as described in Section 8.1) to provide a posterior estimate for

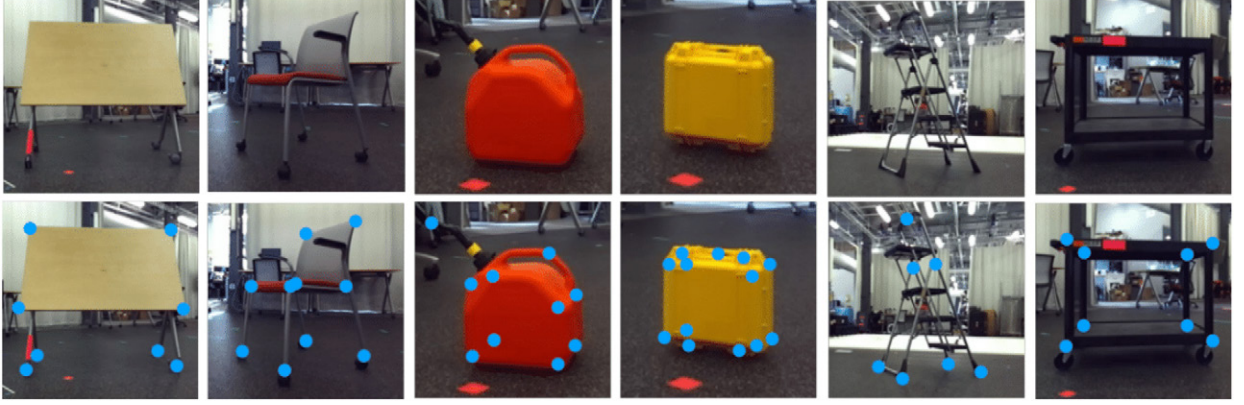


Fig. 13. Top row: Objects used in our experimental setup, table, chair, gascan, pelican case, ladder, and cart. Bottom row: Visualization of the semantic keypoints for each object class.

both the robot state and the associated poses for all tracked objects, by simultaneously solving the data association problem arising when several objects of the same class exist in the map. As described in Bowman et al. (2017), except for providing an estimate for all poses tracked in the environment, this algorithm facilitates loop-closure recognition based on viewpoint-independent semantic information (i.e., tracked objects), rather than low-level geometric features such as points, lines, or planes.

For a single frame detection, the 3D pose of each object with respect to the camera is recovered using the estimated 2D locations of the associated object keypoints. By denoting with $\mathbf{S} \in \mathbb{R}^{3 \times k}$ the 3D locations of the keypoints in the canonical pose of an object instance with k keypoints, the goal is to estimate the rotation $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ and translation $\mathbf{T} \in \mathbb{R}^{3 \times 1}$ of the object, such that the distance of the projected 3D keypoints from their corresponding detected 2D locations is minimized. To incorporate the detection confidence for each keypoint in the optimization, we define the matrix $\mathbf{D} \in \mathbb{R}^{k \times k}$. This is a diagonal matrix that features the detection confidences d_i for each keypoint i in its diagonal. The optimization problem is then formulated as

$$\min_{\mathbf{R}, \mathbf{T}} \frac{1}{2} \left\| (\tilde{\mathbf{W}}\mathbf{Z} - \mathbf{R}\mathbf{S} - \mathbf{T}\mathbf{I}^\top) \mathbf{D}^{\frac{1}{2}} \right\|_F^2 \quad (63)$$

where $\tilde{\mathbf{W}} \in \mathbb{R}^{3 \times k}$ represents the normalized homogeneous coordinates of the 2D keypoints and $\mathbf{Z} \in \mathbb{R}^{k \times k}$ is a diagonal matrix, that features the depths z_i for each keypoint i in its diagonal.

After the estimation of the object's 3D pose from a *single frame measurement* as described above, the 3D positions of its corresponding semantic keypoints are then independently tracked and the object's pose is appropriately updated, as more frame measurements are added. Once a sufficient number of frame measurements¹⁴ has been incorporated so that the 3D keypoint positions can be triangulated, the object is considered to be *localized* and is permanently added to the map. The reader is referred to

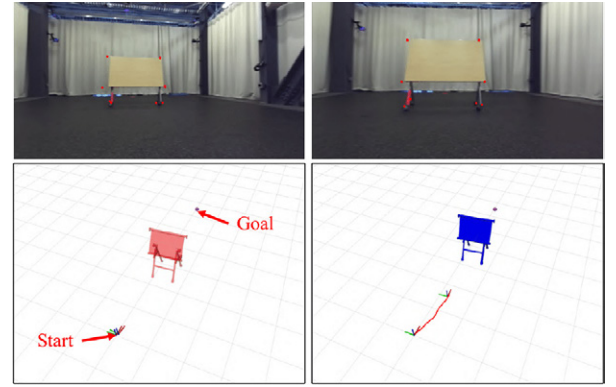


Fig. 14. Illustration of the object localization process using the semantic mapping pipeline from Bowman et al. (2017). Left: The robot starts navigating toward its goal and discovers a familiar obstacle (table). The obstacle is temporarily included in the semantic map, after its 3D pose is estimated using a single frame measurement (65) (red). Right: Once a sufficient number of frame measurements has been incorporated and the 3D pose has been accordingly updated, the object is permanently localized and included in the semantic map (blue).

Bowman et al. (2017) for more details. Figure 14 shows an example of this localization process. It should be noted that for our onboard implementation, where inference using the object detection and keypoint estimation neural networks is slower, we include in the semantic map both the localized objects, after several frame measurements, and objects resulting from a single frame measurement pose estimation, to allow for faster response to sensory input.

As shown in Figure 2, the meshes of the objects in the semantic map, defined by the corresponding keypoint adjacency properties and the extracted 3D pose, are projected on the robot's plane of motion to provide the aggregated list of known obstacles in the physical space $\tilde{\mathcal{P}}_I$, forwarded to our mapped space recovery module (described in Section 6.1). On the other hand, the posterior estimate of

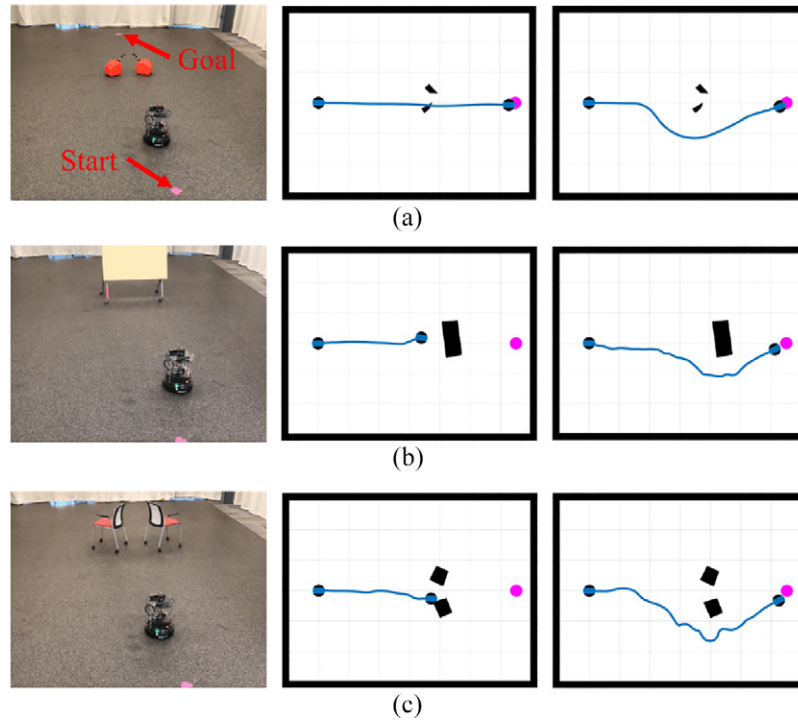


Fig. 15. Physical experiments akin to the numerical simulations depicted in Figure 7, comparing the original doubly reactive algorithm (Arslan and Koditschek, 2016b) (middle column) with our algorithm (right column) in different physical settings (left column), using Turtlebot and offboard perception. (a) Two gascans forming a non-convex trap. (b) Table used as a flat obstacle. (c) Two chairs violating the separation assumptions of (Arslan and Koditschek, 2016b).

the robot pose on the plane, extracted by the semantic mapping module, is forwarded to our reactive planning module (described in Section 6.2).

9. Experimental results

In this section, we provide our experimental results using both the Turtlebot and the Minitaur robot, and the setup described in Section 8. We begin with experiments run using Turtlebot and offboard (Section 9.1) or onboard (Section 9.2) perception, and continue with Minitaur experiments using offboard perception (Section 9.3), to demonstrate the robustness of our method on a more dynamic legged platform. It should be noted that although the perception algorithms, described in Section 8, are run either offboard or onboard, our mapped space recovery and reactive planning modules, described in Algorithms 1 and 2, respectively, are always run onboard each robot's Nvidia TX2 module. The control gains used in our experiments are $k_v = k_\omega = 0.4$, and the maximum linear and angular inputs $v_{\max}, \omega_{\max}$ are set to 0.4. The reader is referred to Extension 2 for more examples and illustrations.

9.1. Experiments with Turtlebot and offboard perception

9.1.1. Comparison with the original doubly reactive algorithm. In this section, we demonstrate experiments similar to the simulations reported in Section 7.1. We first

illustrate various well-understood failures of the original version of the doubly reactive algorithm in (Arslan and Koditschek, 2016b). Collisions result from the presence of short obstacles that cannot be detected by the 2D LIDAR (Figure 15(a)). Confronted by obstacles with flat surfaces (Figure 15(b)), or when separation assumptions are violated (Figure 15(c)), the original algorithm gets stuck in undesired local minima (Figure 15(b) and (c)). In contrast, our new algorithm guarantees safe convergence to the goal in all these cases: short but familiar obstacles (in this case the gascan in column 3 of Figure 13) are recognized by the camera system and localized; once localized, these known geometries can then be appropriately abstracted into the model space (Section 4) which is topologically equivalent but geometrically simplified to meet the requirements of Arslan and Koditschek (2016b). Figure 15 (Extension 2) shows the groundtruth trajectory of the robot, recorded using Vicon, along with 2D projections on the horizontal plane of the obstacles' keypoint meshes, that were used for the construction of the semantic space (Section 3.2). The values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{r_i}}, \mu_{\delta_{ji}} = \mu_{\delta_{r_i}}$ and $\epsilon_{ji} = \epsilon_{r_i}$ used in the diffeomorphism construction are 4.0, 0.05, and 2.0, respectively.

9.1.2. Navigation in a cluttered environment with obstacle merging. We begin the second set of experiments by demonstrating the merging process and the properties of the hybrid controller, reported in Section 5, in a physical

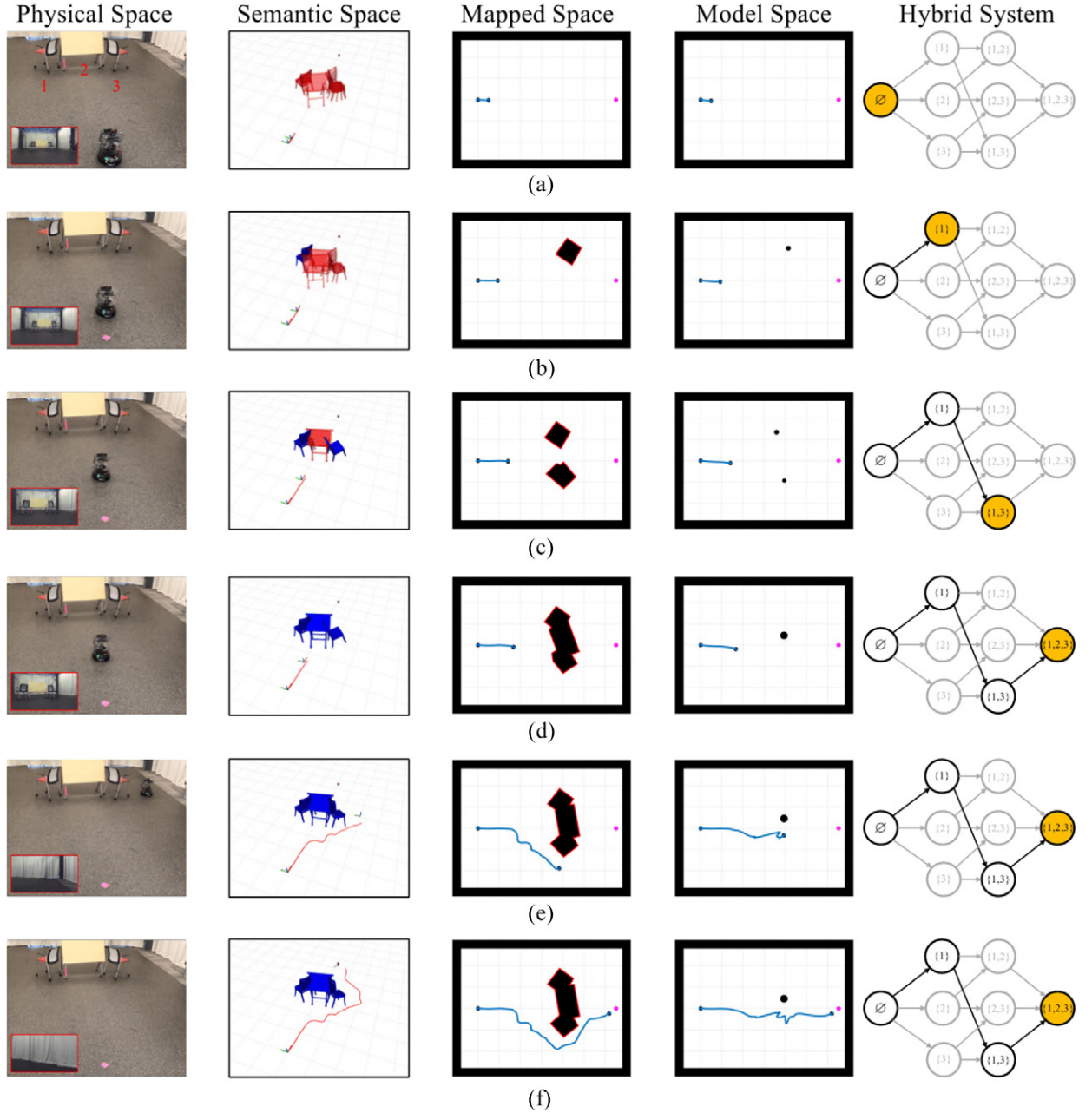


Fig. 16. Illustration of the empirically implemented complete navigation scheme (akin to the numerical simulation depicted in Figure 8) in a physical setting where three familiar obstacles (two chairs and a table) form a non-convex trap. (a) The robot starts navigating toward its designated target in a previously unknown environment, and detects familiar obstacles. The initial mode of the hybrid system is $\mathcal{I} = \emptyset$. (b)–(d) The robot keeps localizing familiar obstacles, and changes its belief about the topological state of the workspace (as evident in the column showing the corresponding model space). (e) Using the information in the semantic space and now being in the terminal (Definition 8) mode $\mathcal{I} = \{1, 2, 3\}$, wherein it has encountered and localized all the environment’s familiar obstacles, the robot is driven by the mapped space transformation (Section 4) of the model space vector field (Arslan and Koditschek, 2016b) to avoid the obstacles, until (f) it converges to the designated goal as guaranteed by the results of Section 5. The right column shows how the robot experiences transitions in the (previously unknown) hybrid system (modes that are never experienced are shown in gray).

setting. As shown in Figure 16 (Extension 2), the robot starts navigating toward its target and localizing obstacles in front of it, until it converges to its target; at the same time, by incorporating more information in its semantic map, it experiences transitions to different modes of the (previously unknown) hybrid system. The values of

$\mu_{\gamma_{ji}} = \mu_{\gamma_{ri}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{ri}}$ and $\epsilon_{ji} = \epsilon_{ri}$ used in this experiment are 4.0, 0.05, and 2.0, respectively.

Finally, Figure 17 (Extension 2) demonstrates navigation in environments cluttered with multiple familiar obstacles. In the first illustration, the robot reactively chooses to navigate through a gap between the gascan and a chair. Despite

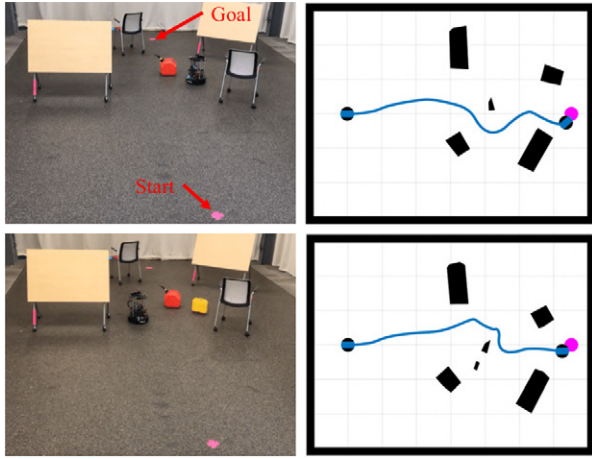


Fig. 17. Navigation among multiple familiar obstacles, using Turtlebot and offboard perception. Top: The robot exploits the gap between the gascan and the chair to safely navigate to the goal. Bottom: When we block this gap by another familiar obstacle (pelican case), the robot reactively chooses to follow another safe and convergent trajectory, by consolidating the semantic triad {gascan, pelican case, chair} into a single, “mapped” obstacle in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$.

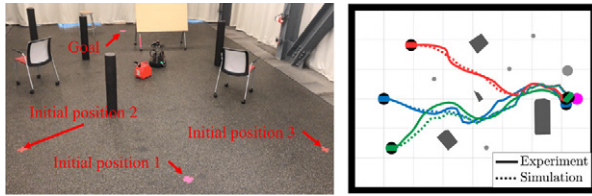


Fig. 18. Navigation among familiar and unknown obstacles, using Turtlebot and offboard perception, from three different initial conditions. Left: A snapshot of the physical workspace. Right: A “bird’s-eye” view of the workspace, with 2D projections of the localized familiar obstacles (dark gray) and unknown obstacles (light gray, groundtruth locations recorded using Vicon), along with groundtruth trajectories from the physical experiments and overlaid numerical simulations in MATLAB.

the blockage of this gap by another familiar obstacle (pelican case) in the second illustration, the robot reactively chooses to follow another safe and convergent trajectory (as guaranteed by the theorems of Section 5), by merging the set gascan–pelican case–chair, and considering them as a single obstacle. The values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{r_i}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{r_i}}$ and $\epsilon_{ji} = \epsilon_{r_i}$ used in this experiment are 1.6, 0.05, and 0.8, respectively.

9.1.3. Navigation among mixed known and unknown obstacles. In the next set of experiments, we consider navigation among multiple familiar and unknown obstacles. Figure 18 (Extension 2) shows that the robot safely converges to the goal from multiple initial conditions, using

vision and the setup described in Section 8 for familiar obstacle detection and localization, and the onboard 2D LIDAR for all the unknown obstacles. In Figure 18, we also overlay trajectories from a MATLAB simulation of a differential drive robot with the same initial conditions and similar control gains; the simulated and physical platform follow similar trajectories in all three cases. The values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{r_i}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{r_i}}$ and $\epsilon_{ji} = \epsilon_{r_i}$ used in these experiments are 2.0, 0.05, and 1.0.

It should be highlighted that even when the object localization process fails, collision avoidance is still guaranteed with the use of the onboard LIDAR. Nevertheless, collisions could result with obstacles that cannot be detected by the 2D horizontal LIDAR (e.g., see Figure 15(a)). One could still think of extensions to the presented sensory infrastructure (e.g., the use of a 3D LIDAR) that could still guarantee safety under such circumstances.

9.2. Experiments with Turtlebot and onboard perception

This section briefly reports on experiments using onboard perception; the reader is referred to Extension 2 for more examples. As described in Section 8.2, here we use both the localized obstacles by the semantic mapping pipeline and raw, not permanently localized obstacles, resulting from a single semantic frame measurement and the optimization problem given in (63). Figure 19 (Extension 2) illustrates an example; the robot detects and avoids the two chairs in front of it, even if they are only temporarily included in the semantic map (in the absence of more frame measurements). The robot then proceeds to localize and avoid the gascan and the two tables and safely converge to the designated goal. The values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{r_i}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{r_i}}$, and $\epsilon_{ji} = \epsilon_{r_i}$ used in this experiment are 2.0, 0.05, and 1.0, respectively.

It should be noted that the object impermanence in the semantic map violates the formal assumptions of Theorems 3 and 4; without permanently localizing an object, the robot could get stuck in an endless loop trying to avoid obstacles that it then “forgets,” in unfavorable workspace configurations (such as those reported in Figure 9).

9.3. Experiments with Minitaur

Finally, Figure 20 (Extension 2) presents illustrative snapshots of two navigation examples on the much more dynamic Minitaur platform. Despite the fact that Minitaur is an imperfect kinematic unicycle and the overall shakiness of the platform, the robot is capable of detecting and localizing familiar obstacles of interest and using that information to safely converge to the target. The values of $\mu_{\gamma_{ji}} = \mu_{\gamma_{r_i}}$, $\mu_{\delta_{ji}} = \mu_{\delta_{r_i}}$, and $\epsilon_{ji} = \epsilon_{r_i}$ used in this experiment are 2.0, 0.05, and 1.0, respectively. The reader is referred to Extension 2 for more examples and illustrations.

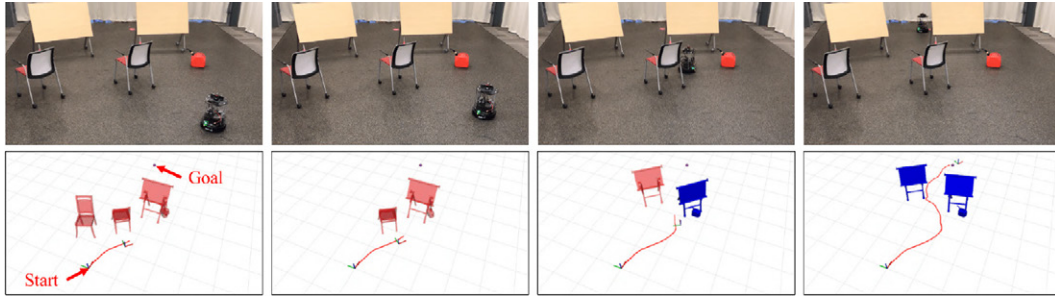


Fig. 19. Navigation among familiar obstacles, using Turtlebot and onboard perception. Top: snapshots of the physical workspace. Bottom: illustrations of the recorded semantic map and the robot's trajectory in RViz (Quigley et al., 2009). The robot detects and avoids the two chairs in front of it, though they are only temporarily included in the semantic map (in the absence of more frame measurements). Then it proceeds to localize and avoid the two tables and the gascan, to safely converge to the goal.

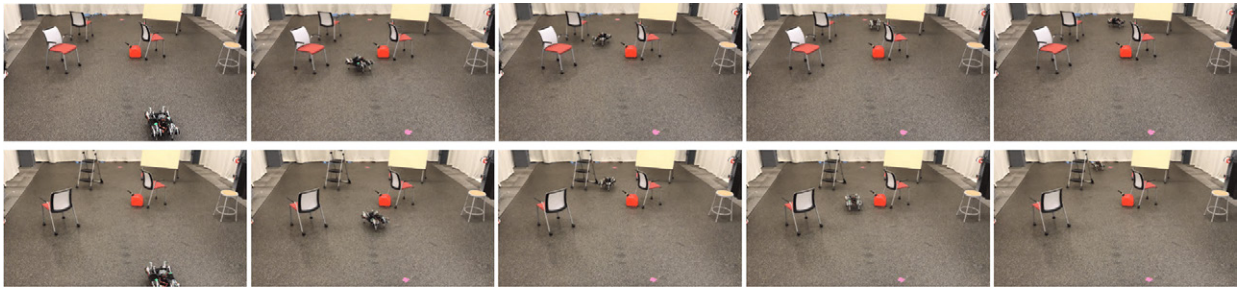


Fig. 20. Snapshots of Minitaur avoiding multiple familiar obstacles in two different settings, using offboard perception.

10. Conclusion and future work

10.1. Conclusion

This article presents a reactive navigation scheme for robots operating in planar workspaces, cluttered with obstacles of familiar geometry but *a priori* unknown placement, and completely unknown, but strongly convex and well-separated obstacles. To the best of the authors' knowledge, this is the first doubly reactive navigation framework (i.e., a scheme where not only the robot's trajectory but also the vector field that generates it are computed online at execution time) that can handle arbitrary polygonal shapes in realtime without the need for specific separation assumptions between the familiar obstacles. The resulting algorithm combines state-of-the-art perception and object recognition techniques (based on neural network architectures) for familiar obstacles, with local range measurements (e.g., LIDAR) for the unknown obstacles, to yield provably correct navigation in geometrically complicated environments. We illustrate the practicability of this approach by reporting empirical results using modest computational hardware on a wheeled robot, and the intrinsic robustness of such reactive schemes by a second implementation on a dynamic legged platform, exhibiting imperfect fidelity to the differential drive model assumed in the formal results.

10.2. Future work

Figures 21 and 22 present snapshots from experiments in settings falling outside the scope of our formal results to

illustrate some of the future directions opened up by the reactive planner presented in this article. Using the feature of semantic inference provided by the semantic mapping pipeline described in Section 8.2, the user can command the robot to target a *semantic* goal, instead of a merely geometric one (considered in this article). Namely, in Figure 21 (Extension 3), we command the Turtlebot robot to move to a geometrically predefined target, unless it sees and localizes a cart; in that case, it is tasked with approaching and facing the cart with its camera. As shown in the bottom row of Figure 21, the robot avoids familiar obstacles, localizes the cart and proceeds to properly approach it, with the right orientation. We take this approach one step further with the example shown in Figure 22 (Extension 3), using the Minitaur platform. Using the mobile manipulation primitives developed in Topping et al. (2019), we task the robot by not only localizing and approaching the cart, but also jumping to grab and mount it. This is a first step toward integrating the reactive planning architecture developed in this work in the multi-layer architecture presented in Vasilopoulos et al. (2018), for accomplishing increasingly complicated mobile manipulation tasks with underactuated legged robots in environments that are semantically partially known and geometrically unknown. Parallel work, relying on the formal guarantees presented in this article, has already demonstrated how the same vector field planning principles and our semantic inference capabilities can be exploited in order to perform more complex missions with predefined logic that involve human following and pose tracking (Vasilopoulos et al., 2020), tasks such as

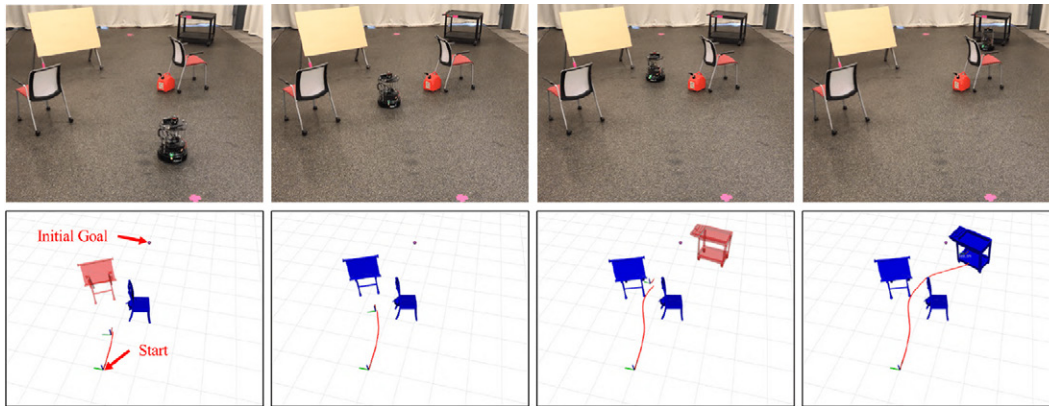


Fig. 21. Navigation toward a semantic target with Turtlebot. The robot is initially tasked with moving to a predefined location, unless it detects and localizes a cart; in that case it has to approach and face the cart. The last column (top, snapshot of the physical workspace; bottom, illustration of the recorded trajectory in RViz) shows that the robot successfully executes the task.

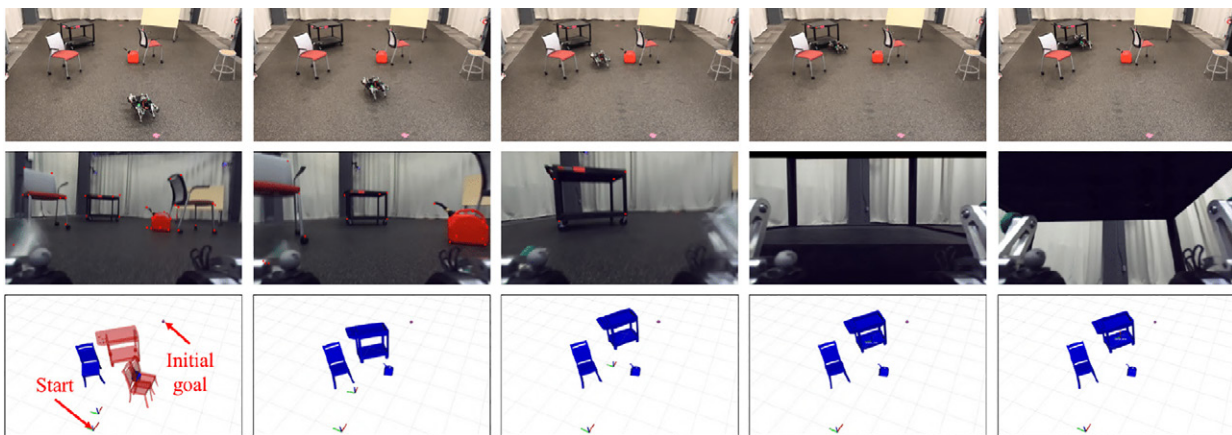


Fig. 22. Using reactive navigation with mobile manipulation primitives on Minitaur. Similarly to Figure 21, the robot is tasked with moving to a predefined location, unless it detects and localizes a cart; in that case it has to approach and jump to mount the cart, using a maneuver from Topping et al. (2019). Top: Recorded snapshots of the physical workspace. Middle: First-person view with semantic keypoints of familiar obstacles shown as red dots. Bottom: RViz illustration of the recorded semantic map.

navigation among movable obstacles (Vasilopoulos et al., 2021), using a linear temporal logic planner (Kantaros et al., 2020) and an interface layer that translates symbolic commands to point navigation tasks, or complicated mobile manipulation tasks with legged robots in unexplored 2.5D environments, using an external geometric planner to rearrange semantically tagged objects of interest (Vega-Brown et al., 2021).

Moreover, a remaining challenge is to generalize the present framework beyond the current restriction to 2D environments, in order to address the challenge of navigating unknown or partially known environments in higher dimension. Even though the currently presented algorithm would be restricted to shapes with genus zero (no holes), one could develop algorithms that “patch” the holes of shapes with non-zero genus when they are not important, affording the use of the same reactive principles for navigation. Work currently in progress investigates whether concepts from the literature on convex decomposition of

polyhedra (Lien and Amato, 2007) could afford such generalization to the problem of navigating 3D workspaces with aerial drones.

Finally, we believe the methods we develop here for generating in realtime simple, topologically equivalent model spaces and pulling back the model controller through the corresponding diffeomorphism can be applied to diverse, philosophically alternative approaches to our purely reactive formulation of motion planning. For example, sampling-based (probabilistically complete) offline planners have been shown to benefit from integration with even geometrically naive locally reactive methods (Arslan et al., 2017) that can mitigate difficulties such as finding paths through narrow passages. We imagine that even greater simplification of the steering and collision-checking issues arising from sampling-based methods in partially “familiar” geometrically complicated environments (Bialkowski et al., 2012; LaValle and Kuffner, 2001) might be achieved by shifting the problem of finding a feasible

path to a topologically equivalent, metrically simple abstracted model wherein planning might be significantly faster. The robot could then be tasked to follow a generated path in the abstract space (e.g., along the lines of Arslan and Koditschek (2017)) and the associated commands can be pulled back to the physical space through the diffeomorphism. Careful future inquiry will be needed to explore such deliberative–reactive hybrid uses for the online topological abstraction of familiar geometry developed here.

Acknowledgments

The authors thank Dr. Omur Arslan for many formative discussions and for sharing his simulation and presentation infrastructure, Prof. Elon Rimon for many interesting discussions pertaining to results from the field of computational geometry, Prof. George Pappas and Sean Bowman for sharing their semantic mapping framework, and T. Turner Topping for assistance with the Minitaur hardware experiments.

Funding

The author disclosed receipt of the following financial support for research, authorship, and/or publication of this article: This work was supported in part by the AFRL (grant number FA865015D1845; subcontract 669737-1), in part by the ONR (grant number N00014-16-1-2817), and by a Vannevar Bush Fellowship held by the last author, sponsored by the Basic Research Office of the Assistant Secretary of Defense for Research and Engineering.

ORCID iDs

Vasileios Vasilopoulos  <https://orcid.org/0000-0003-4681-2410>
Karl Schmeckpeper  <https://orcid.org/0000-0003-4989-2022>

Notes

1. For our hardware implementation, this idealized sensor is reduced to a combination of a LIDAR for distance measurements to obstacles and a monocular camera for object recognition and pose identification.
2. Here, $d(A, B) := \inf\{\|\mathbf{a} - \mathbf{b}\|, \mathbf{a} \in A, \mathbf{b} \in B\}$.
3. Although we make no use in the present article of the discovered unfamiliar objects beyond simply avoiding them, future work now in progress relaxes the convexity requirement to build up in memory an increasingly complete geometric description (treated in the same manner as in the “familiar” case) from whatever subsequent encounters ensue along the way to the goal.
4. This implies that there exists a time $t_P > 0$, such that $\overline{B(\mathbf{x}^{t_P}, R)} \cap \tilde{P} \neq \emptyset$ and $\overline{B(\mathbf{x}^t, R)} \cap \tilde{P} = \emptyset$, for all $t < t_P$, with $\mathbf{x}^t$ denoting the robot position at time t .
5. The star (*) notation in this article is reserved to denote specific positions of interest for the construction of the map between the mapped and the model space, and is not to be confused with notions of optimality.
6. The authors thank Professor Elon Rimon for pointing out these results.
7. If r_i and $\partial\mathcal{F}_e$ share two common edges, we just pick one of them at random.
8. We use the ordered set notation $(*, *, \dots)$ and the matrix notation $[* \quad * \quad \dots]^T$ for vectors interchangeably.

9. This is afforded by the fact that the perceived physical free-space $\mathcal{F}^I$ was explicitly constructed in Section 5.1 to be equal to $\mathcal{F}_{\text{map}}^I$. To be accurate, one must write the identity map from $\mathcal{F}^I$ into $\mathcal{F}_{\text{map}}^I$ as $\iota: \mathcal{F}^I \rightarrow \mathcal{F}_{\text{map}}^I$ and, subsequently, define the control in the physical space as $[D_{\mathbf{x}}\iota]^{-1}\mathbf{u}^I$, with $\mathbf{u}^I: \mathcal{F}_{\text{map}}^I \rightarrow T\mathcal{F}_{\text{map}}^I$ the control strategy in the mapped space, described next. However, because $[D_{\mathbf{x}}\iota]$ resolves to the identity matrix, this construction reduces to direct application of $\mathbf{u}^I$ on the physical space.
10. Here $\Pi_C(\mathbf{q})$ denotes the metric projection of $\mathbf{q}$ on a convex set C .
11. In (40), we construct a diffeomorphism $\bar{\mathbf{h}}^I$ between $\mathcal{F}_{\text{map}}^I \times S^1$ and $\mathcal{F}_{\text{model}}^I \times S^1$. However, for practical purposes, we deal only with one specific chart of S^1 in our control structure, described by the angles $(-\pi, \pi]$. As shown in Astolfi (1999), the discontinuity at $\pm\pi$ does not induce a discontinuity in our controller owing to the use of the atan function in (53b). In contrast, with the use of (53b) as in Astolfi (1999) and Arslan and Koditschek (2016b), the robot never changes heading in $\mathcal{F}_{\text{model}}^I$, which implies that the generated trajectories both in $\mathcal{F}_{\text{model}}^I$ and (by the properties of the diffeomorphism $\bar{\mathbf{h}}^I$) in $\mathcal{F}_{\text{map}}^I$ have no cusps, even though the robot might change heading in $\mathcal{F}_{\text{map}}^I$ because of the more complicated nature of the function ξ^I in (41).
12. For our numerical studies, we simply assume an idealized sensor of fixed range that can instantly recognize and localize obstacles within its range. For our hardware implementation, the semantic mapping and perception pipelines rely mostly on prior work and are briefly described in Section 8.
13. Note as a consequence of these consolidations that the cardinality of the index $\mathcal{I}$ denoting the subset of familiar objects discovered and localized in the semantic space, $\mathcal{P}_{\text{sem}}^I$, will in general be larger than the cardinality of connected components in $\mathcal{P}_{\text{map}}^I$, whose cardinality in turn will generally be larger than that of the connected components in $\mathcal{D}_{\text{map}}^I$. Nevertheless, under the assumption of fixed obstacles, these cardinalities are also fixed functions of $\mathcal{I}$, constant over some fixed subset of robot placements, hence these subsets of semantically identified and localized familiar obstacles, $\mathcal{I} \subseteq \mathcal{N}_{\mathcal{P}}$, comprise the appropriate indices for the modes (i.e., they label the vertices of the graph Γ) of the hybrid system just analyzed in Section 5.1. This situation is illustrated in Section 8.
14. This number depends on the needed camera motion between successive measurements, in order to establish a good baseline for triangulation (Hartley, 1997); in this work, we found that five measurements for the offboard experiments and three measurements for the onboard experiments yielded reasonably fast keypoint localization.
15. In Shapiro (2007), it is assumed that zero is always signed: either $+0$ or -0 , which allows the authors to determine membership of zero either to the set of positive or to the set of negative numbers. This assumption is employed to resolve pathological cases, where the membership of zero causes R-function discontinuities and is not of particular importance in our setting.

References

- Amazon, Inc. (2014) Amazon Prime Air. <http://www.amazon.com/b?node=8037720011> (accessed 12 September 2021).

- Arslan O and Koditschek DE (2016a) Exact robot navigation using power diagrams. In: *IEEE International Conference on Robotics and Automation*, pp. 1–8.
- Arslan O and Koditschek DE (2016b) Sensor-based reactive navigation in unknown convex sphere worlds. In: *The 12th International Workshop on the Algorithmic Foundations of Robotics*.
- Arslan O and Koditschek DE (2017) Smooth extensions of feedback motion planners via reference governors. In: *IEEE International Conference on Robotics and Automation*, pp. 4414–4421.
- Arslan O and Koditschek DE (2018) Sensor-based reactive navigation in unknown convex sphere worlds. *The International Journal of Robotics Research* 38(1–2): 196–223.
- Arslan O, Pacelli V and Koditschek DE (2017) Sensory steering for sampling-based motion planning. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3708–3715.
- Astolfi A (1999) Exponential stabilization of a wheeled mobile robot via discontinuous control. *Journal of Dynamic Systems, Measurement, and Control* 121(1): 121–126.
- Ataka A, Lam H and Althoefer K (2018) Reactive magnetic-field-inspired navigation for non-holonomic mobile robots in unknown environments. In: *IEEE International Conference on Robotics and Automation*, pp. 6983–6988.
- Atanasov N, Zhu M, Daniilidis K and Pappas GJ (2016) Localization from semantic observations via the matrix permanent. *The International Journal of Robotics Research* 35(1–3): 73–99.
- Bansal S, Tolani V, Gupta S, Malik J and Tomlin C (2019) Combining optimal control and learning for visual navigation in novel environments. *arXiv preprint* <https://arxiv.org/abs/1903.02531>.
- Baryshnikov Y and Shapiro B (2014) How to run a centipede: A topological perspective. In: *Geometric Control Theory and Sub-Riemannian Geometry (Springer INdAM Series, Vol. 5.)*. Cham: Springer, pp. 37–51.
- Bhattacharya S, Ghrist R and Kumar V (2015) Persistent homology for path planning in uncertain environments. *IEEE Transactions on Robotics* 31(3): 578–590.
- Bialkowski J, Karaman S, Otte M and Frazzoli E (2012) Efficient collision checking in sampling-based motion planning. In: *The 10th International Workshop on the Algorithmic Foundations of Robotics*.
- Borenstein J and Koren Y (1989) Real-time obstacle avoidance for fast mobile robots. *IEEE Transactions on Systems, Man, and Cybernetics* 19(5): 1179–1187.
- Borenstein J and Koren Y (1991) The vector field histogram - fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation* 7(3): 278–288.
- Bowman SL, Atanasov N, Daniilidis K and Pappas GJ (2017) Probabilistic data association for semantic SLAM. In: *IEEE International Conference on Robotics and Automation*, pp. 1722–1729.
- Brannicky MS (1998) Multiple Lyapunov functions and other analysis tools for switched and hybrid systems. *IEEE Transactions on Automatic Control* 43(4): 475–482.
- Brock O and Khatib O (1999) High-speed navigation using the global dynamic window approach. In: *IEEE International Conference on Robotics and Automation*, pp. 341–346.
- Chaney RW (1990) Piecewise C^k functions in nonsmooth analysis. *Nonlinear Analysis: Theory, Methods and Applications* 15(7): 649–660.
- Chang D and Marsden J (2003) Gyroscopic forces and collision avoidance with convex obstacles. In: *New Trends in Nonlinear Dynamics and Control and their Applications (Lecture Notes in Control and Information Sciences, Vol. 295)*. Berlin: Springer, pp. 145–159.
- Clementini E, Felice PD and van Oosterom P (1993) A small set of formal topological relationships suitable for end-user interaction. In: *Proceedings of the 3rd International Symposium on Advances in Spatial Databases*, pp. 277–295.
- Conner D, Choset H and Rizzi AA (2011) Integrating planning and control for single-bodied wheeled mobile robots. *Autonomous Robots* 30(3): 243–264.
- Dellaert F (2012) Factor graphs and GTSAM: A hands-on introduction. Technical Report GT-RIM-CP&R-2012-002. Available at: <https://research.cc.gatech.edu/borg/sites/edu.borg/files/downloads/gtsam.pdf> (accessed 12 September 2021).
- Douglas DH and Peucker TK (1973) Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica: The International Journal for Geographic Information and Geovisualization* 10: 112–122.
- Egenhofer M and Herring JR (1991) Categorizing binary topological relations between regions, lines, and points in geographic databases. *International Series of Monographs on Computer Science*.
- ElGindy H, Everett H and Toussaint GT (1993) Slicing an ear using prune-and-search. *Pattern Recognition Letters* 14(9): 719–722.
- Farber M (2006) Topology of robot motion planning. In: P Biran, O Cornea and F Lalonde (eds) *Morse Theoretic Methods in Nonlinear Analysis and in Symplectic Topology (NATO Science Series II: Mathematics, Physics and Chemistry, Vol. 217)*. Dordrecht: Springer, pp. 185–230.
- Farber M, Grant M, Lupton G and Oprea J (2019) An upper bound for topological complexity. *Topology and its Applications* 255: 109–125.
- Filippidis IF and Kyriakopoulos KJ (2012) Navigation functions for everywhere partially sufficiently curved worlds. In: *IEEE International Conference on Robotics and Automation*, pp. 2115–2120.
- Fiorini P and Shiller Z (1998) Motion planning in dynamic environments using velocity obstacles. *The International Journal of Robotics Research* 17(7): 760–772.
- Full R and Koditschek DE (1999) Templates and anchors: Neuro-mechanical hypotheses of legged locomotion on land. *Journal of Experimental Biology* 202(23): 3325–3332.
- Gao F, Wu W, Gao W and Shen S (2019) Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments. *Journal of Field Robotics* 36(4): 710–733.
- Garrett CR, Lozano-Perez T and Kaelbling LP (2018) Sampling-based methods for factored task and motion planning. *The International Journal of Robotics Research* 37(13–14): 1796–1825.
- Ghost Robotics (2016) Ghost Robotics Minitaur. <https://www.ghostrobotics.io/about> (accessed 21 September 2021).
- Gupta S, Davidson J, Levine S, Sukthankar R and Malik J (2017) Cognitive mapping and planning for visual navigation. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7272–7281.
- Hartley RI (1997) Triangulation. *Computer Vision and Image Understanding* 68(2): 146–157.
- Henry P, Vollmer C, Ferris B and Fox D (2010) Learning to navigate through crowded environments. In: *IEEE International Conference on Robotics and Automation*, pp. 981–986.

- Hesch JA, Kottas DG, Bowman SL and Roumeliotis SI (2014) Consistency analysis and improvement of vision-aided inertial navigation. *IEEE Transactions on Robotics* 30(1): 158–176.
- Hirsch MW (1976) *Differential Topology*. Berlin: Springer.
- Hokuyo (2019) Scanning Rangefinder UTM-30LX. <https://www.hokuyo-aut.jp/search/single.php?serial=169> (accessed 12 September 2021).
- Ilhan BD, Johnson AM and Koditschek DE (2018) Autonomous legged hill ascent. *Journal of Field Robotics* 35(5): 802–832.
- Jayakumar RP, Madhav MS, Savelli F, Blair HT, Cowan NJ and Knierim JJ (2019) Recalibration of path integration in hippocampal place cells. *Nature* 566(7745): 533–537.
- Johnson AM, Burden SA and Koditschek DE (2016) A hybrid systems model for simple manipulation and self-manipulation systems. *The International Journal of Robotics Research* 35(11): 1354–1392.
- Johnson AM, Hale MT, Haynes GC and Koditschek DE (2011) Autonomous legged hill and stairwell ascent. In: *IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 134–142.
- Kaess M, Johannsson H, Roberts R, Ila V, Leonard JJ and Dellaert F (2012) ISAM2: Incremental smoothing and mapping using the Bayes tree. *The International Journal of Robotics Research* 31(2): 216–235.
- Kantaros Y, Malencia M, Kumar V and Pappas GJ (2020) Reactive temporal logic planning for multiple robots in unknown environments. In: *IEEE International Conference on Robotics and Automation*, pp. 11479–11485.
- Kar A, Tulsiani S, Carreira J and Malik J (2015) Category-specific object reconstruction from a single image. In: *IEEE International Conference on Computer Vision and Pattern Recognition*, pp. 1966–1974.
- Karaman S and Frazzoli E (2012) High-speed flight in an ergodic forest. In: *IEEE International Conference on Robotics and Automation*, pp. 2899–2906.
- Kavraki L, Svestka P, Latombe JC and Overmars MH (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation* 12(4): 566–580.
- Keil JM (1985) Decomposing a polygon into simpler components. *SIAM Journal on Computing* 14(4): 799–817.
- Khatib O (1986) Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research* 5(1): 90–98.
- Kong C, Lin CH and Lucey S (2017) Using locally corresponding CAD models for dense 3D reconstructions from a single image. In: *IEEE International Conference on Computer Vision and Pattern Recognition*, pp. 5603–5611.
- Kumar A, Gupta S, Fouhey D, Levine S and Malik J (2018) Visual memory for robust path following. *Advances in Neural Information Processing Systems* 31: 765–774.
- Kuntz L and Scholtes S (1994) Structural analysis of nonsmooth mappings, inverse functions, and metric projections. *Journal of Mathematical Analysis and Applications* 188(2): 346–386.
- LaValle SM and Kuffner JJ (2001) Randomized kinodynamic planning. *The International Journal of Robotics Research* 20(5): 378–400.
- Lien JM and Amato NM (2007) Approximate convex decomposition of polyhedra. In: *ACM Symposium on Solid and Physical Modeling*. New York: ACM Press, pp. 121–131.
- linuxlewis (2019) TriPy: Simple polygon triangulation algorithms in pure Python. <https://github.com/linuxlewis/tripy>.
- Lionis G, Papageorgiou X and Kyriakopoulos KJ (2008) Towards locally computable polynomial navigation functions for convex obstacle workspaces. In: *IEEE International Conference on Robotics and Automation*, pp. 3725–3730.
- Majumdar A and Tedrake R (2017) Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research* 36(8): 947–982.
- Massey WS (1992) Sufficient conditions for a local homeomorphism to be injective. *Topology and its Applications* 47: 133–148.
- Meisters G (1975) Polygons have ears. *American Mathematical Monthly* 82: 648–651.
- Mohta K, Watterson M, Mulgaonkar Y, et al. (2018) Fast, autonomous flight in GPS-denied and cluttered environments. *Journal of Field Robotics* 35(1): 101–120.
- Mueller CA and Birk A (2018) Conceptualization of object compositions using persistent homology. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1095–1102.
- Newell A, Yang K and Deng J (2016) Stacked hourglass networks for human pose estimation. In: *European Conference on Computer Vision*. Berlin: Springer, pp. 483–499.
- Noreen I, Khan A and Habib Z (2016) Optimal path planning using RRT* based approaches: A survey and future directions. *International Journal of Advanced Computer Science and Applications* 7(11): 97–107.
- NVIDIA (2019) Jetson TX2 Module. <https://developer.nvidia.com/embedded/jetson-tx2> (accessed 12 September 2021).
- O’Rourke J (1987) In: *Art Gallery Theorems and Algorithms (International Series of Monographs on Computer Science)*. Oxford: Oxford University Press.
- Otte M and Frazzoli E (2015) RRT^X: Asymptotically optimal single-query sampling-based motion planning with quick replanning. *The International Journal of Robotics Research* 35(7): 797–822.
- Paranjape AA, Meier KC, Shi X, Chung SJ and Hutchinson S (2015) Motion primitives and 3D path planning for fast flight through a forest. *The International Journal of Robotics Research* 34(3): 357–377.
- Paternain S, Koditschek DE and Ribeiro A (2017) Navigation functions for convex potentials in a space with convex obstacles. *IEEE Transactions on Automatic Control* 63(9): 2944–2959.
- Pavlakos G, Zhou X, Chan A, Derpanis KG and Daniilidis K (2017) 6-DoF object pose from semantic keypoints. In: *IEEE International Conference on Robotics and Automation*, pp. 2011–2018.
- Pokorny FT and Kragic D (2015) Data-driven topological motion planning with persistent cohomology. In: *Robotics: Science and Systems*.
- PolyDecomp (2019) poly_decomp: Decompose 2D polygons into convex pieces. https://github.com/wsilva32/poly_decomp.py (accessed 12 September 2021).
- Quigley M, Conley K, Gerkey BP, et al. (2009) ROS: An open-source robot operating system. In: *ICRA Workshop on Open Source Software*.
- Redmon J and Farhadi A (2018) YOLOv3: An incremental improvement. *arXiv Preprint arXiv:1804.02767*.
- Reverdy P, Ilhan BD and Koditschek DE (2015) A drift-diffusion model for robotic obstacle avoidance. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 6113–6120.

- Revzen S, Ilhan BD and Koditschek DE (2012) Dynamical trajectory replanning for unknown environments. In: *IEEE Conference on Decision and Control*, pp. 3476–3483.
- Rimon E (1990) *Exact Robot Navigation Using Artificial Potential Functions*. PhD Thesis, Yale University.
- Rimon E and Koditschek DE (1989) The construction of analytic diffeomorphisms for exact robot navigation on Star Worlds. *Transactions of the American Mathematical Society* 327(1): 71–116.
- Rimon E and Koditschek DE (1992) Exact robot navigation using artificial potential functions. *IEEE Transactions on Robotics and Automation* 8(5): 501–518.
- Russell SJ and Norvig P (2009) *Artificial Intelligence: A Modern Approach*. Englewood Cliffs, NJ: Prentice Hall.
- Rvachev VL (1963) An analytic description of certain geometric objects. *Doklady Akademii Nauk* 153: 765–767.
- Schling B (2011) *The Boost C++ Libraries*. XML Press.
- Shapely (2019) Shapely: Manipulation and analysis of geometric objects. <https://github.com/Toblerity/Shapely> (accessed 12 September 2021).
- Shapiro A (1988) Sensitivity analysis of nonlinear programs and differentiability properties of metric projections. *SIAM Journal on Control and Optimization* 26(3): 628–645.
- Shapiro V (2007) Semi-analytic geometry with R-functions. *Acta Numerica* 16: 239–303.
- Simmons R (1996) The curvature-velocity method for local obstacle avoidance. In: *IEEE International Conference on Robotics and Automation*, Vol. 4, pp. 3375–3382.
- StereoLabs (2019 a) Positional tracking with the ZED Mini stereo camera. <https://www.stereolabs.com/docs/positional-tracking/> (accessed 12 September 2021).
- StereoLabs (2019b) ZED Mini stereo camera. <https://www.stereolabs.com/zed-mini/> (accessed 12 September 2021).
- Tang S and Kumar V (2018) Autonomous flight. *Annual Review of Control, Robotics and Autonomous Systems* 1(1): 29–52.
- The CGAL Project (2019) *CGAL User and Reference Manual*. 4.14 edition. CGAL Editorial Board. <https://doc.cgal.org/4.14/Manual/packages.html> (accessed 12 September 2021).
- Topping TT, Vasilopoulos V, De A and Koditschek DE (2019) Composition of templates for transitional pedipulation behaviors. In: *The International Symposium on Robotics Research*.
- Trautman P, Ma J, Murray RM and Krause A (2015) Robot navigation in dense human crowds: Statistical models and experimental studies of human–robot cooperation. *The International Journal of Robotics Research* 34(3): 335–356.
- TurtleBot2 (2019) TurtleBot2: Open-source robot development kit for apps on wheels. <https://www.turtlebot.com/turtlebot2/>.
- van den Berg J, Guy SJ, Lin M and Manocha D (2011) *Reciprocal n-Body Collision Avoidance*. Berlin: Springer, pp. 3–19.
- van den Berg J, Lin M and Manocha D (2008) Reciprocal velocity obstacles for real-time multi-agent navigation. In: *IEEE International Conference on Robotics and Automation*, pp. 1928–1935.
- Vasilopoulos V, Arslan O, De A and Koditschek DE (2017) Sensor-based legged robot homing using range-only target localization. In: *IEEE International Conference on Robotics and Biomimetics*, pp. 2630–2637.
- Vasilopoulos V, Kantaros Y, Pappas GJ and Koditschek DE (2021) Reactive planning for mobile manipulation tasks in unexplored semantic environments. In: *IEEE International Conference on Robotics and Automation*.
- Vasilopoulos V and Koditschek DE (2018) Reactive navigation in partially known non-convex environments. In: *13th International Workshop on the Algorithmic Foundations of Robotics (WAFR)*.
- Vasilopoulos V, Pavlakos G, Bowman SL, et al. (2020) Reactive semantic planning in unexplored semantic environments using deep perceptual feedback. *IEEE Robotics and Automation Letters* 5(3): 4455–4462.
- Vasilopoulos V, Topping TT, Vega-Brown W, Roy N and Koditschek DE (2018) Sensor-based reactive execution of symbolic rearrangement plans by a legged mobile manipulator. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3298–3305.
- Vega-Brown W and Roy N (2018) Admissible abstractions for near-optimal task and motion planning. In: *International Joint Conference on Artificial Intelligence*.
- Vega-Brown W, Vasilopoulos V, Castro S, Koditschek DE and Roy N (2021) An hierarchical deliberative–reactive architecture for task and motion planning. *Mobile Robot Laboratory Publications*, Georgia Tech. Available at: <https://smartech.gatech.edu/handle/1853/22224> (accessed 12 September 2021).
- Vlantis P, Vrohidis C, Bechlioulis CP and Kyriakopoulos KJ (2018) Robot navigation in complex workspaces using harmonic maps. In: *IEEE International Conference on Robotics and Automation*, pp. 1726–1731.
- Wooden D, Malchano M, Blankespoor K, Howard A, Rizzi AA and Raibert M (2010) Autonomous navigation for BigDog. In: *IEEE International Conference on Robotics and Automation*, pp. 4736–4741.

Appendix A. Index to Multimedia Extensions

Archives of IJRR multimedia extensions published prior to 2014 can be found at <http://www.ijrr.org>, after 2014 all videos are available on the IJRR YouTube channel at <http://www.youtube.com/user/ijrrmultimedia>

Table of Multimedia Extensions

Extension	Media type	Description
1	Video	Numerical studies
2	Video	Experimental results
3	Video	Navigation toward a semantic target

Appendix B. Implicit representation of obstacles with R-functions

In this work, looking ahead toward handling in a more modular fashion the general class of obstacle shapes encompassed by the star-tree methods from the traditional navigation function literature (Rimon and Koditschek, 1989, 1992), we depart from individuated homogeneous implicit function representation of our memorized catalog elements in favor of the R-function compositions (Rvachev, 1963), explored by Rimon (1990) and explicated within the field of constructive solid geometry by Shapiro (2007). We believe that this modular representation of shape will

be helpful in the effort now in progress to instantiate the posited mapping oracle for obstacles with known geometry, whose triangular mesh can be identified in realtime using state-of-the-art techniques (Kar et al., 2015; Kong et al., 2017; Pavlakos et al., 2017) in order to extract implicit function representations for polygonal obstacles.

B.1. Preliminary definitions

We begin by providing a definition of an R-function (Shapiro, 2007).

Definition 9. A function $\gamma_\Phi: \mathbb{R}^n \rightarrow \mathbb{R}$ is an R-function if there exists a (binary) logic function $\Phi: \mathbb{B} \rightarrow \mathbb{B}$, called the companion function, that satisfies the relation

$$\Phi(S_2(w_1), \dots, S_2(w_n)) = S_2(\gamma_\Phi(w_1, \dots, w_n)) \quad (64)$$

with $(w_1, \dots, w_n) \in \mathbb{R}^n$ and S_2 the Heaviside characteristic function $S_2: \mathbb{R} \rightarrow \mathbb{B}$ of the interval $[0, +\infty)$ defined as¹⁵

$$S_2(x) = \begin{cases} 0, & x \leq -0 \\ 1, & x \geq +0 \end{cases} \quad (65)$$

Informally, a real function γ_Φ is an R-function if it can change its property (sign) only when some of its arguments change the same property (sign) (Shapiro, 2007). For example, the companion logic function for the R-function $\gamma(x, y) = xy$ is $X \Leftrightarrow Y$; we just check that $S_2(xy) = (S_2(x) \Leftrightarrow S_2(y))$.

In this work, we use the following (symbolically written) R-functions (Shapiro, 2007)

$$\neg x := -x \quad (66)$$

$$x_1 \wedge x_2 := x_1 + x_2 - (x_1^p + x_2^p)^{\frac{1}{p}} \quad (67)$$

$$x_1 \vee x_2 := x_1 + x_2 + (x_1^p + x_2^p)^{\frac{1}{p}} \quad (68)$$

with companion logic functions the logical negation $\neg$, conjunction $\wedge$ and disjunction $\vee$, respectively, and p a positive integer. Intuitively, the author in (Shapiro, 2007) uses the triangle inequality with the L_p -norm to derive R-functions with specific properties.

B.2. Description of the algorithm

R-functions have several interesting properties but, most importantly, provide machinery to construct implicit representations for sets built from other, primitive sets. Namely, in order to obtain a real function inequality $\gamma \geq 0$ defining a set Ω constructed from primitive sets Ω_j , it suffices to construct an appropriate R-function and substitute for its arguments the real functions ω_j defining the primitive sets Ω_j implicitly as $\omega_j \geq 0$ (Shapiro, 2007: Theorem 3). In our case, the set Ω would be a polygon P_i we want to represent, the sets Ω_j would be half-spaces induced by the polygon edges, and the functions $\omega_j: \mathbb{R}^2 \rightarrow \mathbb{R}$ their corresponding hyperplane equations, given by

$$\omega_j(\mathbf{x}) = (\mathbf{x} - \mathbf{x}_j)^\top \mathbf{n}_j \quad (69)$$

Here $\mathbf{x}_j$ is any arbitrary point on the edge hyperplane and $\mathbf{n}_j$ its normal vector, pointing towards the polygon's interior.

This result allows us to use a variant of the method presented in Shapiro (2007) and construct representations of polygons in the form of AND-OR trees (Russell and Norvig, 2009), as shown in the example of Figure 23. Briefly, the interior of a polygon can be represented as the intersection of two or more *polygonal chains*, i.e., sequences of edges that meet at the polygon's convex hull. In the same way, each of these chains can then be split recursively into smaller subchains at the vertices of its convex hull to form a tree structure. The root node of the tree is the original polygon, with each other node corresponding to a polygonal chain; the leaves of the tree are single hyperplanes, the edges of the polygon described by functions ω_j . If the split occurs at a concave vertex of the *original* polygon, then the subchains are combined using set union (i.e., disjunction); otherwise, they are combined using set intersection (i.e., conjunction), as shown in Figure 23. In this way, by having as input just the vertices of the polygon in counterclockwise order, we are able to construct an implicit representation for each node of the tree bottom-up, using the R-functions (69) and (70), until we reach the root node of the tree. If we want $\beta_i > 0$ in the exterior of P_i , we can negate the result (i.e., we use the R-function (68)) to obtain the function β_i , which is analytic everywhere except for the polygon vertices Shapiro (2007). This is the reason our results in Section 4 still hold, with the map $\mathbf{h}^T$ being a C^∞ diffeomorphism away from the polygon vertices.

B.3. R-functions as approximations of the distance function

It is important to mention that, away from the corners and in a neighborhood of the polygon, *normalized* R-functions constructed using (68)–(70) behave as smooth p th-order approximations of the (non-differentiable) distance function to the polygon, as shown in Figure 23(b) and (c). The reader is referred to Shapiro (2007) for more details; in our setting, a sufficient condition for normalization is to make sure that for each ω_j given in (71), the corresponding normal vector $\mathbf{n}_j$ has unit norm Shapiro (2007). This property is quite useful for our purposes, as it endows the implicit representation of our polygons with a physical meaning, compared with other representations (e.g., the homogeneous function representations in Rimón and Koditschek (1989)). Numerical experimentation showed that even $p = 2$ gives sufficiently good results in our setting.

Appendix C. Construction of polygonal collars

Definitions 3, 5, and 7 provide the basic guidelines for constructing admissible polygonal collars that fit our formal results. However, there is not a unique way of performing this operation. Here, we describe the method employed in

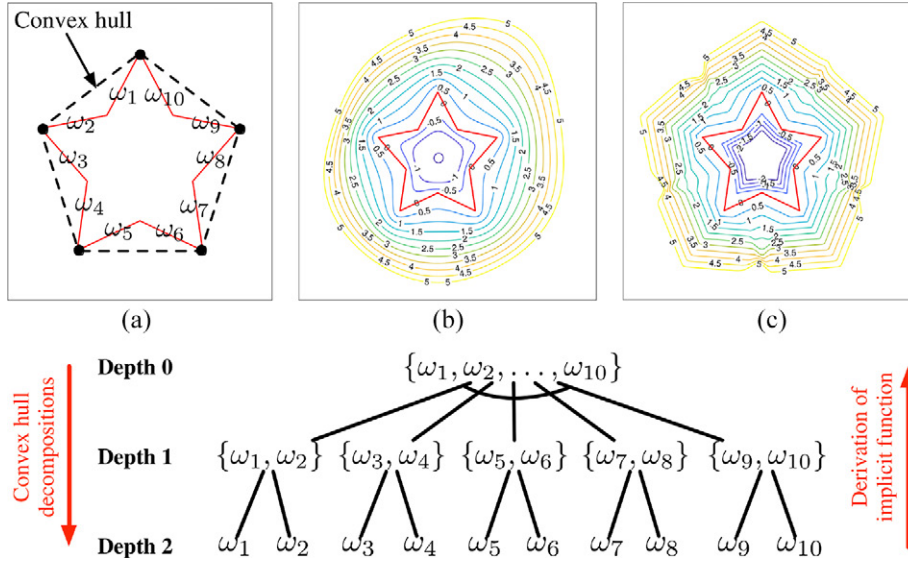


Fig. 23. (a) An example of a star-shaped polygonal obstacle and the corresponding ω_j functions. (b) Level curves of the corresponding implicit function β for $p=2$. (c) Level curves of the corresponding implicit function β for $p=20$. Bottom: The AND-OR tree, constructed by the algorithm described in Appendix B.2 to represent this polygon. The polygon is split at the vertices of the convex hull to generate five subchains at depth 1. Each of these subchains is then split into two subchains at depth 2. The subchains at depth 2 (1) are combined via disjunction (conjunction), because they meet at non-convex (convex) vertices of the original polygon. In this way, we get our implicit function $\beta = \neg((\omega_1 \vee \omega_2) \wedge (\omega_3 \vee \omega_4) \wedge (\omega_5 \vee \omega_6) \wedge (\omega_7 \vee \omega_8) \wedge (\omega_9 \vee \omega_{10}))$.

this article for a single polygon P , contained in either $\mathcal{D}_{\text{map}}^{\mathcal{I}}$ or $\mathcal{B}_{\text{map}}^{\mathcal{I}}$, whose triangulation tree $\mathcal{T}_P := (\mathcal{V}_P, \mathcal{E}_P)$ has already been constructed, according to Section 4.1, and the corresponding centers of transformation $\mathbf{x}_j^*$ have already been identified, according to Definitions 2, 4, and 6 for all triangles $j \in \mathcal{V}_P$. We assume that the value of the corresponding clearance ϵ_P , according to Assumption 2, is also known.

Based on the above, the first step is to stack all triangles in $\mathcal{V}_P$ in order of descending depth, as prescribed by the sequence of purging transformations. Then, for each triangle $j \in \mathcal{V}_P$ in the stack, we first dilate the polygon $\mathcal{Q}_j$ by ϵ_P and take the intersection with $\mathcal{F}_{\text{map},j}^{\mathcal{I}}$. If the triangle j is the root triangle of P and $P \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$, this is enough to give an admissible polygonal collar. In any other case, we have to take the intersection of the generated dilated polygon with the half spaces H_{1j} and H_{2j} , defined as the sets

$$H_{1j} := \left\{ \mathbf{z} \mid (\mathbf{z} - \mathbf{x}_j^*)^\top \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \frac{\mathbf{x}_j^* - \mathbf{x}_{1j}}{\|\mathbf{x}_j^* - \mathbf{x}_{1j}\|} \geq 0 \right\}$$

$$H_{2j} := \left\{ \mathbf{z} \mid (\mathbf{z} - \mathbf{x}_j^*)^\top \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \frac{\mathbf{x}_{2j} - \mathbf{x}_j^*}{\|\mathbf{x}_{2j} - \mathbf{x}_j^*\|} \geq 0 \right\}$$

i.e., the half spaces defined by hyperplanes passing through the center $\mathbf{x}_j^*$ and vertex $\mathbf{x}_{1j}$, and the center $\mathbf{x}_j^*$ and vertex $\mathbf{x}_{2j}$, respectively. The resulting polygon is guaranteed to be convex, but might intersect with triangles in $\mathcal{V}_P$ that will succeed j in the purging transformation. To solve that problem, we can take the difference of this polygon with all triangles that will succeed j in the purging transformation (except for its parent $p(j)$), decompose the final resulting

Algorithm 3 Construction of the polygonal collars $\bar{\mathcal{Q}}_j$ for all triangles $j \in \mathcal{V}_P$ of a polygon P , whose triangulation tree $\mathcal{T}_P := (\mathcal{V}_P, \mathcal{E}_P)$ and associated clearance ϵ_P are known.

```

function CollarConstruction  $P, \epsilon_P$ 
 $\mathcal{V}_P \leftarrow \text{sort}(\mathcal{V}_P)$  ▷ Sort in descending depth
do
 $j \leftarrow \text{pop}(\mathcal{V}_P)$  ▷ Pop next triangle
if  $j$  is root and  $P \in \mathcal{D}_{\text{map}}^{\mathcal{I}}$  then
 $\mathcal{Q}_j \leftarrow \mathbf{x}_{1j}\mathbf{x}_{2j}\mathbf{x}_{3j}\mathbf{x}_{1j}$ 
 $\mathcal{A}_j \leftarrow \text{dilate}(\mathcal{Q}_j, \epsilon_P)$  ▷ Dilate  $\mathcal{Q}_j$  by  $\epsilon_P$ 
 $\bar{\mathcal{Q}}_j \leftarrow \mathcal{A}_j \cap \mathcal{F}_{\text{map},j}^{\mathcal{I}}$ 
else if  $j$  is root and  $P \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$  then
 $\mathcal{Q}_j \leftarrow \mathbf{x}_j^*\mathbf{x}_{2j}\mathbf{x}_{3j}\mathbf{x}_{1j}\mathbf{x}_j^*$ 
 $\mathcal{A}_j \leftarrow \text{dilate}(\mathcal{Q}_j, \epsilon_P)$  ▷ Dilate  $\mathcal{Q}_j$  by  $\epsilon_P$ 
 $\mathcal{R}_j \leftarrow (\mathcal{A}_j \cap \mathcal{F}_{\text{map},j}^{\mathcal{I}}) \cap (H_{1j} \cap H_{2j})$ 
 $\bar{\mathcal{Q}}_j \leftarrow \mathcal{R}_j \cup \mathbf{x}_j^*\mathbf{x}_{2j}\mathbf{x}_{1j}\mathbf{x}_j^*$ 
else
 $\mathcal{Q}_j \leftarrow \mathbf{x}_j^*\mathbf{x}_{2j}\mathbf{x}_{3j}\mathbf{x}_{1j}\mathbf{x}_j^*$ 
 $\mathcal{A}_j \leftarrow \text{dilate}(\mathcal{Q}_j, \epsilon_P)$  ▷ Dilate  $\mathcal{Q}_j$  by  $\epsilon_P$ 
 $\mathcal{R}_j \leftarrow (\mathcal{A}_j \cap \mathcal{F}_{\text{map},j}^{\mathcal{I}}) \cap (H_{1j} \cap H_{2j})$ 
 $\mathcal{L}_j \leftarrow \text{List of triangles that will succeed } j$ 
do
 $i \leftarrow \text{pop}(\mathcal{L}_j)$  ▷ Pop next triangle
if  $i$  is  $p(j)$  then
continue
else
 $\mathcal{R}_j \leftarrow \mathcal{R}_j - i$  ▷ Polygon difference
end if
while  $\mathcal{L}_j \neq \emptyset$ 
 $\{\mathcal{Z}\}_k \leftarrow \text{poly\_decomp}(\mathcal{R}_j)$  ▷ (Keil, 1985)
 $\bar{\mathcal{Q}}_j \leftarrow \mathcal{Z}_k$  such that  $\mathcal{Q}_j \subset \mathcal{Z}_k$ 
end if
while  $\mathcal{V}_P \neq \emptyset$ 
end function

```

polygon into its convex pieces using a variant of Keil's algorithm (Keil, 1985) (implemented in the C++ library CGAL (The CGAL Project, 2019) and in the Python package `poly_decomp` (PolyDecomp, 2019)), and use as $\bar{Q}_j$ the convex piece that includes Q_j , as prescribed by Definitions 3, 5, and 7. The whole procedure is shown in Algorithm 3.

Appendix D. Inductive computation of the diffeomorphism at execution time

From the description of the diffeomorphism $\mathbf{h}^T$ in Section 4, we see that $\mathbf{h}^T$ is constructed in multiple steps by composition. Therefore, we can compute the value of $\mathbf{h}^T(\mathbf{x})$ at $\mathbf{x} \in \mathcal{F}_{\text{map}}^T$ inductively, by setting $\mathbf{h}_0^T(\mathbf{x}) = \mathbf{x}$ and computing $\mathbf{h}_k^T(\mathbf{x}) = \mathbf{h}_{k,k-1}^T \circ \mathbf{h}_{k-1}^T(\mathbf{x})$, with k spanning all triangles in $\mathcal{V}_P$ for all known obstacles P in both $\mathcal{D}_{\text{map}}^T$ and $\mathcal{B}_{\text{map}}^T$, and $\mathbf{h}_{k,k-1}^T$ given either in (16) or (26). We can then see that, due to Lemma 4, $\mathbf{h}_{k,k-1}^T$ can be generally written in the following form

$$\mathbf{h}_{k,k-1}^T(\mathbf{x}) = \sigma_{k,k-1}(\mathbf{x}) \left[\mathbf{x}_{k,k-1}^* + \nu_{k,k-1}(\mathbf{x})(\mathbf{x} - \mathbf{x}_{k,k-1}^*) \right] + (1 - \sigma_{k,k-1}(\mathbf{x}))\mathbf{x} \quad (70)$$

with the switch $\sigma_{k,k-1}$ (see (13) and (21)), deforming factor $\nu_{k,k-1}$ (see (14), (22), and (23)) and center of the transformation $\mathbf{x}_{k,k-1}^*$ (see Definitions 2, 4, and 7) depending on the particular triangle being purged.

We can, therefore, set $D_{\mathbf{x}}\mathbf{h}_0^T := \mathbf{I}$, compute

$$\begin{aligned} D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T &= (\nu_{k,k-1}(\mathbf{x}) - 1)(\mathbf{x} - \mathbf{x}_{k,k-1}^*)\nabla\sigma_{k,k-1}(\mathbf{x})^\top \\ &\quad + \sigma_{k,k-1}(\mathbf{x})(\mathbf{x} - \mathbf{x}_{k,k-1}^*)\nabla\nu_{k,k-1}(\mathbf{x})^\top \\ &\quad + [1 + \sigma_{k,k-1}(\mathbf{x})(\nu_{k,k-1}(\mathbf{x}) - 1)]\mathbf{I} \end{aligned} \quad (71)$$

and use the chain rule to write

$$D_{\mathbf{x}}\mathbf{h}_k^T = \left(D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T \circ \mathbf{h}_{k-1}^T(\mathbf{x}) \right) \cdot D_{\mathbf{x}}\mathbf{h}_{k-1}^T \quad (72)$$

Finally, because (47) requires partial derivatives of $D_{\mathbf{x}}\mathbf{h}^T$, we can follow a similar procedure and the chain rule to compute the partial derivatives $\frac{\partial[D_{\mathbf{x}}\mathbf{h}_k^T]_{ml}}{\partial[\mathbf{x}]_n}$, as functions of $\frac{\partial[D_{\mathbf{x}}\mathbf{h}_{k-1}^T]_{ml}}{\partial[\mathbf{x}]_n}$ and $\frac{\partial[D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T]_{ml}}{\partial[\mathbf{x}]_n} \big|_{\mathbf{h}_{k-1}^T(\mathbf{x})}$, after initially setting all partial derivatives to zero: $\frac{\partial[D_{\mathbf{x}}\mathbf{h}_0^T]_{ml}}{\partial[\mathbf{x}]_n} = 0$, with the indices $m, l, n \in \{1, 2\}$. Namely:

$$\begin{aligned} \frac{\partial[D_{\mathbf{x}}\mathbf{h}_k^T]_{ml}}{\partial[\mathbf{x}]_n} &= \sum_{r=1}^2 \left(\left[D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T \circ \mathbf{h}_{k-1}^T(\mathbf{x}) \right]_{mr} \cdot \frac{\partial[D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T]_{rl}}{\partial[\mathbf{x}]_n} \right. \\ &\quad \left. + [D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T]_{rl} \sum_{s=1}^2 [D_{\mathbf{x}}\mathbf{h}_{k-1}^T]_{sn} \cdot \frac{\partial[D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T]_{mr}}{\partial[\mathbf{x}]_s} \big|_{\mathbf{h}_{k-1}^T(\mathbf{x})} \right) \end{aligned} \quad (73)$$

where, from (71), we can compute

$$\begin{aligned} \frac{\partial[D_{\mathbf{x}}\mathbf{h}_{k,k-1}^T]_{mr}}{\partial[\mathbf{x}]_s} &= (\nu_{k,k-1} - 1) \frac{\partial\sigma_{k,k-1}}{\partial[\mathbf{x}]_r} \delta_{ms} \\ &\quad + ([\mathbf{x}]_m - [\mathbf{x}_{k,k-1}^*]_m) \frac{\partial\sigma_{k,k-1}}{\partial[\mathbf{x}]_r} \frac{\partial\nu_{k,k-1}}{\partial[\mathbf{x}]_s} \\ &\quad + (\nu_{k,k-1} - 1)([\mathbf{x}]_m - [\mathbf{x}_{k,k-1}^*]_m) \frac{\partial^2\sigma_{k,k-1}}{\partial[\mathbf{x}]_r\partial[\mathbf{x}]_s} \\ &\quad + ([\mathbf{x}]_m - [\mathbf{x}_{k,k-1}^*]_m) \frac{\partial\sigma_{k,k-1}}{\partial[\mathbf{x}]_s} \frac{\partial\nu_{k,k-1}}{\partial[\mathbf{x}]_r} \\ &\quad + \sigma_{k,k-1} \frac{\partial\nu_{k,k-1}}{\partial[\mathbf{x}]_r} \delta_{ms} \\ &\quad + \sigma_{k,k-1}([\mathbf{x}]_m - [\mathbf{x}_{k,k-1}^*]_m) \frac{\partial^2\nu_{k,k-1}}{\partial[\mathbf{x}]_r\partial[\mathbf{x}]_s} \\ &\quad + \sigma_{k,k-1} \frac{\partial\nu_{k,k-1}}{\partial[\mathbf{x}]_s} \delta_{mr} + (\nu_{k,k-1} - 1) \frac{\partial\sigma_{k,k-1}}{\partial[\mathbf{x}]_s} \delta_{mr} \end{aligned} \quad (74)$$

by using elements of the Hessians $\nabla^2\sigma_{k,k-1}$, $\nabla^2\nu_{k,k-1}$.

Appendix E. Proofs

E.1. Proofs of results in Section 4

Proof of Lemma 1. With the procedure outlined in Appendix B, the only points where γ_{j_i} and δ_{j_i} are not smooth are vertices of Q_{j_i} and $\bar{Q}_{j_i}$, respectively. Therefore, with the definition of $\sigma_{\delta_{j_i}}$ as in (12) and the use of the smooth, non-analytic function ζ from (9), we see that $\sigma_{\delta_{j_i}}$ is smooth everywhere, because $\mathbf{x}_{j_i}^*$ does not belong in $\mathcal{F}_{\text{map},j_i}^T$ and δ_{j_i} is exactly 0 on the vertices of $\bar{Q}_{j_i}$. Therefore, σ_{j_i} can only be non-smooth on the vertices of Q_{j_i} except for $\mathbf{x}_{j_i}^*$ (i.e., on the vertices of the triangle j_i), and on points where its denominator becomes zero. As both $\sigma_{\gamma_{j_i}}$ and $\sigma_{\delta_{j_i}}$ vary between 0 and 1, this can only happen when $\sigma_{\gamma_{j_i}}(\mathbf{x}) = 1$ and $\sigma_{\delta_{j_i}}(\mathbf{x}) = 0$, i.e., only on $\mathbf{x}_{1j_i}$ and $\mathbf{x}_{2j_i}$. The fact that σ_{j_i} is smooth everywhere else derives immediately from the fact that $\sigma_{\delta_{j_i}}$ is a smooth function, and $\sigma_{\gamma_{j_i}}$ is smooth everywhere except for the triangle vertices. $\square$

Proof of Lemma 2. From Lemma 1, we already know that the switch σ_{j_i} is smooth away from the vertices of j_i . On the other hand, the singular points of the deforming factor ν_{j_i} are the solutions of the equation $(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i} = 0$ and, therefore, lie on the hyperplane passing through $\mathbf{x}_{j_i}^*$ with normal vector $\mathbf{n}_{j_i}$ and, owing to the construction of $\bar{Q}_{j_i}$ as in Definition 3, lie outside of $\bar{Q}_{j_i}$ and do not affect the map $\mathcal{F}_{\text{map},j_i}^T$. Hence, the map $\mathbf{h}_{j_i}^T$ is smooth everywhere in $\mathcal{F}_{\text{map},j_i}^T$, except for the vertices of the triangle j_i , as a composition of smooth functions with the same properties.

Proof of Proposition 1. First of all, the map $\mathbf{h}_{j_i}^T$ is smooth everywhere except for the vertices of the triangle j_i , as shown in Lemma 2. Therefore, in order to prove that $\mathbf{h}_{j_i}^T$ is a C^∞ diffeomorphism away from the triangle vertices

$\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$, we follow the procedure outlined in Massey (1992), also followed in Rimón and Koditschek (1989), to show that:

1. $\mathbf{h}_{j_i}^T$ has a non-singular differential on $\mathcal{F}_{\text{map},j_i}^T$ except for $\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$;
2. $\mathbf{h}_{j_i}^T$ preserves boundaries, i.e., $\mathbf{h}_{j_i}^T(\partial_k \mathcal{F}_{\text{map},j_i}^T) \subset \partial_k \mathcal{F}_{\text{map},p(j_i)}^T$, with k spanning both the indices of familiar obstacles $\mathcal{J}_D^T, \mathcal{J}_B^T$ as well as the indices of unknown obstacles $\mathcal{J}_C$, and $\partial_k \mathcal{F}$ the k th connected component of the boundary of $\mathcal{F}$ with $\partial_0 \mathcal{F}$ the outer boundary of $\mathcal{F}$;
3. the boundary components of $\mathcal{F}_{\text{map},j_i}^T$ and $\mathcal{F}_{\text{map},p(j_i)}^T$ are pairwise homeomorphic, i.e., $\partial_k \mathcal{F}_{\text{map},j_i}^T \cong \partial_k \mathcal{F}_{\text{map},p(j_i)}^T$, with k spanning both the indices of familiar obstacles $\mathcal{J}_D^T, \mathcal{J}_B^T$ as well as the indices of unknown obstacles $\mathcal{J}_C$.

We begin with Property 1 and examine the space away from the triangle vertices $\mathbf{x}_{1j_i}, \mathbf{x}_{2j_i}, \mathbf{x}_{3j_i}$. The case where $\sigma_{\delta_{j_i}}$ is 0 (outside of the polygonal collar $\mathcal{Q}_{j_i}$) is not interesting, because $\mathbf{h}_{j_i}^T$ defaults to the identity map and $D_{\mathbf{x}} \mathbf{h}_{j_i}^T = \mathbf{I}$. When $\sigma_{\delta_{j_i}}$ is not 0, we can compute the jacobian of the map as

$$\begin{aligned} D_{\mathbf{x}} \mathbf{h}_{j_i}^T &= (\nu_{j_i}(\mathbf{x}) - 1)(\mathbf{x} - \mathbf{x}_{j_i}^*) \nabla \sigma_{j_i}(\mathbf{x})^\top \\ &\quad + \sigma_{j_i}(\mathbf{x})(\mathbf{x} - \mathbf{x}_{j_i}^*) \nabla \nu_{j_i}(\mathbf{x})^\top \\ &\quad + [1 + \sigma_{j_i}(\mathbf{x})(\nu_{j_i}(\mathbf{x}) - 1)] \mathbf{I} \end{aligned} \quad (75)$$

For the deforming factor ν_{j_i} we compute from (14)

$$\nabla \nu_{j_i}(\mathbf{x}) = - \frac{(\mathbf{x}_{1j_i} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i}}{\left[(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i} \right]^2} \mathbf{n}_{j_i} \quad (76)$$

Note that we interestingly obtain

$$(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \nu_{j_i}(\mathbf{x}) = -\nu_{j_i}(\mathbf{x}) \quad (77)$$

From (75) it can be seen that $D_{\mathbf{x}} \mathbf{h}_{j_i}^T = \mathbf{A} + \mathbf{u}\mathbf{v}^\top$ with $\mathbf{A} = [1 + \sigma_{j_i}(\mathbf{x})(\nu_{j_i}(\mathbf{x}) - 1)] \mathbf{I}$, $\mathbf{u} = \mathbf{x} - \mathbf{x}_{j_i}^*$ and $\mathbf{v} = (\nu_{j_i}(\mathbf{x}) - 1) \nabla \sigma_{j_i}(\mathbf{x}) + \sigma_{j_i}(\mathbf{x}) \nabla \nu_{j_i}(\mathbf{x})$.

Owing to the fact that $0 \leq \sigma_{j_i}(\mathbf{x}) \leq 1$ and $0 < \nu_{j_i}(\mathbf{x}) < 1$ in the interior of an admissible polygonal collar $\mathcal{Q}_{j_i}$ (see Definition 3), we obtain $1 + \sigma_{j_i}(\mathbf{x})(\nu_{j_i}(\mathbf{x}) - 1) > 0$. Hence, $\mathbf{A}$ is invertible, and by using the matrix determinant lemma and (77), the determinant of $D_{\mathbf{x}} \mathbf{h}_{j_i}^T$ can be computed as

$$\begin{aligned} \det(D_{\mathbf{x}} \mathbf{h}_{j_i}^T) &= \det \mathbf{A} + (\det \mathbf{A}) \mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u} \\ &= [1 + \sigma_{j_i}(\mathbf{x})(\nu_{j_i}(\mathbf{x}) - 1)] \cdot \\ &\quad \cdot \left[(1 - \sigma_{j_i}(\mathbf{x})) + (\nu_{j_i}(\mathbf{x}) - 1)(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \sigma_{j_i}(\mathbf{x}) \right] \end{aligned} \quad (78)$$

Similarly, the trace of $D_{\mathbf{x}} \mathbf{h}_{j_i}^T$ can be computed as

$$\begin{aligned} \text{tr}(D_{\mathbf{x}} \mathbf{h}_{j_i}^T) &= [1 + \sigma_{j_i}(\mathbf{x})(\nu_{j_i}(\mathbf{x}) - 1)] + (1 - \sigma_{j_i}(\mathbf{x})) \\ &\quad + (\nu_{j_i}(\mathbf{x}) - 1)(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \sigma_{j_i}(\mathbf{x}) \end{aligned} \quad (79)$$

In addition, by construction of the switch σ_{j_i} , we see that $\nabla \sigma_{j_i}(\mathbf{x}) = \mathbf{0}$ when $\sigma_{j_i}(\mathbf{x}) = 0$. Hence, using the above expressions, we can show that $\det(D_{\mathbf{x}} \mathbf{h}_{j_i}^T), \text{tr}(D_{\mathbf{x}} \mathbf{h}_{j_i}^T) > 0$ (and, therefore, establish that $D_{\mathbf{x}} \mathbf{h}_{j_i}^T$ is not singular in the interior of $\mathcal{Q}_{j_i}$, because $\mathcal{F}_{\text{map},j_i}^T \subseteq \mathbb{R}^2$) by showing that $(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \sigma_{j_i}(\mathbf{x}) < 0$ when $\sigma_{j_i}(\mathbf{x}) > 0$, where

$$\begin{aligned} \nabla \sigma_{j_i}(\mathbf{x}) &= \frac{\sigma_{\delta_{j_i}}(\mathbf{x})}{\left[\sigma_{\gamma_{j_i}}(\mathbf{x}) \sigma_{\delta_{j_i}}(\mathbf{x}) + (1 - \sigma_{\gamma_{j_i}}(\mathbf{x})) \right]^2} \nabla \sigma_{\gamma_{j_i}}(\mathbf{x}) \\ &\quad + \frac{\sigma_{\gamma_{j_i}}(\mathbf{x}) (1 - \sigma_{\gamma_{j_i}}(\mathbf{x}))}{\left[\sigma_{\gamma_{j_i}}(\mathbf{x}) \sigma_{\delta_{j_i}}(\mathbf{x}) + (1 - \sigma_{\gamma_{j_i}}(\mathbf{x})) \right]^2} \nabla \sigma_{\delta_{j_i}}(\mathbf{x}) \end{aligned} \quad (80)$$

with

$$\nabla \sigma_{\gamma_{j_i}}(\mathbf{x}) = \begin{cases} -\frac{\mu_{\gamma_{j_i}} \sigma_{\gamma_{j_i}}(\mathbf{x})}{(\epsilon_{j_i} - \gamma_{j_i}(\mathbf{x}))^2} \nabla \gamma_{j_i}(\mathbf{x}), & \gamma_{j_i}(\mathbf{x}) < \epsilon_{j_i} \\ \mathbf{0}, & \gamma_{j_i}(\mathbf{x}) \geq \epsilon_{j_i} \end{cases} \quad (81)$$

$$\nabla \sigma_{\delta_{j_i}}(\mathbf{x}) = \begin{cases} \frac{\mu_{\delta_{j_i}} \sigma_{\delta_{j_i}}(\mathbf{x})}{\alpha_{j_i}(\mathbf{x})^2} \nabla \alpha_{j_i}(\mathbf{x}), & \delta_{j_i}(\mathbf{x}) > 0 \\ \mathbf{0}, & \delta_{j_i}(\mathbf{x}) \leq 0 \end{cases} \quad (82)$$

and $\alpha_{j_i}(\mathbf{x}) := \delta_{j_i}(\mathbf{x}) / \|\mathbf{x} - \mathbf{x}_{j_i}^*\|$. Therefore, it suffices to show that when $\sigma_{j_i}(\mathbf{x}) > 0$:

$$(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \gamma_{j_i}(\mathbf{x}) > 0 \quad (83)$$

$$(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \alpha_{j_i}(\mathbf{x}) < 0 \quad (84)$$

Following the procedure outlined in Appendix B for generating implicit functions for polygons, it can be seen that the implicit function γ_{j_i} , describing the exterior of the quadrilateral $\mathcal{Q}_{j_i}$, can be written as

$$\gamma_{j_i}(\mathbf{x}) = \neg((\gamma_{1j_i}(\mathbf{x}) \wedge \gamma_{2j_i}(\mathbf{x})) \wedge (\gamma_{3j_i}(\mathbf{x}) \wedge \gamma_{4j_i}(\mathbf{x}))) \quad (85)$$

with the negation ($\neg$) and conjunction ($\wedge$) operators defined as in (66) and (67), respectively, and $\gamma_{1j_i}, \gamma_{2j_i}, \gamma_{3j_i}, \gamma_{4j_i}$ the hyperplane equations describing $\mathcal{Q}_{j_i}$ defined as follows

$$\gamma_{1j_i}(\mathbf{x}) := (\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{1j_i}, \mathbf{n}_{1j_i} := \mathbf{R}_{\frac{\mathbf{x}_{j_i}^* - \mathbf{x}_{1j_i}}{\|\mathbf{x}_{j_i}^* - \mathbf{x}_{1j_i}\|}} \quad (86)$$

$$\gamma_{2j_i}(\mathbf{x}) := (\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{2j_i}, \mathbf{n}_{2j_i} := \mathbf{R}_{\frac{\mathbf{x}_{2j_i} - \mathbf{x}_{j_i}^*}{\|\mathbf{x}_{2j_i} - \mathbf{x}_{j_i}^*\|}} \quad (87)$$

$$\gamma_{3j_i}(\mathbf{x}) := (\mathbf{x} - \mathbf{x}_{3j_i})^\top \mathbf{n}_{3j_i}, \mathbf{n}_{3j_i} := \mathbf{R}_{\frac{\mathbf{x}_{3j_i} - \mathbf{x}_{2j_i}}{\|\mathbf{x}_{3j_i} - \mathbf{x}_{2j_i}\|}} \quad (88)$$

$$\gamma_{4j_i}(\mathbf{x}) := (\mathbf{x} - \mathbf{x}_{3j_i})^\top \mathbf{n}_{4j_i}, \mathbf{n}_{4j_i} := \mathbf{R}_{\frac{\mathbf{x}_{1j_i} - \mathbf{x}_{3j_i}}{\|\mathbf{x}_{1j_i} - \mathbf{x}_{3j_i}\|}} \quad (89)$$

We, therefore, obtain

$$\begin{aligned}\nabla \gamma_{j_i} = & - \left(1 - \frac{\gamma_{1j_i} \wedge \gamma_{2j_i}}{\sqrt{(\gamma_{1j_i} \wedge \gamma_{2j_i})^2 + (\gamma_{3j_i} \wedge \gamma_{4j_i})^2}} \right) \nabla (\gamma_{1j_i} \wedge \gamma_{2j_i}) \\ & - \left(1 - \frac{\gamma_{3j_i} \wedge \gamma_{4j_i}}{\sqrt{(\gamma_{1j_i} \wedge \gamma_{2j_i})^2 + (\gamma_{3j_i} \wedge \gamma_{4j_i})^2}} \right) \nabla (\gamma_{3j_i} \wedge \gamma_{4j_i})\end{aligned}\quad (90)$$

with

$$\begin{aligned}\nabla (\gamma_{1j_i} \wedge \gamma_{2j_i}) &= \left(1 - \frac{\gamma_{1j_i}}{\sqrt{\gamma_{1j_i}^2 + \gamma_{2j_i}^2}} \right) \nabla \gamma_{1j_i} \\ &\quad + \left(1 - \frac{\gamma_{2j_i}}{\sqrt{\gamma_{1j_i}^2 + \gamma_{2j_i}^2}} \right) \nabla \gamma_{2j_i} \\ &= \left(1 - \frac{\gamma_{1j_i}}{\sqrt{\gamma_{1j_i}^2 + \gamma_{2j_i}^2}} \right) \mathbf{n}_{1j_i} \\ &\quad + \left(1 - \frac{\gamma_{2j_i}}{\sqrt{\gamma_{1j_i}^2 + \gamma_{2j_i}^2}} \right) \mathbf{n}_{2j_i}\end{aligned}\quad (91)$$

$$\begin{aligned}\nabla (\gamma_{3j_i} \wedge \gamma_{4j_i}) &= \left(1 - \frac{\gamma_{3j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) \nabla \gamma_{3j_i} \\ &\quad + \left(1 - \frac{\gamma_{4j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) \nabla \gamma_{4j_i} \\ &= \left(1 - \frac{\gamma_{3j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) \mathbf{n}_{3j_i} \\ &\quad + \left(1 - \frac{\gamma_{4j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) \mathbf{n}_{4j_i}\end{aligned}\quad (92)$$

It is then not hard to show that $(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla (\gamma_{1j_i} \wedge \gamma_{2j_i}) = \gamma_{1j_i} \wedge \gamma_{2j_i}$. The term corresponding to $(\gamma_{3j_i} \wedge \gamma_{4j_i})$ is more complicated, but we can follow a similar procedure to obtain

$$\begin{aligned}(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla (\gamma_{3j_i} \wedge \gamma_{4j_i}) &= \gamma_{3j_i} \wedge \gamma_{4j_i} \\ &\quad - \left(1 - \frac{\gamma_{3j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) (\mathbf{x}_{j_i}^* - \mathbf{x}_{3j_i})^\top \mathbf{n}_{3j_i} \\ &\quad - \left(1 - \frac{\gamma_{4j_i}}{\sqrt{\gamma_{3j_i}^2 + \gamma_{4j_i}^2}} \right) (\mathbf{x}_{j_i}^* - \mathbf{x}_{4j_i})^\top \mathbf{n}_{4j_i} \\ &< \gamma_{3j_i} \wedge \gamma_{4j_i}\end{aligned}\quad (93)$$

because $(\mathbf{x}_{j_i}^* - \mathbf{x}_{3j_i})^\top \mathbf{n}_{3j_i} > 0$ and $(\mathbf{x}_{j_i}^* - \mathbf{x}_{4j_i})^\top \mathbf{n}_{4j_i} > 0$, because $\mathcal{Q}_{j_i}$ is convex. Therefore, using the facts that $(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla (\gamma_{1j_i} \wedge \gamma_{2j_i}) = \gamma_{1j_i} \wedge \gamma_{2j_i}$ and $(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla (\gamma_{3j_i} \wedge \gamma_{4j_i}) < \gamma_{3j_i} \wedge \gamma_{4j_i}$, we can obtain the desired result using (90) as follows

$$\begin{aligned}(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \gamma_{j_i}(\mathbf{x}) &> -((\gamma_{1j_i} \wedge \gamma_{2j_i}) \wedge (\gamma_{3j_i} \wedge \gamma_{4j_i})) \\ &= -((\gamma_{1j_i} \wedge \gamma_{2j_i}) \wedge (\gamma_{3j_i} \wedge \gamma_{4j_i})) \\ &= \gamma_{j_i}(\mathbf{x}) > 0\end{aligned}\quad (94)$$

The proof of (84) follows similar patterns. Here, we focus on δ_{j_i} . The external polygonal collar $\bar{\mathcal{Q}}_{j_i}$ can be assumed to have n sides, which means that we can write $\delta_{j_i} = ((\delta_{1j_i} \wedge \delta_{2j_i}) \wedge \dots \wedge \delta_{nj_i})$. Following the procedure outlined above for the proof of (83), we can expand each term in the conjunction individually and then combine them to obtain

$$(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \delta_{j_i}(\mathbf{x}) < \delta_{j_i}(\mathbf{x}) \quad (95)$$

We also have

$$\begin{aligned}\nabla \alpha_{j_i}(\mathbf{x}) &= \nabla \left(\frac{\delta_{j_i}(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_{j_i}^*\|} \right) \\ &= \frac{\|\mathbf{x} - \mathbf{x}_{j_i}^*\| \nabla \delta_{j_i}(\mathbf{x}) - \delta_{j_i}(\mathbf{x}) \frac{\mathbf{x} - \mathbf{x}_{j_i}^*}{\|\mathbf{x} - \mathbf{x}_{j_i}^*\|}}{\|\mathbf{x} - \mathbf{x}_{j_i}^*\|^2}\end{aligned}\quad (96)$$

which gives the desired result using (95)

$$(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \alpha_{j_i}(\mathbf{x}) = \frac{(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \nabla \delta_{j_i}(\mathbf{x}) - \delta_{j_i}(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_{j_i}^*\|} < 0 \quad (97)$$

This concludes the proof that $\mathbf{h}_{j_i}^\mathcal{I}$ satisfies Property 1.

Next, we focus on Property 2. Pick a point $\mathbf{x} \in \partial_k \mathcal{F}_{\text{map}, j_i}^\mathcal{I}$. This point could lie:

1. on the outer boundary of $\mathcal{F}_{\text{map}, j_i}^\mathcal{I}$ and away from P_i ;
2. on the boundary of one of the $|\mathcal{J}_C|$ unknown but visible convex obstacles;
3. on the boundary of one of the $(|\mathcal{J}_D^\mathcal{I}| + |\mathcal{J}_B^\mathcal{I}| - 1)$ familiar obstacles that are not P_i ;
4. on the boundary of P_i but not on the boundary of the triangle j_i ;
5. on the boundary of the triangle j_i .

In the first four cases, we have $\mathbf{h}_{j_i}^\mathcal{I}(\mathbf{x}) = \mathbf{x}$, whereas in the last case, we have

$$\mathbf{h}_{j_i}^\mathcal{I}(\mathbf{x}) = \mathbf{x}_{j_i}^* + \frac{(\mathbf{x}_{1j_i} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i}}{(\mathbf{x} - \mathbf{x}_{j_i}^*)^\top \mathbf{n}_{j_i}} (\mathbf{x} - \mathbf{x}_{j_i}^*) \quad (98)$$

It can be verified that $(\mathbf{h}_{j_i}^\mathcal{I}(\mathbf{x}) - \mathbf{x}_{1j_i})^\top \mathbf{n}_{j_i} = 0$, which means that $\mathbf{x}$ is sent to the shared hyperplane between j_i and $p(j_i)$ as desired. This shows that we always have $\mathbf{h}_{j_i}^\mathcal{I}(\mathbf{x}) \in \partial_k \mathcal{F}_{\text{map}, p(j_i)}^\mathcal{I}$ and the map satisfies Property 2.

Finally, Property 3 is derived from above and the fact that each boundary segment $\partial_k \mathcal{F}_{\text{map}, j_i}^\mathcal{I}$ is a 1D manifold, the boundary of either a convex set or a polygon, both of which are homeomorphic to S^1 and, therefore, the corresponding boundary $\partial_k \mathcal{F}_{\text{map}, p(j_i)}^\mathcal{I}$. $\square$

Proof of Lemma 3. The proof follows similar patterns with the proof of Lemma 1. With the procedure outlined in Appendix B, the only points where γ_{r_i} and δ_{r_i} are not smooth are vertices of $\mathcal{Q}_{r_i}$ and $\bar{\mathcal{Q}}_{r_i}$, respectively. Therefore,

with the definition of $\sigma_{\delta_{r_i}}$ as in (20) and the use of the smooth, non-analytic function ζ from (9), we see that $\sigma_{\delta_{r_i}}$ is smooth everywhere, because $\mathbf{x}_i^*$ does not belong in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and δ_{r_i} is exactly 0 on the vertices of $\overline{\mathcal{Q}_{r_i}}$. Therefore, σ_{r_i} can only be non-smooth on the vertices of $\mathcal{Q}_{r_i}$ (i.e., on the vertices of the triangle r_i), and on points where its denominator becomes zero. As both $\sigma_{\gamma_{r_i}}$ and $\sigma_{\delta_{r_i}}$ vary between 0 and 1, this can only happen when $\sigma_{\gamma_{r_i}}(\mathbf{x}) = 1$ and $\sigma_{\delta_{r_i}}(\mathbf{x}) = 0$, which is not allowed by Definition 5, requiring $\mathcal{Q}_{r_i} \subset \overline{\mathcal{Q}_{r_i}}$. The fact that σ_{r_i} is smooth everywhere else derives immediately from the fact that $\sigma_{\delta_{r_i}}$ is a smooth function, and $\sigma_{\gamma_{r_i}}$ is smooth everywhere except for the triangle vertices.

Proof of Proposition 2. The proof follows similar patterns with the proof of Proposition 1. As shown in Lemma 5, the map $\hat{\mathbf{h}}^{\mathcal{I}}$ is smooth in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ away from any sharp corners. Therefore, we need to focus again on the Massey conditions (Massey, 1992) and show that:

1. $\hat{\mathbf{h}}^{\mathcal{I}}$ has a non-singular differential on $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ away from any sharp corners;
2. $\hat{\mathbf{h}}^{\mathcal{I}}$ preserves boundaries, i.e., $\hat{\mathbf{h}}^{\mathcal{I}}(\partial_k \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}) \subset \partial_k \mathcal{F}_{\text{model}}^{\mathcal{I}}$, with k spanning both the indices of familiar obstacles $\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}, \mathcal{J}_{\mathcal{B}}^{\mathcal{I}}$ as well as the indices of unknown obstacles $\mathcal{J}_{\mathcal{C}}^{\mathcal{I}}$;
3. the boundary components of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}}$ are pairwise homeomorphic, i.e., $\partial_k \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}} \cong \partial_k \mathcal{F}_{\text{model}}^{\mathcal{I}}$, with k spanning both the indices of familiar obstacles $\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}, \mathcal{J}_{\mathcal{B}}^{\mathcal{I}}$ as well as the indices of unknown obstacles $\mathcal{J}_{\mathcal{C}}^{\mathcal{I}}$.

We begin with Property 1 and examine the space away from any sharp corners in $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$. By construction of the polygonal collars $\overline{\mathcal{Q}_{r_i}}$ and the definition of $\hat{\mathbf{h}}^{\mathcal{I}}$ in (26), we see that $\hat{\mathbf{h}}^{\mathcal{I}}$ is either the identity map (which implies that $D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}} = \mathbf{I}$), or depends only on a single switch σ_{r_k} . In that case, we can isolate the k th term of the map Jacobian to write

$$\begin{aligned} D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}} &= D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}}|_k = (\nu_{r_k}(\mathbf{x}) - 1)(\mathbf{x} - \mathbf{x}_k^*) \nabla \sigma_{r_k}(\mathbf{x})^{\top} \\ &\quad + \sigma_{r_k}(\mathbf{x})(\mathbf{x} - \mathbf{x}_k^*) \nabla \nu_{r_k}(\mathbf{x})^{\top} \\ &\quad + [1 + \sigma_{r_k}(\mathbf{x})(\nu_{r_k}(\mathbf{x}) - 1)] \mathbf{I} \end{aligned} \quad (99)$$

It is then straightforward to follow exactly the same procedure outlined in the proof of Proposition 1 and show that $\det(D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}}|_k), \text{tr}(D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}}|_k) > 0$ for all $\mathbf{x} \in \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ away from sharp corners.

Next, we focus on Property 2. Pick a point $\mathbf{x} \in \partial_k \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$. This point could lie:

1. on the outer boundary of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$, but not on a root triangle corresponding to an obstacle $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$;
2. on the boundary of one of the $\mathcal{J}_{\mathcal{C}}^{\mathcal{I}}$ unknown but visible convex obstacles;
3. on the outer boundary of $\hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ and on a root triangle corresponding to an obstacle $P_i \in \mathcal{B}_{\text{map}}^{\mathcal{I}}$; or

4. on the boundary of one of the $|\mathcal{J}_{\mathcal{D}}^{\mathcal{I}}|$ root triangles corresponding to obstacles in $\mathcal{D}_{\text{map}}^{\mathcal{I}}$.

In the first two cases, we have $\hat{\mathbf{h}}^{\mathcal{I}}(\mathbf{x}) = \mathbf{x}$, in the third case we have $\hat{\mathbf{h}}^{\mathcal{I}}(\mathbf{x}) \in \partial_0 \mathcal{F}_{\text{model}}^{\mathcal{I}} = \partial \mathcal{F}_e$ by construction of (21) and (23), whereas in the last case

$$\hat{\mathbf{h}}^{\mathcal{I}}(\mathbf{x}) = \mathbf{x}_k^* + \frac{\rho_k}{\|\mathbf{x} - \mathbf{x}_k^*\|} (\mathbf{x} - \mathbf{x}_k^*) \quad (100)$$

for some $k \in \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}$, sending $\mathbf{x}$ to the boundary of the k th disk in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. This shows that we always have $\hat{\mathbf{h}}^{\mathcal{I}}(\mathbf{x}) \in \partial_k \mathcal{F}_{\text{model}}^{\mathcal{I}}$ and the map satisfies Property 2.

Finally, Property 3 derives from above and the fact that each boundary segment $\partial_k \hat{\mathcal{F}}_{\text{map}}^{\mathcal{I}}$ is a 1D manifold, the boundary of either a convex set or a triangle, both of which are homeomorphic to S^1 and, therefore, the corresponding boundary $\partial_k \mathcal{F}_{\text{model}}^{\mathcal{I}}$. $\square$

E.2. Proofs of results in Section 5

Proof of Lemma 6. The proof of this lemma derives immediately from Arslan and Koditschek (2016b: Propositions 5 and 11) and Assumption 4, from which we can infer that the set of stationary points of the vector field $D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}} \cdot \mathbf{u}^{\mathcal{I}}(\mathbf{x})$, defined on $\mathcal{F}_{\text{model}}^{\mathcal{I}}$, is $\{\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)\} \cup \{\mathbf{s}_i\}_{i \in \mathcal{J}_{\mathcal{D}}^{\mathcal{I}}} \cup \{\mathbf{g}_k\}_{k \in \mathcal{J}_{\mathcal{C}}^{\mathcal{I}}}$, with $\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)$ being a locally stable equilibrium of $D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}} \cdot \mathbf{u}^{\mathcal{I}}(\mathbf{x})$ and each other point being a non-degenerate saddle, because Arslan and Koditschek (2016b: Assumption 2) is satisfied for the obstacles in $\mathcal{F}_{\text{model}}^{\mathcal{I}}$. To complete the proof, we just have to note that the index of an isolated zero of a vector field does not change under diffeomorphisms of the domain (Hirsch, 1976). $\square$

Proof of Proposition 4. Consider the smooth Lyapunov function candidate $V^{\mathcal{I}}(\mathbf{x}) = \|\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)\|^2$. Using (38) and writing $\mathbf{y} = \mathbf{h}^{\mathcal{I}}(\mathbf{x})$ and $\mathbf{y}_d = \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)$, we obtain

$$\begin{aligned} \frac{dV^{\mathcal{I}}}{dt} &= 2(\mathbf{y} - \mathbf{y}_d)^{\top} (D_{\mathbf{x}} \hat{\mathbf{h}}^{\mathcal{I}}) \dot{\mathbf{x}} \\ &= -2k(\mathbf{y} - \mathbf{y}_d)^{\top} (\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)) \\ &= -2k(\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d) + \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d) - \mathbf{y}_d)^{\top} \\ &\quad (\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)) \\ &= -2k \|\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)\|^2 \\ &\quad + 2k(\mathbf{y}_d - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d))^{\top} (\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)) \\ &\leq -2k \|\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)\|^2 \leq 0 \end{aligned} \quad (101)$$

because $\mathbf{y} \in \mathcal{L}\mathcal{F}(\mathbf{y})$, which implies that

$$(\mathbf{y}_d - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d))^{\top} (\mathbf{y} - \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)) \leq 0 \quad (102)$$

because either $\mathbf{y}_d = \Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)$, or $\mathbf{y}_d$ and $\mathbf{y}$ are separated by a hyperplane passing through $\Pi_{\mathcal{L}\mathcal{F}(\mathbf{y})}(\mathbf{y}_d)$. Therefore, similarly to Arslan and Koditschek (2016b), using LaSalle's invariance principle we see that every trajectory

starting in $\mathcal{F}_{\text{map}}^{\mathcal{I}}$ approaches the largest invariant set in $\{\mathbf{x} \in \mathcal{F}_{\text{map}}^{\mathcal{I}} | \dot{V}^{\mathcal{I}}(\mathbf{x}) = 0\}$, i.e., the equilibrium points of (38). The desired result follows from Lemma 6, because $\mathbf{x}_d$ is the only locally stable equilibrium of our control law and the rest of the stationary points are non-degenerate saddles, whose regions of attraction have empty interior in $\mathcal{F}_{\text{map}}^{\mathcal{I}}$. $\square$

Proof of Proposition 5. Note that the Jacobian of $\bar{\mathbf{h}}^{\mathcal{I}}$ will be given by

$$D_{\bar{\mathbf{x}}} \bar{\mathbf{h}}^{\mathcal{I}} = \begin{bmatrix} D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}} & \mathbf{0}_{2 \times 1} \\ D_{\mathbf{x}} \xi^{\mathcal{I}} & \frac{\partial \xi^{\mathcal{I}}}{\partial \psi} \end{bmatrix} \quad (103)$$

As we already have from Corollary 3 that $D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}$ is non-singular, it suffices to show that $\frac{\partial \xi^{\mathcal{I}}}{\partial \psi} \neq 0$ for all $\bar{\mathbf{x}} \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$. From (42) we can derive

$$\frac{\partial \xi^{\mathcal{I}}}{\partial \psi} = \frac{\det(D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}})}{\|\mathbf{e}(\bar{\mathbf{x}})\|^2} \quad (104)$$

Therefore, we immediately get that $\frac{\partial \xi^{\mathcal{I}}}{\partial \psi} \neq 0$ for all $\bar{\mathbf{x}} \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ because $\det(D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}) \neq 0$ and $\|\mathbf{e}(\bar{\mathbf{x}})\| \neq 0$ for all $\bar{\mathbf{x}} \in \mathcal{F}_{\text{map}}^{\mathcal{I}}$, because $D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}$ is non-singular on $\mathcal{F}_{\text{map}}^{\mathcal{I}}$. This implies that $D_{\bar{\mathbf{x}}} \bar{\mathbf{h}}^{\mathcal{I}}$ is non-singular on $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$. $\square$

Next, we note that $\partial(\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1) = \partial \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$, because S^1 is a manifold without boundary. Similarly, $\partial(\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1) = \partial \mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$. Hence, we can easily complete the proof following a similar procedure with the end of the proofs of Propositions 1 and 2 to show that $\bar{\mathbf{h}}^{\mathcal{I}}$ preserves boundaries, and the boundaries of $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ and $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ are pairwise homeomorphic.

Proof of Theorem 2. We have already established that $\|\mathbf{e}(\bar{\mathbf{x}})\|$ and $\frac{\partial \xi^{\mathcal{I}}}{\partial \psi}$ are non-zero for all $\bar{\mathbf{x}} \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ in the proof of Proposition 5, which implies that v and ω can have no singular points. In addition, note that $\|\mathbf{e}(\bar{\mathbf{x}})\|$, $\frac{\partial \xi^{\mathcal{I}}}{\partial \psi}$ and $D_{\mathbf{x}} \xi^{\mathcal{I}} [\cos \psi \quad \sin \psi]^{\top}$ are all smooth away from corners in $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$. Hence, the uniqueness and existence of the flow generated by control law (52) can be established similarly to Arslan and Koditschek (2016b) through the flow properties of the controller in Astolfi (1999) (that we use here in (53)) and the facts that metric projections onto moving convex cells are piecewise continuously differentiable (Kuntz and Scholtes, 1994; Shapiro, 1988), and the composition of piecewise continuously differentiable functions is piecewise continuously differentiable and, therefore, locally Lipschitz (Chaney, 1990).

Next, as shown in (43), the vector field $\mathbf{B}(\psi) \bar{\mathbf{u}}^{\mathcal{I}}$ on $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ is the pullback of the complete vector field $\mathbf{B}(\varphi) \bar{\mathbf{v}}^{\mathcal{I}}$, guaranteed to retain $\mathcal{F}_{\text{model}}^{\mathcal{I}} \times S^1$ positively invariant under its flow as shown in Arslan and Koditschek (2016b), under the smooth change of coordinates $\bar{\mathbf{h}}^{\mathcal{I}}$ away from sharp corners in $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$. This shows that the free-space $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ is positively invariant under law (52).

Finally, consider the smooth Lyapunov function candidate $V^{\mathcal{I}}(\mathbf{x}) = \|\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)\|^2$. Then, by writing $\mathbf{y} = \mathbf{h}^{\mathcal{I}}(\mathbf{x})$ and $\mathbf{y}_d = \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)$, we obtain

$$\begin{aligned} \frac{dV^{\mathcal{I}}}{dt} &= 2(\mathbf{y} - \mathbf{y}_d)^{\top} (D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}) \dot{\mathbf{x}} \\ &= 2v^{\mathcal{I}} (\mathbf{y} - \mathbf{y}_d)^{\top} (D_{\mathbf{x}} \mathbf{h}^{\mathcal{I}}) \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix} \\ &= 2\hat{v}^{\mathcal{I}} (\mathbf{y} - \mathbf{y}_d)^{\top} \begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix} \\ &= -2k_v (\mathbf{y} - \mathbf{x}_d)^{\top} \begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix} \begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix}^{\top} \\ &\quad (\mathbf{y} - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d)) \\ &= -2k_v (\mathbf{y} - \mathbf{y}_d)^{\top} (\mathbf{y} - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d)) \end{aligned}$$

because $\begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix} \begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix}^{\top}$ is just the projection operator on the line defined by the vector $\begin{bmatrix} \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) & \sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix}^{\top}$, with which $(\mathbf{y} - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d))$ is already parallel. Following this result, we obtain

$$\frac{dV^{\mathcal{I}}}{dt} \leq -2k_v \|\mathbf{h}^{\mathcal{I}}(\mathbf{y}) - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d)\|^2 \leq 0 \quad (105)$$

because, similarly to the proof of Proposition 4, we have

$$(\mathbf{y}_d - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d))^{\top} (\mathbf{y} - \Pi_{\mathcal{LF}(\mathbf{y}) \cap H_{\parallel}}(\mathbf{y}_d)) \leq 0 \quad (106)$$

Therefore, using LaSalle's invariance principle, we see that every trajectory starting in $\mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ approaches the largest invariant set in $\{(\mathbf{x}, \psi) \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1 | \dot{V}^{\mathcal{I}}(\mathbf{x}) = 0\} = \{(\mathbf{x}, \psi) \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1 | \mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \Pi_{\mathcal{LF}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}) \cap H_{\parallel}}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d))\}$. At the same time, we know from (53) that $\mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \Pi_{\mathcal{LF}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}) \cap H_{\parallel}}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d))$ implies $v^{\mathcal{I}} = 0$. From (52), for $v^{\mathcal{I}} = 0$, we get that $\omega^{\mathcal{I}}$ will be zero at points where $\hat{\omega}^{\mathcal{I}}$ is zero, i.e., at points $(\mathbf{x}, \psi) \in \mathcal{F}_{\text{map}}^{\mathcal{I}} \times S^1$ where

$$\begin{bmatrix} -\sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix}^{\top} (\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{y}_{d,G}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}), \xi^{\mathcal{I}}(\psi))) = 0 \quad (107)$$

with the angular local goal $\mathbf{y}_{d,G}$ defined as in (55). Therefore, the largest invariant set in $\{(\mathbf{x}, \psi) | \mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \Pi_{\mathcal{LF}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}) \cap H_{\parallel}}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d))\}$ is the set of points $\bar{\mathbf{x}} = (\mathbf{x}, \psi)$ where the following two conditions are satisfied

$$\mathbf{h}^{\mathcal{I}}(\mathbf{x}) = \Pi_{\mathcal{LF}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}) \cap H_{\parallel}}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)) \quad (108)$$

$$\times \begin{bmatrix} -\sin \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \\ \cos \xi^{\mathcal{I}}(\bar{\mathbf{x}}) \end{bmatrix}^{\top} (\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{y}_{d,G}(\mathbf{h}^{\mathcal{I}}(\mathbf{x}), \xi^{\mathcal{I}}(\psi))) = 0 \quad (109)$$

Using a similar argument to Arslan and Koditschek (2016b: Proposition 12), we can, therefore, verify that the set of stationary points of law (52) is given by

$$\begin{aligned} & \{\mathbf{x}_d\} \times (-\pi, \pi] \\ & \bigcup \left\{ (\mathbf{q}, \psi) \mid \mathbf{q} \in \{(\mathbf{h}^{\mathcal{I}})^{-1}(\mathbf{s}_i)\}_{i \in \mathcal{J}_d^{\mathcal{I}}} \bigcup_{k \in \mathcal{J}_c} \mathcal{G}_k, \right. \\ & \left. \times \begin{bmatrix} -\sin \xi^{\mathcal{I}}(\mathbf{q}, \psi) \\ \cos \xi^{\mathcal{I}}(\mathbf{q}, \psi) \end{bmatrix}^{\top} (\mathbf{q} - \mathbf{x}_d) = 0 \right\} \end{aligned} \quad (110)$$

using (39). We can then invoke a similar argument to Proposition 4 to show that $\mathbf{x}_d$ locally attracts with any orientation ψ , whereas any configuration associated with any other equilibrium point is a non-degenerate saddle whose stable manifold is a set of measure zero, and the result follows. $\square$

Proof of Lemma 7. We can show this by contradiction. Assume that the robot is in mode $\mathcal{I}$ and two guards $G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_1}$ and $G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_2}$, indexed by two different subsets $\mathcal{I}_1 \neq \mathcal{I}_2$, each playing the role of $\mathcal{I}_u$ in (29), nevertheless overlap, $G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_1} \cap G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_2} \neq \emptyset$. That means that there exists at least one state $\mathbf{x} \in \mathcal{F}^{\mathcal{I}}$, such that $\mathbf{x} \in G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_1}$ and $\mathbf{x} \in G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_2}$, for two non-empty sets $\mathcal{I}_1, \mathcal{I}_2$ with $\mathcal{I} \cap \mathcal{I}_1 = \emptyset$, $\mathcal{I} \cap \mathcal{I}_2 = \emptyset$. As $\mathcal{I}_1 \neq \mathcal{I}_2$ by assumption, this implies that there exists at least one index i that is contained in one of these index sets, but is not contained in the other. Without loss of generality, assume that $i \in \mathcal{I}_1$ and $i \notin \mathcal{I}_2$. We immediately arrive at a contradiction, because the requirement $\mathbf{x} \in G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_2}$ requires $\overline{B(\mathbf{x}, R)} \cap \tilde{\mathcal{P}}_{\mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_2)} = \emptyset$, but we know that the requirement $\mathbf{x} \in G^{\mathcal{I}, \mathcal{I} \cup \mathcal{I}_1}$ implies $\overline{B(\mathbf{x}, R)} \cap \tilde{P}_i \neq \emptyset$ with $\tilde{P}_i \in \tilde{\mathcal{P}}_{\mathcal{N}_p \setminus (\mathcal{I} \cup \mathcal{I}_2)}$. $\square$

Proof of Lemma 8. If the system is in the terminal mode $\mathcal{I} = \mathcal{N}_p$, according to Definition 8, then finite time escape through the boundary of the hybrid domain is not possible, because the vector field $\mathbf{u}^{\mathcal{I}}$ leaves its domain positively invariant under its flow, as described in Theorem 1. For $\mathcal{I} \neq \mathcal{N}_p$, the only way in which the flow can escape is through the boundary of an obstacle $\tilde{P} \notin \{\tilde{P}_i\}_{i \in \mathcal{I}}$, because $\mathbf{u}^{\mathcal{I}}$ guarantees safety only against familiar obstacles in $\mathcal{I}$ and any unknown obstacles encountered along the way. We are going to show that this cannot happen by contradiction. Assume that at time t_0 the robot is at $\mathbf{x}_0 \in \mathcal{F}^{\mathcal{I}}$, and at time $t_1 > t_0$ it crosses the boundary of an obstacle $\tilde{P} \notin \{\tilde{P}_i\}_{i \in \mathcal{I}}$. This means that the robot travels distance $d > 0$ between t_0 and t_1 in mode $\mathcal{I}$, without triggering a transition to another hybrid mode $\mathcal{I}'$ that includes $\tilde{P}$ (and, therefore, guarantees safety against it by Theorem 1), which is impossible

because the sensor footprint has a positive radius R and $\overline{B(\mathbf{x}', R)}$ would have hit $\tilde{P}_i$ at some time $t < t_1$ before colliding with it.

Moreover, the restriction of the reset map in each separate mode is just the identity transform, which, by the argument made previously, implies that the discrete transition itself is never blocking, assuming that the initial condition lies in the freespace $\mathcal{F}$. This is because $\mathcal{F} \subseteq \mathcal{F}^{\mathcal{I}}$ for all modes $\mathcal{I} \in 2^{\mathcal{N}_p}$.

Finally, hybrid ambiguity is avoided by the construction of the guard in (29); if the robot at a position $\mathbf{x}^-$ in the interior of the domain is in mode $\mathcal{I}$ at time t^- before a discrete transition and in mode $\mathcal{I}'$ at time t^+ after the transition, we are guaranteed that the sensor footprint $\overline{B(\mathbf{x}^+, R)}$ after the transition does not intersect any obstacle $\tilde{P}_i$ with $i \notin \mathcal{I}'$. This implies that $\mathbf{x}^+$ lies in the interior of the domain and away from the guard, and the application of the reset map provides the unique extension to the execution.

Proof of Theorem 3 As stated in Section 5.1, the hybrid system described in the tuple $\mathbf{H}$ has $2^{|\mathcal{N}_p|}$ modes. Unique piecewise continuous differentiability of the flow derives immediately by the unique continuous differentiability of the control law $U^{\mathcal{I}}$, defined in (38) for each separate mode $\mathcal{I}$, as summarized in Theorem 1. Moreover, positive invariance derives from the first part of the proof of Lemma 8, which guarantees that the hybrid flow cannot escape from the hybrid domain through a point on the boundary of the domain in the complement of the guard, or the guard itself.

For stability, we note that each mode (indexed by $\mathcal{I} \in 2^{\mathcal{N}_p}$) is associated with a candidate Lyapunov function $V^{\mathcal{I}}(\mathbf{x}) = \|\mathbf{h}^{\mathcal{I}}(\mathbf{x}) - \mathbf{h}^{\mathcal{I}}(\mathbf{x}_d)\|^2$, as shown in the proof of Proposition 4. Moreover, also by the results of Proposition 4, $\mathbf{x}_d$ is the unique asymptotically stable equilibrium of each control vector field $U^{\mathcal{I}}$, thus, almost every execution that remains in mode $\mathcal{I}$ for all future time has a trajectory that asymptotically approaches the goal. Then, the key for the proof is the observation that once the robot exits a mode defined by $\mathcal{I}$, it can never re-enter it. This is because the robot stores information in its semantic map and this knowledge can only be incremental; in the worst case, the robot will explore all familiar obstacles in the environment, and stay in mode $\mathcal{I} = \mathcal{N}_p$ for all following time.

Based on this observation, we note that the collection of functions $\{V^{\mathcal{I}} \mid \mathcal{I} \in 2^{\mathcal{N}_p}\}$ are *Lyapunov-like*, in the sense of Branicky (1998: Definition 2.2), for all time their corresponding mode is active, because they never reset. We complete the proof by invoking Branicky (1998: Theorem 2.3), which states that if a collection of Lyapunov-like functions for a hybrid system are associated with corresponding vector fields that share the same equilibrium, then the hybrid system itself is Lyapunov stable around this equilibrium.